Complete Kubernetes Learning Path: Basic to Advanced

I'll create a comprehensive, in-depth learning guide covering all these topics with internal workflows.

Phase 1: Kubernetes Fundamentals & Architecture

1.1 Kubernetes Fundamentals

What is Kubernetes?
- **Definition**: Open-source container orchestration platform
- **Purpose**: Automates deployment, scaling, and management of containerized applications
- **Origin**: Built by Google, now maintained by CNCF

Core Problems Kubernetes Solves:
1. **Container orchestration** across multiple hosts
2. **Auto-scaling** based on demand
3. **Self-healing** - automatically replaces failed containers
4. **Service discovery** and load balancing
5. **Automated rollouts** and rollbacks
6. **Secret and configuration** management

1.2 Kubernetes Architecture Deep Dive

Cluster Architecture Overview

```
        ■■■
```

```
■                       ■                           ■
■ ■■■■■■■■■■■■■■■■■■■■■■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■ ■
■ ■   API Server     ■ ■ ■ Kubelet          ■   ■
■ ■   (kube-api)     ■ ■ ■                  ■   ■
■ ■■■■■■■■■■■■■■■■■■■■■■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■ ■
■ ■■■■■■■■■■■■■■■■■■■■■■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■ ■
■ ■   etcd           ■ ■ ■ Kube-proxy       ■   ■
■ ■   (Key-Value DB) ■ ■ ■                  ■   ■
■ ■■■■■■■■■■■■■■■■■■■■■■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■ ■
■ ■■■■■■■■■■■■■■■■■■■■■■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■ ■
■ ■   Scheduler      ■ ■ ■ Container        ■   ■
■ ■                  ■ ■ ■ Runtime          ■   ■
■ ■■■■■■■■■■■■■■■■■■■■■■ ■ ■   (containerd)     ■   ■
■ ■■■■■■■■■■■■■■■■■■■■■■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■ ■
■ ■   Controller     ■ ■                          ■
■ ■   Manager        ■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■ ■
■ ■■■■■■■■■■■■■■■■■■■■■■ ■ ■   Pods            ■   ■
■ ■■■■■■■■■■■■■■■■■■■■■■ ■ ■   (Containers)     ■   ■
■ ■   Cloud          ■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■ ■
■ ■   Controller     ■ ■                          ■
■ ■   Manager        ■ ■                          ■
■ ■■■■■■■■■■■■■■■■■■■■■■ ■                          ■
■■■■■■■■■■■■■■■■■■■
```

Internal Workflow:

```
        Client Request → Authentication → Authorization → Admission Control → Validation → etcd
```

Detailed Process:

1. **Receives REST requests** (kubectl, UI, API)
2. **Authentication**: Validates who you are
- Certificate-based
- Token-based
- Service Account tokens
3. **Authorization**: Validates what you can do
- RBAC (Role-Based Access Control)
- ABAC (Attribute-Based Access Control)
4. **Admission Controllers**: Modify/validate requests
- MutatingAdmissionWebhook
- ValidatingAdmissionWebhook
- PodSecurityPolicy
5. **Writes to etcd**: Persists the desired state
6. **Returns response** to client

Example Flow - Creating a Pod:

```
        kubectl create pod → API Server → Authenticate → Authorize →
        Validate YAML → Admission Controllers → Write to etcd →
        Scheduler notified → Node assigned → Kubelet creates container
```

---

2. etcd

Purpose: Distributed key-value store for cluster state

What it Stores:

```
/registry/pods/default/nginx-pod
/registry/deployments/default/nginx-deployment
/registry/services/default/nginx-service
/registry/secrets/default/my-secret
/registry/configmaps/default/my-config
```

Internal Structure:

- Uses **Raft consensus algorithm** for distributed consistency
- **Leader election** for high availability
- **Write-ahead logging** for durability
- **Snapshot and compaction** for efficiency

Data Flow:

```
        API Server writes → etcd Leader → Replicates to followers →
        Acknowledged → API Server confirms
```

Critical Operations:

```bash
# Backup etcd
ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-snapshot.db \
--endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key

# Restore etcd
ETCDCTL_API=3 etcdctl snapshot restore /backup/etcd-snapshot.db \
--data-dir=/var/lib/etcd-restore
```

```

---

##### **3. Scheduler (kube-scheduler)**

**Purpose**: Assigns pods to nodes based on resource requirements

**Scheduling Workflow**:
```
        Watch for unscheduled pods → Filtering (Predicate) →
        Scoring (Priority) → Select best node → Bind pod to node
```

**Detailed Process**:

**Phase 1: Filtering (Predicates)**
```
        Available Nodes → Filter nodes that satisfy:
        ■■■ Sufficient CPU/Memory
        ■■■ Node Selector matches
        ■■■ Taints/Tolerations compatible
        ■■■ Pod affinity/anti-affinity rules
        ■■■ Volume availability
        ■■■ Port availability
```

**Phase 2: Scoring (Priorities)**
```
        Filtered Nodes → Score based on:
        ■■■ Resource balance (CPU/Memory spread)
        ■■■ Node affinity preferences
        ■■■ Image locality (is image already on node?)
        ■■■ Inter-pod affinity
```
```

```
        ■■■ Custom priorities
```

Example:

```yaml
# Pod with scheduling constraints
apiVersion: v1
kind: Pod
metadata:
name: nginx
spec:
nodeSelector:
disktype: ssd
affinity:
nodeAffinity:
requiredDuringSchedulingIgnoredDuringExecution:
nodeSelectorTerms:
- matchExpressions:
- key: kubernetes.io/zone
operator: In
values:
- us-east-1a
containers:
- name: nginx
image: nginx
resources:
requests:
memory: "64Mi"
cpu: "250m"
limits:
memory: "128Mi"
cpu: "500m"
```

```

**Scheduling Decision Flow**:
```
1. Scheduler watches API server for pods with nodeName=""
2. For each pod:
a. Get all nodes
b. Filter: Remove nodes that don't meet requirements
- Node1: Has 500m CPU available ✓
- Node2: Only 100m CPU available ✗
- Node3: Has taint=NoSchedule ✗
c. Score remaining nodes:
- Node1: Score 85 (better resource balance)
- Node4: Score 72 (less balanced)
d. Select highest score (Node1)
e. Bind pod to Node1 (update pod.spec.nodeName in etcd)
3. Kubelet on Node1 sees the binding and starts the pod
```

---

##### **4. Controller Manager (kube-controller-manager)**

**Purpose**: Runs controller loops that regulate cluster state

**Key Controllers**:

**a) Node Controller**
```
Responsibility: Monitor node health
Loop Interval: Every 5 seconds

Workflow:
1. Watch node heartbeats
```
```

2. If node doesn't respond:

```
        - Wait 40 seconds → Mark as Unknown
        - Wait 5 minutes → Evict pods from node
```

3. Update node status in etcd

```

**b) Replication Controller**

```
Responsibility: Maintain desired replica count

Workflow:
1. Watch ReplicaSet/Deployment in etcd
2. Compare desired vs actual pod count
3. If actual < desired:
- Create new pods
4. If actual > desired:
- Delete excess pods
5. Reconcile every 15 seconds
```

**c) Endpoints Controller**

```
Responsibility: Populate Endpoints object for Services

Workflow:
1. Watch Services and Pods
2. When Service is created:
- Find pods matching selector
- Create Endpoints object with pod IPs
3. Update Endpoints when pods change
```

d) Service Account Controller
```

```
Responsibility: Create default ServiceAccounts

Workflow:
1. Watch for new namespaces
2. Create default ServiceAccount in each namespace
3. Generate and mount tokens for pods
```

Reconciliation Loop Pattern:
```go
// Pseudo-code for controller loop
for {
desiredState := getDesiredStateFromEtcd()
currentState := getCurrentStateFromCluster()

if desiredState != currentState {
reconcile(desiredState, currentState)
}

sleep(reconciliationInterval)
}
```

---

5. Cloud Controller Manager (CCM)

Purpose: Integrates with cloud provider APIs

Controllers:
```
1. Node Controller: Labels nodes with cloud-specific info
2. Route Controller: Configures cloud network routes
```

3. Service Controller: Creates cloud load balancers

4. Volume Controller: Creates cloud storage volumes

```

**Example - LoadBalancer Service Flow**:

```

1. User creates Service type=LoadBalancer

2. Service Controller detects new service

3. Calls AWS/GCP/Azure API to create LB

4. Updates Service with external IP

5. Configures LB to route to node ports

```

---

#### **Worker Node Components**

##### **1. Kubelet**

**Purpose**: Agent that runs on each node and manages pods

**Internal Workflow**:

```

```
Watch API Server → Pod assigned to node →

Pull image → Create container → Monitor health →

Report status → Handle pod lifecycle
```

```

**Detailed Process**:

**Pod Lifecycle Management**:

```

1. Pod Assigned to Node (via scheduler)

↓

2. Kubelet detects new pod assignment

↓

3. Check if image exists locally

↓

4. If not, pull image from registry

↓

5. Create pod sandbox (network namespace)

↓

6. Start init containers (if defined)

↓

7. Wait for init containers to complete

↓

8. Start main containers

↓

9. Configure networking (CNI plugin)

↓

10. Mount volumes

↓

11. Inject ConfigMaps/Secrets

↓

12. Execute startupProbe

↓

13. Execute livenessProbe (continuous)

↓

14. Execute readinessProbe (continuous)

↓

15. Report pod status to API server

```

**Container Runtime Interface (CRI)**:

```

Kubelet → CRI → containerd/CRI-O → Container

```

**Example kubelet operations**:

```bash
# Kubelet creates container via CRI
        kubelet → CRI.CreateContainer() → containerd → runc → Container process

# Kubelet monitors container
        kubelet → CRI.ContainerStatus() → Check if running
```

Pod Status Reporting:

```yaml
# Kubelet updates pod status in API server
status:
phase: Running
conditions:
- type: Initialized
status: "True"
- type: Ready
status: "True"
- type: ContainersReady
status: "True"
- type: PodScheduled
status: "True"
containerStatuses:
- name: nginx
ready: true
restartCount: 0
state:
running:
startedAt: "2024-01-15T10:30:00Z"
```
```

---

2. Kube-proxy

Purpose: Network proxy that maintains network rules

Modes of Operation:

a) iptables Mode (default)

```
        Service created → kube-proxy watches →
        Creates iptables rules → Traffic redirected to pods
```

Internal Flow:

```
1. Service "nginx-service" created with ClusterIP 10.96.0.10
2. kube-proxy on each node creates iptables rules:

# Redirect traffic to service IP
-A KUBE-SERVICES -d 10.96.0.10/32 -p tcp -m tcp --dport 80 \
-j KUBE-SVC-NGINX

# Load balance to pod endpoints
-A KUBE-SVC-NGINX -m statistic --mode random --probability 0.33 \
-j KUBE-SEP-POD1
-A KUBE-SVC-NGINX -m statistic --mode random --probability 0.50 \
-j KUBE-SEP-POD2
-A KUBE-SVC-NGINX -j KUBE-SEP-POD3

# Forward to actual pod IPs
-A KUBE-SEP-POD1 -p tcp -m tcp -j DNAT --to-destination 10.244.1.5:80
-A KUBE-SEP-POD2 -p tcp -m tcp -j DNAT --to-destination 10.244.2.8:80
-A KUBE-SEP-POD3 -p tcp -m tcp -j DNAT --to-destination 10.244.3.2:80
```

```

**b) IPVS Mode** (better performance)
```
Uses Linux IPVS (IP Virtual Server) for load balancing
Better performance for large clusters (1000+ services)
```

**Traffic Flow Example**:
```
        Client Pod (10.244.1.10) → Service IP (10.96.0.10:80) →
        iptables/IPVS rules → Random pod (10.244.2.8:80)
```

---

##### **3. Container Runtime**

**Purpose**: Runs containers

**Evolution**:
```
        Docker → dockershim (deprecated) → containerd/CRI-O
```

**containerd Architecture**:
```
        Kubelet → CRI plugin → containerd → runc → Container

Components:
1. CRI plugin: Implements Kubernetes CRI
2. containerd: Manages container lifecycle
3. runc: Low-level container runtime (OCI compliant)
```
```

Container Creation Flow:
```
1. Kubelet calls CRI.CreateContainer()
2. containerd pulls image (if needed)
3. containerd creates container config
4. containerd calls runc to create container
5. runc sets up namespaces, cgroups
6. runc starts container process
7. containerd monitors container
8. Kubelet monitors via containerd
```

Phase 2: Cluster Setup with kubeadm

2.1 Prerequisites

System Requirements:
```
- 2 GB RAM minimum
- 2 CPUs minimum
- Unique hostname, MAC address, product_uuid
- Certain ports open (6443, 2379-2380, 10250, 10251, 10252)
- Swap disabled
- Container runtime installed (containerd)
```

2.2 Cluster Setup - Step by Step

Step 1: Prepare All Nodes (Master + Workers)

```bash
# 1. Disable swap (Kubernetes requirement)
sudo swapoff -a
sudo sed -i '/ swap / s/^/#/' /etc/fstab

# 2. Load kernel modules
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF

sudo modprobe overlay
sudo modprobe br_netfilter

# 3. Configure sysctl parameters
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF

sudo sysctl --system

# 4. Install containerd
sudo apt-get update
sudo apt-get install -y containerd

# Configure containerd
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml

# Enable SystemdCgroup (required for kubeadm)
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
```

```
sudo systemctl restart containerd
sudo systemctl enable containerd

# 5. Install kubeadm, kubelet, kubectl
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl

curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | \
sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] \
https://pkgs.k8s.io/core:/stable:/v1.29/deb/ /' | \
sudo tee /etc/apt/sources.list.d/kubernetes.list

sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
```

Step 2: Initialize Control Plane (Master Node)

```bash
# Initialize cluster
sudo kubeadm init \
--pod-network-cidr=10.244.0.0/16 \
--apiserver-advertise-address=192.168.1.100 \
--control-plane-endpoint=k8s-master:6443

# What happens during init:
# 1. Preflight checks (ports, swap, etc.)
# 2. Generate certificates (CA, apiserver, kubelet, etc.)
# 3. Generate kubeconfig files
```

```
# 4. Generate static pod manifests for control plane
# 5. Wait for control plane to be ready
# 6. Upload configuration to ConfigMap
# 7. Mark master as control-plane
# 8. Install CoreDNS
# 9. Install kube-proxy
```

Internal Init Process:

```
kubeadm init
        ↓
[preflight] Checks
        ↓
[certs] Generate certificates
- /etc/kubernetes/pki/ca.crt
- /etc/kubernetes/pki/apiserver.crt
- /etc/kubernetes/pki/apiserver-kubelet-client.crt
- /etc/kubernetes/pki/front-proxy-ca.crt
- /etc/kubernetes/pki/etcd/ca.crt
        ↓
[kubeconfig] Generate kubeconfig files
- /etc/kubernetes/admin.conf
- /etc/kubernetes/kubelet.conf
- /etc/kubernetes/controller-manager.conf
- /etc/kubernetes/scheduler.conf
        ↓
[control-plane] Create static pod manifests
- /etc/kubernetes/manifests/kube-apiserver.yaml
- /etc/kubernetes/manifests/kube-controller-manager.yaml
- /etc/kubernetes/manifests/kube-scheduler.yaml
- /etc/kubernetes/manifests/etcd.yaml
```

```
        ↓
[kubelet-start] Start kubelet service
        ↓
[wait-control-plane] Wait for pods to start
        ↓
[upload-config] Upload cluster configuration
        ↓
[mark-control-plane] Add labels and taints
        ↓
[bootstrap-token] Create join token
        ↓
[addons] Install CoreDNS and kube-proxy
```

Generated Certificates Structure:

```
/etc/kubernetes/pki/
        ■■■ ca.crt                          # Cluster CA
        ■■■ ca.key
        ■■■ apiserver.crt                   # API Server cert
        ■■■ apiserver.key
        ■■■ apiserver-kubelet-client.crt    # API Server to Kubelet
        ■■■ apiserver-kubelet-client.key
        ■■■ front-proxy-ca.crt              # Front proxy CA
        ■■■ front-proxy-ca.key
        ■■■ front-proxy-client.crt
        ■■■ front-proxy-client.key
        ■■■ sa.key                          # Service Account signing key
        ■■■ sa.pub
        ■■■ etcd/
        ■■■ ca.crt                      # etcd CA
        ■■■ ca.key
```

```
        ■■■ server.crt
        ■■■ server.key
        ■■■ peer.crt
        ■■■ peer.key
```

```bash
# Configure kubectl for current user
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

# Verify control plane
kubectl get nodes
kubectl get pods -n kube-system
```

Step 3: Install CNI Plugin (Networking)

Why CNI is needed:
```
Without CNI: Pods cannot communicate across nodes
With CNI: Creates overlay network for pod-to-pod communication
```

Calico Installation (example):
```bash
kubectl apply -f https://docs.projectcalico.org/manifests/calico.yaml
```

What Calico does:
```
```

1. Creates network interfaces on each node
2. Assigns IP addresses to pods from CIDR range
3. Sets up routing rules (BGP or VXLAN)
4. Implements Network Policies
```

**Pod Networking Flow with Calico**:
```
        Pod A (10.244.1.5) on Node1 → Pod B (10.244.2.8) on Node2

1. Pod A sends packet to 10.244.2.8
2. Packet goes to Node1's veth interface
3. Calico routing: Packet needs Node2
4. Encapsulation (VXLAN) or BGP routing
5. Packet sent to Node2
6. Node2 decapsulates packet
7. Routed to Pod B's veth interface
8. Pod B receives packet
```

---

#### **Step 4: Join Worker Nodes**

```bash
# On worker nodes, run the join command from master init output:
sudo kubeadm join k8s-master:6443 \
--token abcdef.0123456789abcdef \
--discovery-token-ca-cert-hash sha256:hash_value

# What happens during join:
# 1. Download cluster info using bootstrap token
# 2. Verify CA certificate hash
# 3. Generate kubelet.conf

```
# 4. Start kubelet
# 5. Request certificate from API server
# 6. Node appears in cluster
```

Internal Join Process:

```
kubeadm join
        ↓
[preflight] Checks
        ↓
[discovery] Download cluster info
- Connect to API server with bootstrap token
- Download cluster-info ConfigMap
- Verify CA certificate hash
        ↓
[kubelet-start] Generate kubelet config
- Create /etc/kubernetes/kubelet.conf
- Create /var/lib/kubelet/config.yaml
        ↓
[kubelet-start] Start kubelet
        ↓
[certificate-request] Request node certificate
- Kubelet sends CSR to API server
- Controller manager approves CSR
- Kubelet receives signed certificate
        ↓
[node-registration] Node registered in cluster
```

Verify cluster:

```bash
```

```bash
kubectl get nodes
# Output:
# NAME STATUS ROLES AGE VERSION
# k8s-master Ready control-plane 10m v1.29.0
# k8s-worker1 Ready <none> 5m v1.29.0
# k8s-worker2 Ready <none> 5m v1.29.0
```

2.3 Cluster Components Verification

```bash
# Check all system pods
kubectl get pods -n kube-system

# Expected pods:
# - coredns-* (DNS)
# - etcd-master (Data store)
# - kube-apiserver-master (API)
# - kube-controller-manager (Controllers)
# - kube-scheduler-master (Scheduler)
# - kube-proxy-* (Network proxy on each node)
# - calico-node-* (CNI on each node)

# Check component health
kubectl get cs # deprecated but useful
kubectl get --raw='/readyz?verbose'

# Check node details
kubectl describe node k8s-master
```

2.4 High Availability (HA) Setup

HA Architecture:

```
        ■■■
```

```
mode tcp
balance roundrobin
option tcp-check
server master1 192.168.1.101:6443 check
server master2 192.168.1.102:6443 check
server master3 192.168.1.103:6443 check

# 2. Initialize first master
kubeadm init \
--control-plane-endpoint="loadbalancer.example.com:6443" \
--upload-certs \
--pod-network-cidr=10.244.0.0/16

# 3. Join additional masters
kubeadm join loadbalancer.example.com:6443 \
--token <token> \
--discovery-token-ca-cert-hash sha256:<hash> \
--control-plane \
--certificate-key <cert-key>
```

Phase 3: Kubernetes Objects & Workloads

3.1 Pods - The Smallest Deployable Unit

What is a Pod?

```
Pod = Wrapper around one or more containers
- Shares network namespace (same IP, ports)
- Shares storage volumes
- Co-located, co-scheduled
- Atomic unit of deployment
```

```

**Pod Architecture**:
```
        ■■■

3. Scheduler assigns pod to node

4. Kubelet on node pulls image and starts container

```

**Declarative Approach** (recommended):

```yaml

# pod.yaml

apiVersion: v1

kind: Pod

metadata:

name: nginx-pod

labels:

app: nginx

environment: production

annotations:

description: "Nginx web server pod"

spec:

containers:

- name: nginx

image: nginx:1.25

ports:

- containerPort: 80

protocol: TCP

resources:

requests:

memory: "64Mi"

cpu: "250m"

limits:

memory: "128Mi"

cpu: "500m"

```
```

```bash
# Apply the configuration
kubectl apply -f pod.yaml

# Difference between create and apply:
# - create: Fails if resource exists
# - apply: Creates or updates (declarative management)
```

Pod Lifecycle States

```
Pod Lifecycle:

        Pending → ContainerCreating → Running → Succeeded/Failed → Terminated

Detailed States:
1. Pending: Waiting for scheduler
2. ContainerCreating: Image pulling, container starting
3. Running: At least one container running
4. Succeeded: All containers terminated successfully (exit 0)
5. Failed: All containers terminated, at least one failed
6. Unknown: State cannot be determined
7. CrashLoopBackOff: Container keeps crashing
```

Pod Phase Transitions:
```yaml
# View pod status
kubectl get pod nginx-pod -o yaml

status:
phase: Running
```

conditions:

- type: Initialized # Init containers completed

status: "True"

- type: Ready # Pod can serve traffic

status: "True"

- type: ContainersReady # All containers ready

status: "True"

- type: PodScheduled # Pod assigned to node

status: "True"

containerStatuses:

- name: nginx

state:

running:

startedAt: "2024-01-15T10:30:00Z"

ready: true

restartCount: 0

image: nginx:1.25

imageID: docker-pullable://nginx@sha256:...

```

---

#### **Pod Configuration Deep Dive**

##### **1. Commands and Arguments**

```yaml

apiVersion: v1

kind: Pod

metadata:

name: command-demo

spec:

containers:

- name: ubuntu

image: ubuntu:22.04

# Override ENTRYPOINT

command: ["/bin/sh"]

# Override CMD

args: ["-c", "while true; do echo hello; sleep 10; done"]

```

Docker vs Kubernetes:

```

Docker Kubernetes

```
ENTRYPOINT →   command

CMD   →   args
```

```

**Internal Execution**:

```

1. Kubelet receives pod spec

2. Calls CRI: CreateContainer with command/args

3. containerd creates container with:

- Entrypoint: /bin/sh

- Cmd: -c "while true..."

4. Container runs the command

```

---

##### **2. Environment Variables**

```yaml

apiVersion: v1

kind: Pod

metadata:

```
name: env-demo
spec:
containers:
- name: nginx
image: nginx
env:
Static value
- name: ENV_TYPE
value: "production"

From ConfigMap
- name: CONFIG_KEY
valueFrom:
configMapKeyRef:
name: app-config
key: database.url

From Secret
- name: DB_PASSWORD
valueFrom:
secretKeyRef:
name: db-secret
key: password

From Pod metadata (downward API)
- name: POD_NAME
valueFrom:
fieldRef:
fieldPath: metadata.name

- name: POD_IP
valueFrom:
fieldRef:
```

```
fieldPath: status.podIP

# From resource limits
- name: MEMORY_LIMIT
valueFrom:
resourceFieldRef:
containerName: nginx
resource: limits.memory

# Load all keys from ConfigMap
envFrom:
- configMapRef:
name: app-config
- secretRef:
name: app-secrets
```

How Environment Variables are Injected:
```
1. Pod created with env definitions
2. Kubelet resolves values:
- Fetches ConfigMap/Secret from API server
- Evaluates fieldRef from pod metadata
3. Passes environment variables to container runtime
4. containerd sets env vars in container process
5. Application reads env vars
```

---

3. ConfigMaps and Secrets

ConfigMaps - Non-sensitive configuration:

```bash
# Create ConfigMap imperatively
kubectl create configmap app-config \
--from-literal=database.url=postgres://db:5432 \
--from-literal=log.level=debug \
--from-file=config.properties

# Create from file
kubectl create configmap nginx-config \
--from-file=nginx.conf
```

```yaml
# Declarative ConfigMap
apiVersion: v1
kind: ConfigMap
metadata:
name: app-config
data:
database.url: "postgres://db:5432/myapp"
log.level: "info"
config.json: |
{
"feature_flags": {
"new_ui": true,
"beta_features": false
}
}
```

Using ConfigMaps in Pods:

```yaml
```

```
apiVersion: v1
kind: Pod
metadata:
name: configmap-demo
spec:
containers:
- name: app
image: myapp:latest

# Method 1: As environment variables
envFrom:
- configMapRef:
name: app-config

# Method 2: Specific key as env var
env:
- name: DATABASE_URL
valueFrom:
configMapKeyRef:
name: app-config
key: database.url

# Method 3: As volume mount (file)
volumeMounts:
- name: config-volume
mountPath: /etc/config
readOnly: true

volumes:
- name: config-volume
configMap:
name: app-config
items: # Optional: select specific keys
```

- key: config.json
path: app-config.json
```

**ConfigMap Update Behavior**:
```
Environment Variables: NOT updated (requires pod restart)
Volume Mounts: Updated automatically (kubelet syncs every 60s)
```

---

**Secrets** - Sensitive data:

```bash
# Create Secret imperatively
kubectl create secret generic db-secret \
--from-literal=username=admin \
--from-literal=password=secretpass

# Create TLS secret
kubectl create secret tls my-tls-secret \
--cert=path/to/cert.crt \
--key=path/to/key.key

# Create Docker registry secret
kubectl create secret docker-registry regcred \
--docker-server=registry.example.com \
--docker-username=user \
--docker-password=pass \
--docker-email=user@example.com
```

```yaml

```
Declarative Secret (base64 encoded)
apiVersion: v1
kind: Secret
metadata:
name: db-secret
type: Opaque
data:
username: YWRtaW4= # echo -n "admin" | base64
password: c2VjcmV0cGFzcw== # echo -n "secretpass" | base64

Or use stringData (plain text, K8s encodes it)
stringData:
username: admin
password: secretpass
```

Using Secrets in Pods:

```yaml
apiVersion: v1
kind: Pod
metadata:
name: secret-demo
spec:
containers:
- name: app
image: myapp

# As environment variable
env:
- name: DB_PASSWORD
valueFrom:
secretKeyRef:
```

```
name: db-secret
key: password

# As volume mount
volumeMounts:
- name: secret-volume
mountPath: /etc/secrets
readOnly: true

# Pull image from private registry
imagePullSecrets:
- name: regcred

volumes:
- name: secret-volume
secret:
secretName: db-secret
defaultMode: 0400 # File permissions
```

Secret Storage in etcd:
```
■■ Secrets are base64 encoded, NOT encrypted by default in etcd
To encrypt: Enable encryption at rest in API server

# /etc/kubernetes/manifests/kube-apiserver.yaml
--encryption-provider-config=/etc/kubernetes/encryption-config.yaml
```

Encryption Configuration:
```yaml
apiVersion: apiserver.config.k8s.io/v1
kind: EncryptionConfiguration
resources:
```

- resources:
- secrets
providers:
- aescbc:
keys:
- name: key1
secret: <base64-encoded-32-byte-key>
- identity: {}
```

---

### **3.2 Pod Health and Lifecycle Management**

#### **Restart Policies**

```yaml
apiVersion: v1
kind: Pod
metadata:
name: restart-demo
spec:
restartPolicy: Always # Always | OnFailure | Never
containers:
- name: app
image: myapp
```

Restart Policy Behavior:
```
Always (default for Deployments):
        - Container exits with any status → Restart
- Used for long-running services
```
```

OnFailure (for Jobs):

```
        - Container exits with non-zero → Restart
        - Container exits with 0 → Don't restart
```

Never (for one-time tasks):

```
        - Container exits → Never restart
```

Backoff Delay:

```
Container crashes repeatedly:
1st crash: Restart immediately
2nd crash: Wait 10s
3rd crash: Wait 20s
4th crash: Wait 40s
...
Max wait: 5 minutes

State: CrashLoopBackOff
```

Health Probes

Three Types of Probes:

1. Startup Probe - Is the application started?

```yaml
apiVersion: v1
kind: Pod
metadata:
name: startup-probe-demo
spec:
```

containers:

- name: slow-starter

image: myapp

startupProbe:

httpGet:

path: /healthz

port: 8080

initialDelaySeconds: 0

periodSeconds: 10

failureThreshold: 30 # 30 * 10 = 5 minutes to start

successThreshold: 1

timeoutSeconds: 5

```

**When to use**: Applications with long startup times

**Behavior**: Liveness and readiness probes disabled until startup succeeds

---

**2. Liveness Probe** - Is the application alive?

```yaml

apiVersion: v1

kind: Pod

metadata:

name: liveness-probe-demo

spec:

containers:

- name: app

image: myapp

livenessProbe:

# HTTP check

httpGet:

path: /health

port: 8080

httpHeaders:

- name: Custom-Header

value: Awesome

initialDelaySeconds: 15

periodSeconds: 10

failureThreshold: 3

successThreshold: 1

timeoutSeconds: 5

```

Alternative probe types:

```yaml

# TCP Socket

livenessProbe:

tcpSocket:

port: 8080

initialDelaySeconds: 15

periodSeconds: 10

# Command execution

livenessProbe:

exec:

command:

- cat

- /tmp/healthy

initialDelaySeconds: 5

periodSeconds: 5

# gRPC (Kubernetes 1.24+)

livenessProbe:

grpc:

port: 9090

initialDelaySeconds: 5
```

Liveness Probe Workflow:
```
Every 10 seconds (periodSeconds):
1. Kubelet sends HTTP GET /health request
2. If response:
- 200-399: Success
- Other: Failure
3. If timeout (5s): Failure
4. If 3 consecutive failures (failureThreshold):
- Mark container as unhealthy
- Kill container
- Restart based on restartPolicy
```

Failure Example:
```
Time Event
0s Container started
15s First liveness probe (initialDelaySeconds)
25s Probe succeeds (HTTP 200)
35s Probe succeeds
45s Probe fails (HTTP 500)
55s Probe fails (2/3)
65s Probe fails (3/3)
65s Container killed
66s New container started
```

```

3. Readiness Probe - Is the application ready to serve traffic?

```yaml
apiVersion: v1
kind: Pod
metadata:
name: readiness-probe-demo
spec:
containers:
- name: app
image: myapp
ports:
- containerPort: 8080
readinessProbe:
httpGet:
path: /ready
port: 8080
initialDelaySeconds: 5
periodSeconds: 5
failureThreshold: 3
successThreshold: 1
```

Readiness Probe Workflow:

```
Every 5 seconds:
1. Kubelet sends HTTP GET /ready request
2. If success:
- Mark pod as Ready
- Add pod to Service endpoints
- Traffic can be routed to this pod
3. If failure:
- Mark pod as Not Ready
```

- Remove pod from Service endpoints
- No traffic routed to this pod
- Pod NOT restarted (unlike liveness)
```

**Readiness vs Liveness**:
```
Liveness Probe Failed:
        → Container is killed and restarted

Readiness Probe Failed:
        → Pod removed from service endpoints
        → Container keeps running
        → Can recover without restart
```

---

**Probe Configuration Best Practices**:
```yaml
# Example with all probes
apiVersion: v1
kind: Pod
metadata:
name: complete-probe-example
spec:
containers:
- name: app
image: myapp
ports:
- containerPort: 8080

# Startup: For slow-starting apps

```
startupProbe:
httpGet:
path: /startup
port: 8080
initialDelaySeconds: 0
periodSeconds: 10
failureThreshold: 30 # 5 min startup time allowed

# Liveness: Detect deadlocks
livenessProbe:
httpGet:
path: /healthz
port: 8080
initialDelaySeconds: 30
periodSeconds: 10
failureThreshold: 3
timeoutSeconds: 5

# Readiness: Traffic management
readinessProbe:
httpGet:
path: /ready
port: 8080
initialDelaySeconds: 10
periodSeconds: 5
failureThreshold: 3
successThreshold: 1
```

Probe Timing Diagram:

```
Container Start
        ↓
```

```
        0s ■■■
```

PreStop Hook:

```yaml
apiVersion: v1
kind: Pod
metadata:
name: lifecycle-demo
spec:
containers:
- name: app
image: myapp
lifecycle:
# Executed before container is terminated
preStop:
exec:
command: ["/bin/sh", "-c", "nginx -s quit; sleep 30"]
# Executed immediately after container starts
postStart:
exec:
command: ["/bin/sh", "-c", "echo Hello > /usr/share/message"]
terminationGracePeriodSeconds: 60
```

Graceful Shutdown Flow:

```
User deletes pod
        ↓
        Pod status → Terminating
        ↓
1. Remove from endpoints (no new traffic)
        ↓
2. PreStop hook executes (e.g., /nginx -s quit)
        ↓
```

3. SIGTERM sent to main process

↓

4. Application drains existing connections

↓

5. Wait up to 60s (terminationGracePeriodSeconds)

↓

6. If still running → SIGKILL

↓

Container terminated

```

---

### **3.3 Namespaces - Logical Isolation**

**Purpose of Namespaces**:

```
- Logical separation of resources
- Multi-tenancy (dev, staging, prod)
- Resource quotas and limits per namespace
- RBAC policies per namespace
```

**Default Namespaces**:

```bash
kubectl get namespaces

# Output:
# default - Default namespace for resources
# kube-system - Kubernetes system components
# kube-public - Publicly accessible (even to unauthenticated users)
# kube-node-lease - Node heartbeat data (leases)
```
```

Create Namespace:

```bash
# Imperative
kubectl create namespace development

# Declarative
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Namespace
metadata:
name: development
labels:
environment: dev
team: backend
EOF
```

Using Namespaces:

```bash
# Create resources in namespace
kubectl apply -f pod.yaml -n development

# Set default namespace for context
kubectl config set-context --current --namespace=development

# View resources across all namespaces
kubectl get pods --all-namespaces
kubectl get pods -A
```

Namespace Resource Quotas:

```yaml
apiVersion: v1
```

kind: ResourceQuota

metadata:

name: dev-quota

namespace: development

spec:

hard:

requests.cpu: "10"

requests.memory: 20Gi

limits.cpu: "20"

limits.memory: 40Gi

pods: "50"

services: "10"

persistentvolumeclaims: "20"

```

**Limit Ranges** (default limits for pods):

```yaml

apiVersion: v1

kind: LimitRange

metadata:

name: dev-limitrange

namespace: development

spec:

limits:

- max:

cpu: "2"

memory: 4Gi

min:

cpu: "100m"

memory: 128Mi

default: # Default limits

cpu: "500m"

memory: 512Mi

defaultRequest: # Default requests

cpu: "250m"

memory: 256Mi

type: Container

```

Cross-Namespace Communication:

```

Pod in namespace "frontend" accessing service in "backend":

Service DNS: <service-name>.<namespace>.svc.cluster.local

Example:

database.backend.svc.cluster.local

Short form (same namespace):

database

Within cluster:

database.backend

```

3.4 ReplicaSets - Self-Healing and Scaling

Purpose:

```

- Ensure specified number of pod replicas are running

- Self-healing: Replace failed pods

- Scaling: Increase/decrease pod count

```

ReplicaSet Architecture:
```

```
        ■■■
```

```
template: # Pod template
metadata:
labels:
app: nginx # Must match selector
spec:
containers:
- name: nginx
image: nginx:1.25
ports:
- containerPort: 80
```

Internal Workflow:

```
1. ReplicaSet created in etcd
2. ReplicaSet controller watches for ReplicaSets
3. Controller counts pods matching selector (app=nginx)
4. Current count = 0, Desired = 3
5. Controller creates 3 pods from template
6. Scheduler assigns pods to nodes
7. Kubelet starts containers
8. Controller continuously monitors:
- Every 15 seconds, count pods
- If count < desired: Create more pods
- If count > desired: Delete excess pods
- If pod dies: Create replacement
```

Reconciliation Loop:

```go
// Pseudo-code
for {
```

```
desiredReplicas := replicaSet.Spec.Replicas
currentPods := getPods(replicaSet.Spec.Selector)

if len(currentPods) < desiredReplicas {
createPods(desiredReplicas - len(currentPods))
} else if len(currentPods) > desiredReplicas {
deletePods(len(currentPods) - desiredReplicas)
}

sleep(15 * time.Second)
}
```

Scaling ReplicaSet:

```bash
# Imperative scaling
kubectl scale replicaset nginx-replicaset --replicas=5

# Declarative scaling (edit YAML and apply)
kubectl edit replicaset nginx-replicaset
# Change replicas: 3 to replicas: 5
kubectl apply -f replicaset.yaml
```

Self-Healing Example:

```
1. ReplicaSet has 3 pods running
2. One pod crashes (node failure, OOM, etc.)
3. Controller detects: current=2, desired=3
4. Controller creates new pod
5. Scheduler assigns to healthy node
6. Kubelet starts container
7. System back to desired state (3 pods)
```

All happens automatically in seconds!

```

---

### **3.5 Deployments - Declarative Updates**

**Why Deployments?**

```

ReplicaSets: Manage pod replicas

Deployments: Manage ReplicaSets + Rolling updates/rollbacks

        Deployment → ReplicaSet → Pods

```

**Deployment Architecture**:

```

        ┌───┐

        │   Deployment Controller   │

        └───┘

        ↓ manages

        ┌───┐

        │   Deployment   │

        │ replicas: 3   │

        │ strategy: RollingUpdate   │

        └───┘

        ↓ creates/manages

        ┌──────────────────────────────┐

        │ ReplicaSet │ ReplicaSet │

        │ (old v1.24) │ (new v1.25) │

        └──────────────────────────────┘

        ↓   ↓

        ┌───────────┐   ┌───────────┐

        │ Pods   │   │ Pods   │

```
        ■ (old)  ■       ■ (new)  ■
        ■■■■■■■■■■       ■■■■■■■■■■
```

Deployment Definition:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
name: nginx-deployment
labels:
app: nginx
spec:
replicas: 3
selector:
matchLabels:
app: nginx

# Deployment strategy
strategy:
type: RollingUpdate
rollingUpdate:
maxSurge: 1 # Max pods above desired during update
maxUnavailable: 1 # Max pods unavailable during update

# Revision history limit
revisionHistoryLimit: 10

# Pod template
template:
metadata:
labels:
app: nginx
```

```
spec:
containers:
- name: nginx
image: nginx:1.25
ports:
- containerPort: 80
resources:
requests:
cpu: 100m
memory: 128Mi
limits:
cpu: 250m
memory: 256Mi
```

Create Deployment:

```bash
# Imperative
kubectl create deployment nginx --image=nginx:1.25 --replicas=3

# Declarative
kubectl apply -f deployment.yaml

# Check status
kubectl get deployments
kubectl get rs
kubectl get pods
```

Deployment Strategies

1. RollingUpdate (default) - Zero downtime:

```yaml
strategy:
type: RollingUpdate
rollingUpdate:
maxSurge: 1 # Can have 4 pods during update (3+1)
maxUnavailable: 1 # Can have 2 running during update (3-1)
```

Rolling Update Workflow (3 replicas, image update):

```
Initial state: 3 pods running (nginx:1.24)

1. Create new ReplicaSet for nginx:1.25
2. Scale new RS to 1 (maxSurge=1)
Old: 3 pods (nginx:1.24)
New: 1 pod (nginx:1.25)
Total: 4 pods

3. Wait for new pod to be Ready

4. Scale down old RS by 1 (maxUnavailable=1)
Old: 2 pods
New: 1 pod
Total: 3 pods

5. Scale new RS to 2
Old: 2 pods
New: 2 pods
Total: 4 pods

6. Scale down old RS to 1
Old: 1 pod
New: 2 pods
Total: 3 pods
```

7. Scale new RS to 3

Old: 1 pod

New: 3 pods

Total: 4 pods

8. Scale down old RS to 0

Old: 0 pods

New: 3 pods

Total: 3 pods

Update complete!

```

**Step-by-step visualization**:

```

Time Old RS New RS Total Ready

0s 3 0 3 3 (Initial)

2s 3 1 4 3 (New pod creating)

5s 3 1 4 4 (New pod ready)

6s 2 1 3 3 (Old pod terminating)

8s 2 2 4 4 (2nd new pod ready)

10s 1 2 3 3

12s 1 3 4 4

14s 0 3 3 3 (Update complete)

```

---

**2. Recreate** - Downtime during update:

```yaml

strategy:

type: Recreate

```
```

Recreate Workflow:

```
1. Scale old ReplicaSet to 0 (all pods terminated)
Downtime starts here!
2. Wait for all old pods to terminate
3. Create new ReplicaSet
4. Scale new ReplicaSet to desired replicas
5. Wait for new pods to be ready
Downtime ends here
```

When to use Recreate:

```
- Can't have old and new versions running simultaneously
- Database schema changes
- Stateful applications with version incompatibility
```

Updating Deployments

Update Image:

```bash
# Imperative
kubectl set image deployment/nginx-deployment nginx=nginx:1.26

# Declarative (edit YAML)
# Change image: nginx:1.25 to nginx:1.26
kubectl apply -f deployment.yaml

# Check rollout status
kubectl rollout status deployment/nginx-deployment
```

```
# Output during rollout:
# Waiting for deployment "nginx-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
# Waiting for deployment "nginx-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
# Waiting for deployment "nginx-deployment" rollout to finish: 1 old replicas are pending termination...
# deployment "nginx-deployment" successfully rolled out
```

Watch Rollout:

```bash
# Terminal 1: Watch pods
watch kubectl get pods

# Terminal 2: Update deployment
kubectl set image deployment/nginx-deployment nginx=nginx:1.26

# You'll see:
# nginx-deployment-7d6b8f9c-abc12 1/1 Running 0 5m
# nginx-deployment-7d6b8f9c-def34 1/1 Running 0 5m
# nginx-deployment-7d6b8f9c-ghi56 1/1 Running 0 5m
# nginx-deployment-8c9a7b6d-jkl78 0/1 Pending 0 1s <- New pod
# nginx-deployment-8c9a7b6d-jkl78 0/1 ContainerCreating 0 2s
# nginx-deployment-8c9a7b6d-jkl78 1/1 Running 0 5s
# nginx-deployment-7d6b8f9c-abc12 1/1 Terminating 0 5m <- Old pod
# ...
```

Rollback Deployments

Revision History:

```bash
```

```
# View rollout history
kubectl rollout history deployment/nginx-deployment

# Output:
# REVISION CHANGE-CAUSE
# 1 kubectl apply --filename=deployment.yaml
# 2 kubectl set image deployment/nginx-deployment nginx=nginx:1.26
# 3 kubectl set image deployment/nginx-deployment nginx=nginx:1.27

# View specific revision details
kubectl rollout history deployment/nginx-deployment --revision=2
```

Rollback to Previous Version:

```bash
# Rollback to previous revision
kubectl rollout undo deployment/nginx-deployment

# Rollback to specific revision
kubectl rollout undo deployment/nginx-deployment --to-revision=2

# Pause rollout (for testing)
kubectl rollout pause deployment/nginx-deployment

# Resume rollout
kubectl rollout resume deployment/nginx-deployment
```

Rollback Workflow:

```
1. User runs: kubectl rollout undo deployment/nginx
2. Deployment controller detects undo
3. Finds previous ReplicaSet (nginx:1.25)
4. Scales old RS from 0 to desired replicas
```

5. Scales current RS down to 0
6. Uses RollingUpdate strategy
7. Rollback complete
```

---

#### **Deployment Patterns**

**Blue-Green Deployment**:
```yaml
# Blue deployment (current)
apiVersion: apps/v1
kind: Deployment
metadata:
name: myapp-blue
spec:
replicas: 3
selector:
matchLabels:
app: myapp
version: blue
template:
metadata:
labels:
app: myapp
version: blue
spec:
containers:
- name: myapp
image: myapp:1.0

---

```
# Green deployment (new version)
apiVersion: apps/v1
kind: Deployment
metadata:
name: myapp-green
spec:
replicas: 3
selector:
matchLabels:
app: myapp
version: green
template:
metadata:
labels:
app: myapp
version: green
spec:
containers:
- name: myapp
image: myapp:2.0

---
# Service (switch by changing selector)
apiVersion: v1
kind: Service
metadata:
name: myapp-service
spec:
selector:
app: myapp
version: blue # Change to "green" to switch traffic
ports:
```

- port: 80
```

**Canary Deployment** (gradual rollout):
```yaml
# Stable deployment (90% traffic)
apiVersion: apps/v1
kind: Deployment
metadata:
name: myapp-stable
spec:
replicas: 9
selector:
matchLabels:
app: myapp
track: stable
template:
metadata:
labels:
app: myapp
track: stable
spec:
containers:
- name: myapp
image: myapp:1.0

---
# Canary deployment (10% traffic)
apiVersion: apps/v1
kind: Deployment
metadata:
name: myapp-canary

```
spec:
replicas: 1
selector:
matchLabels:
app: myapp
track: canary
template:
metadata:
labels:
app: myapp
track: canary
spec:
containers:
- name: myapp
image: myapp:2.0

---
# Service (routes to both)
apiVersion: v1
kind: Service
metadata:
name: myapp-service
spec:
selector:
app: myapp # Matches both stable and canary
ports:
- port: 80

# Traffic distribution:
# 9 pods stable + 1 pod canary = 90:10 ratio
```

Phase 4: Services - Networking and Load Balancing

4.1 Service Fundamentals

Why Services?
```
Problem:
- Pods are ephemeral (IPs change when recreated)
- Clients need stable endpoint
- Load balancing across pod replicas

Solution: Services
- Stable virtual IP (ClusterIP)
- DNS name
- Load balancing
- Service discovery
```

Service Types:
```
1. ClusterIP (default) - Internal cluster access only
2. NodePort - Exposes on each node's IP:port
3. LoadBalancer - Cloud provider's load balancer
4. ExternalName - DNS CNAME redirect
```

4.2 ClusterIP Service

Definition:
```yaml
apiVersion: v1
```

kind: Service

metadata:

name: nginx-service

spec:

type: ClusterIP # Default, can omit

selector:

app: nginx # Matches pods with this label

ports:

- protocol: TCP

port: 80 # Service port

targetPort: 8080 # Pod port

```

**How ClusterIP Works**:

```
        ┌───┐
        │ Service: nginx-service   │
        │ ClusterIP: 10.96.0.10   │
        │ Port: 80   │
        └───┘
        ↓ (selector: app=nginx)
        ┌─────────────────────────────┐
        │ Pod 1 │ Pod 2 │ Pod 3 │
        │ 10.244 │ 10.244 │ 10.244 │
        │ .1.5   │ .2.8   │ .3.2   │
        │ :8080   │ :8080   │ :8080   │
        └─────────────────────────────┘
```

**Internal Workflow**:

```

1. Service created with ClusterIP: 10.96.0.10

2. Endpoints controller watches Service

3. Finds pods matching selector (app=nginx)

4. Creates Endpoints object with pod IPs:

- 10.244.1.5:8080

- 10.244.2.8:8080

- 10.244.3.2:8080

5. kube-proxy on each node watches Service/Endpoints

6. Creates iptables/IPVS rules to route traffic

7. CoreDNS creates DNS entry: nginx-service.default.svc.cluster.local

```

**Check Endpoints**:

```bash

kubectl get endpoints nginx-service

# Output:

# NAME ENDPOINTS AGE

# nginx-service 10.244.1.5:8080,10.244.2.8:8080,... 5m

```

Traffic Flow:

```

Client Pod (10.244.1.10)

↓

Request to nginx-service:80 (or 10.96.0.10:80)

↓

DNS resolves to 10.96.0.10

↓

Packet hits iptables rules (kube-proxy)

↓

Random pod selected (10.244.2.8:8080)

↓

Packet DNAT to pod IP

```
            ↓
Response back to client
```

iptables Rules (created by kube-proxy):

```bash
# View service rules
sudo iptables -t nat -L KUBE-SERVICES | grep nginx-service

# Chain structure:
-A KUBE-SERVICES -d 10.96.0.10/32 -p tcp -m tcp --dport 80 \
-j KUBE-SVC-NGINX

# Load balancing to endpoints:
-A KUBE-SVC-NGINX -m statistic --mode random --probability 0.33 \
-j KUBE-SEP-POD1
-A KUBE-SVC-NGINX -m statistic --mode random --probability 0.50 \
-j KUBE-SEP-POD2
-A KUBE-SVC-NGINX -j KUBE-SEP-POD3

# DNAT to pod IPs:
-A KUBE-SEP-POD1 -p tcp -m tcp -j DNAT --to-destination 10.244.1.5:8080
```

4.3 NodePort Service

Definition:

```yaml
apiVersion: v1
kind: Service
metadata:
name: nginx-nodeport
```

spec:

type: NodePort

selector:

app: nginx

ports:

- protocol: TCP

port: 80 # ClusterIP port

targetPort: 8080 # Pod port

nodePort: 30080 # Port on each node (30000-32767)

```

**How NodePort Works**:

```
External Client
        ↓
Node IP:30080 (any node)
        ↓
iptables routes to ClusterIP:80
        ↓
ClusterIP load balances to pod
        ↓
Pod receives request
```

**NodePort Architecture**:

```
        ┌───┐
        │ External Client   │
        │ (Internet, corporate network)   │
        └───┘
        ↓
        ┌──────────────────────────────────┐
```
```

```
        ■ Node 1  ■ Node 2  ■ Node 3  ■
        ■:30080   ■:30080   ■:30080   ■
        ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        ↓ (All nodes listen on :30080)
        ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        ■ Service (ClusterIP:80)      ■
        ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        ↓
        ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        ■  Pod 1  ■  Pod 2  ■  Pod 3  ■
        ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Access NodePort Service:

```bash
# From outside cluster
curl http://node1-ip:30080
curl http://node2-ip:30080 # Works from any node

# From inside cluster
curl http://nginx-nodeport:80 # ClusterIP still works
```

When to use NodePort:

```
✓ Development/testing
✓ On-premises clusters without LoadBalancer
✗ Production (use LoadBalancer instead)
✗ Security concerns (opens port on all nodes)
```

4.3 LoadBalancer Service

Definition:

```yaml
apiVersion: v1
kind: Service
metadata:
name: nginx-lb
spec:
type: LoadBalancer
selector:
app: nginx
ports:
- protocol: TCP
port: 80
targetPort: 8080
```

How LoadBalancer Works:

```
Cloud Provider (AWS/GCP/Azure)
        ↓
Creates external load balancer
        ↓ (Assigns external IP: 203.0.113.50)
Load Balancer routes to NodePorts
        ↓ (e.g., :30080 on all nodes)
NodePort routes to ClusterIP
        ↓
ClusterIP load balances to pods
```

LoadBalancer Architecture:

```
```

```
        ■■■
```

Check LoadBalancer Status:

```bash
kubectl get svc nginx-lb

# Output:
# NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
# nginx-lb LoadBalancer 10.96.0.20 203.0.113.50 80:30080/TCP 5m

# EXTERNAL-IP shows:
# - <pending>: LB being created
# - IP address: LB ready
```

Cloud-Specific Annotations:

```yaml
# AWS ELB
apiVersion: v1
kind: Service
metadata:
name: nginx-lb
annotations:
service.beta.kubernetes.io/aws-load-balancer-type: "nlb" # Network LB
service.beta.kubernetes.io/aws-load-balancer-internal: "true" # Internal LB
spec:
type: LoadBalancer
selector:
app: nginx
ports:
- port: 80

---
# GCP Load Balancer
```

metadata:

annotations:

cloud.google.com/load-balancer-type: "Internal" # Internal LB

```

---

### **4.4 ExternalName Service**

**Purpose**: Create DNS CNAME to external service

**Definition**:

```yaml

apiVersion: v1

kind: Service

metadata:

name: external-db

spec:

type: ExternalName

externalName: database.example.com # External DNS name

```

How it works:

```

Pod queries: external-db.default.svc.cluster.local

↓

CoreDNS returns CNAME: database.example.com

↓

Pod connects to database.example.com

```

Use case:

```

Migrating external service into cluster:

1. Create ExternalName pointing to external DB
2. Pods use service name (external-db)
3. Later, deploy DB in cluster
4. Change service to ClusterIP
5. No code changes in pods!
```

4.5 Service Discovery

DNS-Based Discovery:
```
Service DNS Format:
<service-name>.<namespace>.svc.<cluster-domain>

Examples:
nginx-service.default.svc.cluster.local
database.production.svc.cluster.local

Short forms (from same namespace):
nginx-service
nginx-service.default
```

CoreDNS Configuration:
```yaml
# CoreDNS ConfigMap
apiVersion: v1
kind: ConfigMap
metadata:
name: coredns
namespace: kube-system
data:

```
Corefile: |
.:53 {
errors
health
ready
kubernetes cluster.local in-addr.arpa ip6.arpa {
pods insecure
fallthrough in-addr.arpa ip6.arpa
}
prometheus :9153
forward . /etc/resolv.conf
cache 30
loop
reload
loadbalance
}
```

DNS Query Flow:

```
Pod (10.244.1.5)
        ↓
DNS query: nginx-service.default.svc.cluster.local
        ↓
Query sent to CoreDNS pod (kube-dns service: 10.96.0.10:53)
        ↓
CoreDNS queries Kubernetes API for service
        ↓
Returns ClusterIP: 10.96.0.20
        ↓
Pod connects to 10.96.0.20
        ↓
```

kube-proxy routes to pod
```

**Pod DNS Configuration**:
```yaml
# Automatically injected into pods
# /etc/resolv.conf inside pod:
nameserver 10.96.0.10 # kube-dns service
search default.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

4.6 Headless Services

Purpose: Direct pod-to-pod communication (no load balancing)

Definition:
```yaml
apiVersion: v1
kind: Service
metadata:
name: nginx-headless
spec:
clusterIP: None # Makes it headless
selector:
app: nginx
ports:
- port: 80
```

How Headless Service Works:
```

Regular Service:

```
        DNS query → Returns ClusterIP
```

Headless Service:

```
        DNS query → Returns all pod IPs directly
```

DNS Response:

```bash
# Regular service
nslookup nginx-service.default.svc.cluster.local
# Returns: 10.96.0.20 (ClusterIP)

# Headless service
nslookup nginx-headless.default.svc.cluster.local
# Returns: 10.244.1.5, 10.244.2.8, 10.244.3.2 (all pod IPs)
```

Use Cases:

```
1. StatefulSets (stable network identity)
2. Custom load balancing
3. Service meshes (Istio, Linkerd)
4. Databases with replication
```

StatefulSet with Headless Service:

```yaml
apiVersion: v1
kind: Service
metadata:
name: nginx-headless
spec:
```

clusterIP: None

selector:

app: nginx

ports:

- port: 80

---

apiVersion: apps/v1

kind: StatefulSet

metadata:

name: nginx

spec:

serviceName: nginx-headless

replicas: 3

selector:

matchLabels:

app: nginx

template:

metadata:

labels:

app: nginx

spec:

containers:

- name: nginx

image: nginx

Creates pods with predictable DNS names:

nginx-0.nginx-headless.default.svc.cluster.local

nginx-1.nginx-headless.default.svc.cluster.local

nginx-2.nginx-headless.default.svc.cluster.local

```

---
```

4.7 Session Affinity

Purpose: Route requests from same client to same pod

Configuration:

```yaml
apiVersion: v1
kind: Service
metadata:
name: nginx-service
spec:
selector:
app: nginx
sessionAffinity: ClientIP # Default: None
sessionAffinityConfig:
clientIP:
timeoutSeconds: 10800 # 3 hours
ports:
- port: 80
```

How it works:

```
Client IP: 192.168.1.100
        ↓
        First request → Routed to Pod 1
        ↓
        Subsequent requests from 192.168.1.100 → Always Pod 1
        ↓
        After 3 hours of inactivity → Affinity expires
```

Phase 5: Advanced Pod Patterns

5.1 Init Containers

Purpose: Run initialization tasks before main containers

Use Cases:
```
- Wait for dependent services
- Clone git repositories
- Set up configuration files
- Database migrations
- Pre-populate data
```

Init Container Behavior:
```
1. Run sequentially (one at a time)
2. Must complete successfully before next init container
3. Main containers start only after all init containers succeed
        4. If init container fails → Pod restarts (based on restartPolicy)
```

Example:
```yaml
apiVersion: v1
kind: Pod
metadata:
name: init-demo
spec:
# Init containers (run first)
initContainers:
```

```
- name: wait-for-database
image: busybox:1.36
command: ['sh', '-c']
args:
- |
until nslookup database.default.svc.cluster.local; do
echo "Waiting for database service..."
sleep 2
done
echo "Database service is available!"

- name: clone-repo
image: alpine/git:latest
command: ['git', 'clone']
args:
- 'https://github.com/example/app.git'
- '/app'
volumeMounts:
- name: app-code
mountPath: /app

- name: setup-permissions
image: busybox:1.36
command: ['chmod', '-R', '755', '/app']
volumeMounts:
- name: app-code
mountPath: /app

# Main containers (run after init containers)
containers:
- name: app
image: myapp:latest
volumeMounts:
```

```
- name: app-code
mountPath: /app

volumes:
- name: app-code
emptyDir: {}
```

Execution Flow:
```
Pod created
        ↓
Init Container 1: wait-for-database
- Runs DNS lookup
- Waits until service exists
- Exits successfully
        ↓
Init Container 2: clone-repo
- Clones git repository
- Saves to /app volume
- Exits successfully
        ↓
Init Container 3: setup-permissions
- Changes file permissions
- Exits successfully
        ↓
Main Container: app
- Starts with code ready in /app
- Runs normally
```

Pod Status During Init:
```bash
```

```
kubectl get pods

# Output shows init container progress:
# NAME READY STATUS RESTARTS AGE
# init-demo 0/1 Init:0/3 0 5s (First init container running)
# init-demo 0/1 Init:1/3 0 10s (Second init container running)
# init-demo 0/1 Init:2/3 0 15s (Third init container running)
# init-demo 0/1 PodInitializing 0 20s (All init containers done)
# init-demo 1/1 Running 0 25s (Main container running)
```

Debugging Init Containers:

```bash
# View init container logs
kubectl logs init-demo -c wait-for-database

# Describe pod to see init container status
kubectl describe pod init-demo
```

5.2 Multi-Container Pods (Sidecar Pattern)

Sidecar Containers: Helper containers that extend main container

Common Patterns:

```
1. Sidecar: Logging, monitoring, proxying
2. Ambassador: Proxy to external services
3. Adapter: Transform data format
```

Sidecar Pattern - Logging Example

```yaml
apiVersion: v1
kind: Pod
metadata:
name: sidecar-logging
spec:
containers:
# Main application container
- name: app
image: myapp:latest
volumeMounts:
- name: logs
mountPath: /var/log/app
# App writes logs to /var/log/app/access.log

# Sidecar: Log shipping container
- name: log-shipper
image: fluent/fluent-bit:latest
volumeMounts:
- name: logs
mountPath: /var/log/app
# Reads logs from shared volume
# Ships to Elasticsearch/Splunk

volumes:
- name: logs
emptyDir: {}
```

How it works:
```
Main Container (app)
          ↓
```

```
        Writes logs → /var/log/app/access.log
        ↓ (shared volume)
Sidecar Container (log-shipper)
        ↓
Reads logs ← /var/log/app/access.log
        ↓
        Ships to → Elasticsearch
```

Ambassador Pattern - Service Proxy

```yaml
apiVersion: v1
kind: Pod
metadata:
name: ambassador-demo
spec:
containers:
# Main application
- name: app
image: myapp:latest
env:
- name: DATABASE_HOST
value: "localhost" # Connects to ambassador on localhost
- name: DATABASE_PORT
value: "5432"

# Ambassador: Database proxy
- name: db-proxy
image: haproxy:latest
ports:
```

- containerPort: 5432

Proxies traffic to different DBs based on environment

Dev: dev-db.example.com

Prod: prod-db.example.com

```

**Architecture**:

```
        ■■■

```yaml
apiVersion: v1
kind: Pod
metadata:
name: adapter-demo
spec:
containers:
# Main application (legacy format)
- name: legacy-app
image: old-app:latest
volumeMounts:
- name: logs
mountPath: /var/log
# Writes logs in custom format

# Adapter: Converts to standard format
- name: log-adapter
image: log-converter:latest
volumeMounts:
- name: logs
mountPath: /var/log
# Reads custom format
# Outputs JSON format for monitoring system

volumes:
- name: logs
emptyDir: {}
```

5.3 DaemonSets

Purpose: Run one pod per node (automatically)

Use Cases:
```
- Node monitoring (Prometheus node exporter)
- Log collection (Fluentd, Filebeat)
- Networking (CNI agents, kube-proxy)
- Storage (Ceph, GlusterFS agents)
- Security (Falco)
```

DaemonSet Definition:
```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
name: fluentd-logging
namespace: kube-system
labels:
app: fluentd
spec:
selector:
matchLabels:
app: fluentd
template:
metadata:
labels:
app: fluentd
spec:
# Tolerate control plane taint (run on all nodes)
tolerations:
- key: node-role.kubernetes.io/control-plane
```

effect: NoSchedule

containers:

- name: fluentd

image: fluent/fluentd:latest

resources:

limits:

memory: 200Mi

requests:

cpu: 100m

memory: 200Mi

volumeMounts:

- name: varlog

mountPath: /var/log

- name: varlibdockercontainers

mountPath: /var/lib/docker/containers

readOnly: true

volumes:

- name: varlog

hostPath:

path: /var/log

- name: varlibdockercontainers

hostPath:

path: /var/lib/docker/containers

```

**DaemonSet Behavior**:

```

Cluster with 3 nodes:

↓

DaemonSet controller creates 3 pods

↓

```
1 pod scheduled on each node
        ↓
New node added to cluster
        ↓
DaemonSet controller automatically creates pod on new node
        ↓
Node removed
        ↓
Pod on that node automatically deleted
```

DaemonSet vs Deployment:

```
Deployment:
- Runs specified number of replicas
- Can run multiple pods per node
- Scheduler decides pod placement

DaemonSet:
- Runs exactly 1 pod per node
- Ignores default scheduler
- Automatically adds/removes pods when nodes change
```

Check DaemonSet:

```bash
kubectl get daemonsets -n kube-system

# Output:
# NAME DESIRED CURRENT READY UP-TO-DATE AVAILABLE
# fluentd-logging 3 3 3 3 3
# kube-proxy 3 3 3 3 3
# calico-node 3 3 3 3 3
```

DESIRED: Number of nodes

CURRENT: Running pods

```

**Update DaemonSet**:

```yaml

spec:

updateStrategy:

type: RollingUpdate

rollingUpdate:

maxUnavailable: 1 # Update one node at a time

```

Selective Node Deployment:

```yaml

spec:

template:

spec:

# Run only on nodes with SSD

nodeSelector:

disktype: ssd

# Or use node affinity

affinity:

nodeAffinity:

requiredDuringSchedulingIgnoredDuringExecution:

nodeSelectorTerms:

- matchExpressions:

- key: kubernetes.io/hostname

operator: In

values:

- worker-1

- worker-2
```

5.4 Static Pods

What are Static Pods?
```
- Managed directly by kubelet (not API server)
- Defined in manifest files on node
- Used for control plane components
```

Static Pod Location:
```
Default path: /etc/kubernetes/manifests/

Kubelet watches this directory
        Any YAML file added → Pod created
        File removed → Pod deleted
```

Control Plane as Static Pods:
```bash
ls /etc/kubernetes/manifests/

# Output:
# kube-apiserver.yaml
# kube-controller-manager.yaml
# kube-scheduler.yaml
# etcd.yaml
```
```

Example Static Pod:

```yaml
# /etc/kubernetes/manifests/static-web.yaml
apiVersion: v1
kind: Pod
metadata:
name: static-web
spec:
containers:
- name: nginx
image: nginx:latest
ports:
- containerPort: 80
```

How Static Pods Work:

```
1. Kubelet reads /etc/kubernetes/manifests/
2. Finds static-web.yaml
3. Creates pod directly (no API server involved)
4. Kubelet creates "mirror pod" in API server (read-only)
5. Pod appears in `kubectl get pods` with node name suffix
```

Mirror Pod:

```bash
kubectl get pods

# Output:
# NAME READY STATUS RESTARTS AGE
# static-web-node1 1/1 Running 0 5m

# Note: Pod name includes node name
```

Cannot be deleted via kubectl (kubelet recreates it)
```

**Configure Static Pod Path**:
```yaml
# /var/lib/kubelet/config.yaml
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
staticPodPath: /etc/kubernetes/manifests
```

Static Pods vs Regular Pods:
```
Static Pods:
- Managed by kubelet directly
- No scheduler involved
- Cannot use ConfigMaps/Secrets
- Used for critical system components
- Survive API server failures

Regular Pods:
- Managed by control plane
- Scheduled by kube-scheduler
- Can use all Kubernetes features
- Require API server
```

This completes the first major part. Due to length, I'll continue with the remaining topics in the next response. Would you like me to continue with:

- Gateway API and advanced networking
- Storage (Volumes, PVs, PVCs)
```

- Pod scheduling (node selectors, affinity, taints)
- Resource management
- Security (RBAC, Network Policies)
- Cluster operations and troubleshooting

Please let me know, and I'll continue with the same depth!

Kubernetes Deep Dive - Part 2: Advanced Networking, Storage, Scheduling & Security

Phase 6: Kubernetes Gateway API - Modern Traffic Management

6.1 Gateway API Overview

What is Gateway API?
```
Next-generation Ingress replacement
- More expressive routing
- Role-oriented design
- Portable across implementations
- Built for service mesh integration
```

Gateway API vs Ingress:
```
Ingress:
- Single resource type
- Limited routing capabilities
- Implementation-specific annotations
```

- No traffic splitting

Gateway API:

- Multiple resource types (Gateway, HTTPRoute, etc.)
- Advanced routing (header-based, weight-based)
- Standardized across providers
- Built-in traffic splitting, mirroring

```

**Gateway API Architecture**:

```
        ■■■

```
        ■          Application Developer                 ■
        ■         (Creates HTTPRoute)                    ■
        ■■■
```

name: nginx-gateway-config
```

**Common GatewayClass Implementations**:
```

1. nginx.org/nginx-gateway-controller

2. istio.io/gateway-controller

3. projectcontour.io/gateway-controller

4. traefik.io/gateway-controller
```

**Installation Example (NGINX Gateway)**:

```bash

# Install Gateway API CRDs

kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.0.0/standard-install.yaml

# Install NGINX Gateway Controller

kubectl apply -f https://github.com/nginxinc/nginx-gateway-fabric/releases/download/v1.0.0/deploy.yaml

# Verify GatewayClass

kubectl get gatewayclass
```

6.3 Gateway Resource

Purpose: Represents a load balancer with listeners

Gateway Definition:

```yaml

apiVersion: gateway.networking.k8s.io/v1

kind: Gateway

metadata:

```
name: production-gateway
namespace: default
spec:
gatewayClassName: nginx-gateway-class

listeners:
HTTP listener (port 80)
- name: http
protocol: HTTP
port: 80
hostname: "*.example.com"
allowedRoutes:
namespaces:
from: All # Allow routes from all namespaces

HTTPS listener (port 443)
- name: https
protocol: HTTPS
port: 443
hostname: "*.example.com"
tls:
mode: Terminate
certificateRefs:
- name: example-com-tls
kind: Secret
allowedRoutes:
namespaces:
from: Same # Only routes from same namespace

TCP listener (custom port)
- name: database
protocol: TCP
port: 5432
```

allowedRoutes:

kinds:

- kind: TCPRoute

```

**Gateway Workflow**:

```

1. User creates Gateway resource

2. Gateway controller (e.g., NGINX) detects Gateway

3. Controller provisions infrastructure:

- Creates LoadBalancer Service

- Configures listeners (ports 80, 443, 5432)

- Sets up TLS termination

4. Cloud provider assigns external IP

5. Gateway status updated with IP address

6. DNS configured to point to external IP

```

**Check Gateway Status**:

```bash

kubectl get gateway production-gateway

# Output:

# NAME CLASS ADDRESS PROGRAMMED AGE

# production-gateway nginx-gateway-class 203.0.113.100 True 5m

kubectl describe gateway production-gateway

# Status shows:

# Listeners:

# - Name: http

# Port: 80

# Protocol: HTTP

```
Ready: True
- Name: https
Port: 443
Protocol: HTTPS
Ready: True
```

6.4 HTTPRoute Resource

Purpose: Define HTTP routing rules

Basic HTTPRoute:

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
name: example-route
namespace: default
spec:
# Attach to gateway
parentRefs:
- name: production-gateway
namespace: default

# Hostname matching
hostnames:
- "www.example.com"
- "api.example.com"

# Routing rules
rules:
# Rule 1: Path-based routing
```

```
- matches:
- path:
type: PathPrefix
value: /api/v1
backendRefs:
- name: api-v1-service
port: 8080
weight: 100

# Rule 2: Exact path match
- matches:
- path:
type: Exact
value: /login
backendRefs:
- name: auth-service
port: 8080

# Rule 3: Header-based routing
- matches:
- path:
type: PathPrefix
value: /api
headers:
- name: X-API-Version
value: "v2"
backendRefs:
- name: api-v2-service
port: 8080

# Rule 4: Query parameter matching
- matches:
- path:
```

type: PathPrefix

value: /search

queryParams:

- name: beta

value: "true"

backendRefs:

- name: search-beta-service

port: 8080

Default rule (catch-all)

- backendRefs:

- name: default-service

port: 8080

```

**Traffic Flow**:

```

External Request: https://www.example.com/api/v1/users

↓

1. DNS resolves to Gateway IP (203.0.113.100)

↓

2. Request hits Gateway on port 443

↓

3. TLS terminated at Gateway

↓

4. Gateway matches hostname: www.example.com

↓

5. Gateway evaluates HTTPRoute rules:

```
- Path: /api/v1 → matches Rule 1
```

↓

6. Traffic forwarded to: api-v1-service:8080

↓

7. Service load balances to backend pods
↓
8. Response returned through Gateway
```

---

#### **Advanced HTTPRoute Features**

**1. Traffic Splitting (Canary Deployments)**:
```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
name: canary-route
spec:
parentRefs:
- name: production-gateway

hostnames:
- "app.example.com"

rules:
- matches:
- path:
type: PathPrefix
value: /
backendRefs:
# 90% traffic to stable version
- name: app-stable
port: 8080
weight: 90
# 10% traffic to canary version

- name: app-canary

port: 8080

weight: 10

```

Traffic Distribution:

```

100 requests to app.example.com

↓

Gateway distributes:

↓

```
90 requests → app-stable service

10 requests → app-canary service
```

Gradually increase canary weight:

```
weight: 10 → 30 → 50 → 100 (full rollout)
```

```

---

**2. Request/Response Header Modification**:

```yaml

apiVersion: gateway.networking.k8s.io/v1

kind: HTTPRoute

metadata:

name: header-modification-route

spec:

parentRefs:

- name: production-gateway

hostnames:

- "api.example.com"

rules:

```
- matches:
- path:
type: PathPrefix
value: /api

Modify request headers
filters:
- type: RequestHeaderModifier
requestHeaderModifier:
add:
- name: X-Custom-Header
value: "added-by-gateway"
set:
- name: X-Forwarded-Proto
value: "https"
remove:
- "X-Internal-Header"

Modify response headers
- type: ResponseHeaderModifier
responseHeaderModifier:
add:
- name: X-Cache-Status
value: "HIT"
remove:
- "Server" # Remove server header for security

backendRefs:
- name: api-service
port: 8080
```

---

3. Request Redirection:

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
name: redirect-route
spec:
parentRefs:
- name: production-gateway

rules:
# HTTP to HTTPS redirect
- matches:
- path:
type: PathPrefix
value: /
filters:
- type: RequestRedirect
requestRedirect:
scheme: https
statusCode: 301

# Domain redirect
- matches:
- path:
type: PathPrefix
value: /old-path
filters:
- type: RequestRedirect
requestRedirect:
path:
type: ReplaceFullPath
```

replaceFullPath: /new-path

statusCode: 302

```

---

**4. URL Rewriting**:

```yaml

apiVersion: gateway.networking.k8s.io/v1

kind: HTTPRoute

metadata:

name: rewrite-route

spec:

parentRefs:

- name: production-gateway

rules:

- matches:

- path:

type: PathPrefix

value: /api/v1

filters:

- type: URLRewrite

urlRewrite:

path:

type: ReplacePrefixMatch

        replacePrefixMatch: /v1  # /api/v1/users → /v1/users

backendRefs:

- name: api-service

port: 8080

```
```

---

5. Request Mirroring (Traffic Shadowing):

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
name: mirror-route
spec:
parentRefs:
- name: production-gateway

rules:
- matches:
- path:
type: PathPrefix
value: /api

filters:
# Mirror traffic to test service
- type: RequestMirror
requestMirror:
backendRef:
name: api-test-service
port: 8080

# Primary traffic
backendRefs:
- name: api-production-service
port: 8080
```

How Mirroring Works:

```
Client request to /api/users
        ↓
Gateway receives request
        ↓
1. Forward to api-production-service (wait for response)
2. Simultaneously mirror to api-test-service (fire-and-forget)
        ↓
Client receives response from production
Test service processes request in background (for testing)
```

---

6. Timeout Configuration:

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
name: timeout-route
spec:
parentRefs:
- name: production-gateway

rules:
- matches:
- path:
type: PathPrefix
value: /api

timeouts:
request: 30s # Total request timeout
backendRequest: 10s # Backend response timeout
```

backendRefs:

- name: api-service

port: 8080

```

---

**7. Retry Policy**:

```yaml

apiVersion: gateway.networking.k8s.io/v1

kind: HTTPRoute

metadata:

name: retry-route

spec:

parentRefs:

- name: production-gateway

rules:

- matches:

- path:

type: PathPrefix

value: /api

filters:

- type: ExtensionRef

extensionRef:

group: gateway.nginx.org

kind: RetryPolicy

name: api-retry-policy

backendRefs:

- name: api-service

port: 8080

---

# Custom RetryPolicy (implementation-specific)

apiVersion: gateway.nginx.org/v1

kind: RetryPolicy

metadata:

name: api-retry-policy

spec:

retries: 3

retryOn:

- 5xx

- timeout

- reset

perTryTimeout: 5s

```

6.5 Advanced Gateway API Patterns

Multi-Tenancy with Namespaces

```yaml

# Cluster operator creates Gateway

apiVersion: gateway.networking.k8s.io/v1

kind: Gateway

metadata:

name: shared-gateway

namespace: gateway-system

spec:

gatewayClassName: nginx-gateway-class

listeners:

- name: http

```
protocol: HTTP
port: 80
allowedRoutes:
namespaces:
from: All # Allow routes from any namespace

Team A creates HTTPRoute in their namespace
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
name: team-a-route
namespace: team-a
spec:
parentRefs:
- name: shared-gateway
namespace: gateway-system

hostnames:
- "team-a.example.com"

rules:
- backendRefs:
- name: team-a-service
port: 8080

Team B creates HTTPRoute in their namespace
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
name: team-b-route
namespace: team-b
```

spec:

parentRefs:

- name: shared-gateway

namespace: gateway-system

hostnames:

- "team-b.example.com"

rules:

- backendRefs:

- name: team-b-service

port: 8080

```

**Benefits**:

```

- Single Gateway shared across teams

- Teams manage their own routes

- Namespace isolation

- Independent deployments

```

---

#### **Cross-Namespace Routing (with ReferenceGrant)**

**Problem**: HTTPRoute in namespace A wants to route to Service in namespace B

**Solution**: ReferenceGrant

```yaml

# Service in namespace backend

apiVersion: v1

kind: Service

metadata:

```
name: shared-api-service
namespace: backend
spec:
selector:
app: api
ports:
- port: 8080

ReferenceGrant allows frontend namespace to reference backend service
apiVersion: gateway.networking.k8s.io/v1beta1
kind: ReferenceGrant
metadata:
name: allow-frontend-to-backend
namespace: backend
spec:
from:
- group: gateway.networking.k8s.io
kind: HTTPRoute
namespace: frontend
to:
- group: ""
kind: Service
name: shared-api-service

HTTPRoute in namespace frontend
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
name: frontend-route
namespace: frontend
```

spec:

parentRefs:

- name: production-gateway

rules:

- backendRefs:

- name: shared-api-service

namespace: backend # Cross-namespace reference

port: 8080

```

---

#### **TLS Configuration**

**TLS Termination at Gateway**:

```yaml

# Create TLS secret

apiVersion: v1

kind: Secret

metadata:

name: example-com-tls

namespace: default

type: kubernetes.io/tls

data:

tls.crt: <base64-encoded-cert>

tls.key: <base64-encoded-key>

---

# Gateway with TLS

apiVersion: gateway.networking.k8s.io/v1

kind: Gateway

metadata:

```
name: tls-gateway
spec:
gatewayClassName: nginx-gateway-class

listeners:
HTTPS listener
- name: https
protocol: HTTPS
port: 443
hostname: "*.example.com"
tls:
mode: Terminate # Terminate TLS at gateway
certificateRefs:
- name: example-com-tls
kind: Secret

Redirect HTTP to HTTPS
- name: http
protocol: HTTP
port: 80
hostname: "*.example.com"

HTTPRoute with HTTPS redirect
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
name: https-redirect
spec:
parentRefs:
- name: tls-gateway
sectionName: http # Attach to HTTP listener
```

rules:

- filters:

- type: RequestRedirect

requestRedirect:

scheme: https

statusCode: 301

```

**TLS Passthrough** (don't terminate TLS):

```yaml

apiVersion: gateway.networking.k8s.io/v1alpha2

kind: TLSRoute

metadata:

name: tls-passthrough

spec:

parentRefs:

- name: tls-gateway

hostnames:

- "secure.example.com"

rules:

- backendRefs:

- name: secure-backend

port: 443

# Gateway listener for passthrough

listeners:

- name: tls-passthrough

protocol: TLS

port: 443

tls:

mode: Passthrough # Don't terminate, pass to backend

```

6.6 Gateway API vs Traditional Ingress

Comparison Example:

Ingress (old way):
```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
name: example-ingress
annotations:
nginx.ingress.kubernetes.io/rewrite-target: /
nginx.ingress.kubernetes.io/ssl-redirect: "true"
# Implementation-specific annotations
spec:
ingressClassName: nginx
tls:
- hosts:
- example.com
secretName: example-tls
rules:
- host: example.com
http:
paths:
- path: /api
pathType: Prefix
backend:
service:
name: api-service
```
```

port:

number: 8080

```

**Gateway API (new way)**:

```yaml

# GatewayClass (once per cluster)

apiVersion: gateway.networking.k8s.io/v1

kind: GatewayClass

metadata:

name: nginx

spec:

controllerName: nginx.org/nginx-gateway-controller

---

# Gateway (cluster operator)

apiVersion: gateway.networking.k8s.io/v1

kind: Gateway

metadata:

name: production-gateway

spec:

gatewayClassName: nginx

listeners:

- name: https

protocol: HTTPS

port: 443

hostname: "*.example.com"

tls:

mode: Terminate

certificateRefs:

- name: example-tls

---

# HTTPRoute (application developer)

apiVersion: gateway.networking.k8s.io/v1

kind: HTTPRoute

metadata:

name: api-route

spec:

parentRefs:

- name: production-gateway

hostnames:

- "example.com"

rules:

- matches:

- path:

type: PathPrefix

value: /api

filters:

- type: URLRewrite

urlRewrite:

path:

type: ReplacePrefixMatch

replacePrefixMatch: /

backendRefs:

- name: api-service

port: 8080

```

Key Differences:

```

Ingress:

- Single resource
- Annotations for features
- Mixed concerns (infra + app)

Gateway API:
- Separation of concerns
- Type-safe configuration
- Portable across implementations
- Advanced features built-in
```

Phase 7: Storage in Kubernetes

7.1 Storage Concepts

Storage Types:
```
1. Ephemeral Storage (emptyDir)
- Temporary, deleted with pod

2. Node Storage (hostPath)
- Persists on node filesystem

3. Network Storage (NFS, Ceph, etc.)
- Shared across nodes

4. Cloud Storage (EBS, GCE PD, Azure Disk)
- Managed by cloud provider
```

7.2 Volumes
```

Volume Lifecycle:

```

Volume tied to pod lifecycle

        Pod created → Volume mounted

        Pod deleted → Volume deleted (for ephemeral)

Volume remains (for persistent)
```

emptyDir Volume

Purpose: Temporary storage shared between containers in pod

```yaml
apiVersion: v1
kind: Pod
metadata:
name: emptydir-demo
spec:
containers:
- name: writer
image: busybox
command: ['sh', '-c']
args:
- |
while true; do
date >> /data/log.txt
sleep 5
done
volumeMounts:
- name: shared-data
mountPath: /data
```

```
- name: reader
image: busybox
command: ['sh', '-c']
args:
- 'tail -f /data/log.txt'
volumeMounts:
- name: shared-data
mountPath: /data

volumes:
- name: shared-data
emptyDir: {}
```

emptyDir Storage Location:
```
Default: /var/lib/kubelet/pods/<pod-uid>/volumes/kubernetes.io~empty-dir/

Medium options:
emptyDir: {} # Disk storage
emptyDir:
medium: Memory # RAM (tmpfs)
sizeLimit: 1Gi
```

Use Cases:
```
✓ Cache/scratch space
✓ Sharing data between containers
✓ Checkpoint for long computation
✗ Persistent data (use PV/PVC)
```

---

hostPath Volume

Purpose: Mount directory from host node

```yaml
apiVersion: v1
kind: Pod
metadata:
name: hostpath-demo
spec:
containers:
- name: app
image: nginx
volumeMounts:
- name: host-logs
mountPath: /var/log/nginx

volumes:
- name: host-logs
hostPath:
path: /var/log/pods # Path on host
type: DirectoryOrCreate
```

hostPath Types:
```yaml
type: DirectoryOrCreate # Create dir if doesn't exist
type: Directory # Must be existing directory
type: FileOrCreate # Create file if doesn't exist
type: File # Must be existing file
type: Socket # Must be existing Unix socket
```

type: CharDevice # Must be character device

type: BlockDevice # Must be block device

```

**Security Warning**:

```

■■ hostPath exposes host filesystem

- Pod can read sensitive host files

- Pod can write to host filesystem

- Use only for trusted workloads

- Better alternatives: PV/PVC

```

**Use Cases**:

```

✓ DaemonSet accessing host files (/var/log, /var/lib/docker)

✓ Static pod manifests (control plane)

✗ Regular applications (security risk)

✗ Multi-node clusters (data on specific node)

```

---

#### **ConfigMap and Secret Volumes**

**ConfigMap as Volume**:

```yaml

apiVersion: v1

kind: ConfigMap

metadata:

name: app-config

data:

config.json: |

```
{
"database": "postgres",
"port": 5432
}
app.properties: |
log.level=INFO
max.connections=100

---
apiVersion: v1
kind: Pod
metadata:
name: configmap-volume-demo
spec:
containers:
- name: app
image: myapp
volumeMounts:
- name: config
mountPath: /etc/config
readOnly: true

volumes:
- name: config
configMap:
name: app-config
items: # Optional: select specific keys
- key: config.json
path: database-config.json
- key: app.properties
path: application.properties
```

Inside Container:

```bash
ls /etc/config/
# Output:
# database-config.json
# application.properties

cat /etc/config/database-config.json
# {
# "database": "postgres",
# "port": 5432
# }
```

ConfigMap Volume Behavior:

```
        ConfigMap updated → Files updated in pod (within ~60 seconds)
```

Application must reload config to see changes

Atomic update:

1. New directory created: /etc/config/..data_tmp
2. Files written to new directory

```
        3. Symlink updated: /etc/config/..data → ..data_tmp
```

4. Applications see new files

```

---

**Secret as Volume**:

```yaml
apiVersion: v1
kind: Secret
metadata:
```
```

name: db-credentials

type: Opaque

stringData:

username: admin

password: secretpassword

---

apiVersion: v1

kind: Pod

metadata:

name: secret-volume-demo

spec:

containers:

- name: app

image: myapp

volumeMounts:

- name: credentials

mountPath: /etc/secrets

readOnly: true

volumes:

- name: credentials

secret:

secretName: db-credentials

defaultMode: 0400 # File permissions (read-only for owner)

```

**Inside Container**:

```bash

ls -l /etc/secrets/

# -r-------- 1 root root 5 Jan 15 10:30 username

# -r-------- 1 root root 16 Jan 15 10:30 password

cat /etc/secrets/username

# admin

```

Projected Volumes

Purpose: Combine multiple volume sources into single directory

```yaml
apiVersion: v1
kind: Pod
metadata:
name: projected-volume-demo
spec:
containers:
- name: app
image: myapp
volumeMounts:
- name: all-in-one
mountPath: /projected-volume
readOnly: true

volumes:
- name: all-in-one
projected:
sources:
# 1. Secret
- secret:
name: db-credentials
items:
- key: username

path: secrets/db-user

# 2. ConfigMap

- configMap:

name: app-config

items:

- key: config.json

path: config/app.json

# 3. Downward API (pod metadata)

- downwardAPI:

items:

- path: labels

fieldRef:

fieldPath: metadata.labels

- path: annotations

fieldRef:

fieldPath: metadata.annotations

# 4. ServiceAccount token

- serviceAccountToken:

path: token

expirationSeconds: 3600

audience: api

```

Directory Structure:

```

/projected-volume/

```
    ■■■ secrets/
    ■   ■■■ db-user
    ■■■ config/
    ■   ■■■ app.json
```

```
        ■■■ labels
        ■■■ annotations
        ■■■ token
```

Downward API Volume

Purpose: Expose pod/container metadata as files

```yaml
apiVersion: v1
kind: Pod
metadata:
name: downward-api-demo
labels:
app: myapp
version: v1.0
annotations:
build-id: "12345"
spec:
containers:
- name: app
image: myapp
resources:
requests:
cpu: 100m
memory: 128Mi
limits:
cpu: 500m
memory: 256Mi
volumeMounts:
```

```
- name: podinfo
mountPath: /etc/podinfo

volumes:
- name: podinfo
downwardAPI:
items:
# Pod metadata
- path: pod-name
fieldRef:
fieldPath: metadata.name
- path: pod-namespace
fieldRef:
fieldPath: metadata.namespace
- path: pod-ip
fieldRef:
fieldPath: status.podIP
- path: labels
fieldRef:
fieldPath: metadata.labels

# Container resources
- path: cpu-limit
resourceFieldRef:
containerName: app
resource: limits.cpu
- path: memory-request
resourceFieldRef:
containerName: app
resource: requests.memory
```

Inside Container:

```bash
cat /etc/podinfo/pod-name
# downward-api-demo

cat /etc/podinfo/labels
# app="myapp"
# version="v1.0"

cat /etc/podinfo/cpu-limit
# 1 (500m = 0.5 cores, shown as millicores)
```

7.3 Persistent Volumes (PV) and Claims (PVC)

Persistent Volume Workflow:

```
        ┌─────────────────────────────────────────┐
        │  1. Admin creates PersistentVolume      │
        │     (NFS, EBS, Ceph, etc.)              │
        └─────────────────────────────────────────┘
        ↓
        ┌─────────────────────────────────────────┐
        │  2. User creates PersistentVolumeClaim  │
        │     (requests storage)                  │
        └─────────────────────────────────────────┘
        ↓
        ┌─────────────────────────────────────────┐
        │  3. Kubernetes binds PVC to PV          │
        │     (matching capacity, access mode)    │
        └─────────────────────────────────────────┘
        ↓
        ┌─────────────────────────────────────────┐
```

```
        ■  4. Pod uses PVC as volume             ■
        ■■■
```

server: nfs-server.example.com

path: /data/volumes/pv-nfs-data

Mount options

mountOptions:

- hard

- nfsvers=4.1

```

**Access Modes**:

```

ReadWriteOnce (RWO):

- Volume mounted as read-write by single node

- Most common for block storage (EBS, GCE PD)

ReadOnlyMany (ROX):

- Volume mounted as read-only by many nodes

- Useful for shared configuration

ReadWriteMany (RWX):

- Volume mounted as read-write by many nodes

- Requires shared filesystem (NFS, CephFS, GlusterFS)

```

**Reclaim Policies**:

```

Retain:

- PV retained when PVC deleted

- Admin must manually reclaim/delete

- Data preserved

Delete:

- PV and backing storage deleted when PVC deleted

- Default for dynamic provisioning

Recycle (deprecated):

- Basic scrub (rm -rf /volume/*)
- Use dynamic provisioning instead

```

**PV Lifecycle States**:

```
Available: No PVC bound, ready for claim
Bound: Bound to PVC
Released: PVC deleted, but not yet reclaimed
Failed: Automatic reclamation failed
```

---

**PV with AWS EBS**:

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
name: pv-ebs-data
spec:
capacity:
storage: 20Gi
accessModes:
- ReadWriteOnce # EBS supports only RWO
persistentVolumeReclaimPolicy: Delete
storageClassName: gp3

awsElasticBlockStore:
volumeID: vol-0abcd1234efgh5678
fsType: ext4
```
```

```

**PV with GCE Persistent Disk**:
```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
name: pv-gce-pd
spec:
capacity:
storage: 20Gi
accessModes:
- ReadWriteOnce
storageClassName: standard

gcePersistentDisk:
pdName: gce-disk-name
fsType: ext4
```

PersistentVolumeClaim (PVC)

PVC Definition:
```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
name: pvc-data
namespace: default
spec:
# Access modes (must match PV)
```
```

accessModes:

- ReadWriteOnce

Storage class

storageClassName: slow

Storage size request

resources:

requests:

storage: 5Gi # Request 5Gi (PV has 10Gi)

Optional: Selector to match specific PV

selector:

matchLabels:

type: nfs

```

**PVC Binding Logic**:

```

1. PVC created with request: 5Gi, RWO, storageClass=slow

2. Kubernetes searches for PV matching:

- Access mode: RWO

- Storage: >= 5Gi

- Storage class: slow

- Selector (if specified)

3. If match found: Bind PVC to PV

4. If no match: PVC stays Pending

- If dynamic provisioning enabled: Create new PV

```
5. PV state: Available → Bound

6. PVC state: Pending → Bound
```

```

**Check PVC Status**:
```

```bash
kubectl get pvc

# Output:
# NAME STATUS VOLUME CAPACITY ACCESS MODES STORAGECLASS AGE
# pvc-data Bound pv-nfs-data 10Gi RWO,ROX,RWX slow 2m

kubectl describe pvc pvc-data
# Events show binding details
```

Using PVC in Pods

```yaml
apiVersion: v1
kind: Pod
metadata:
name: pod-with-pvc
spec:
containers:
- name: app
image: nginx
volumeMounts:
- name: data
mountPath: /usr/share/nginx/html

volumes:
- name: data
persistentVolumeClaim:
claimName: pvc-data # Reference PVC
```

Data Persistence:

```
1. Pod writes to /usr/share/nginx/html
2. Data written to PV (NFS server)
3. Pod deleted
4. New pod created with same PVC
5. Data still available (persisted on NFS)
```

7.4 StorageClass - Dynamic Provisioning

Purpose: Automatically create PVs when PVCs are created

StorageClass Workflow:

```
User creates PVC
        ↓
References StorageClass
        ↓
Provisioner (cloud provider) creates PV automatically
        ↓
PV bound to PVC
        ↓
Pod uses PVC
```

StorageClass Definition (AWS EBS):

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
```

```
name: fast-ssd
provisioner: ebs.csi.aws.com # AWS EBS CSI driver
parameters:
type: gp3 # EBS volume type (gp2, gp3, io1, io2)
iopsPerGB: "50"
encrypted: "true"
kmsKeyId: arn:aws:kms:us-east-1:123456789012:key/abcd-1234
volumeBindingMode: WaitForFirstConsumer # Delay binding until pod scheduled
allowVolumeExpansion: true
reclaimPolicy: Delete
mountOptions:
- debug
```

Provisioners:

```
AWS EBS: ebs.csi.aws.com
GCE PD: pd.csi.storage.gke.io
Azure Disk: disk.csi.azure.com
NFS: nfs.csi.k8s.io
Ceph RBD: rbd.csi.ceph.com
Local: kubernetes.io/no-provisioner
```

Volume Binding Modes:

```
Immediate:
- PV created immediately when PVC created
- PV created in random zone
- Problem: Pod might be scheduled in different zone

WaitForFirstConsumer:
- PV creation delayed until pod using PVC is scheduled
```

- PV created in same zone as pod
- Recommended for topology-aware storage (EBS, GCE PD)
```

---

**Using Dynamic Provisioning**:
```yaml
# 1. StorageClass (created by admin)
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
name: fast-ssd
provisioner: ebs.csi.aws.com
parameters:
type: gp3
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true

---
# 2. PVC (created by user) - no PV needed!
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
name: pvc-dynamic
spec:
accessModes:
- ReadWriteOnce
storageClassName: fast-ssd # Reference StorageClass
resources:
requests:
storage: 20Gi

---

# 3. Pod using PVC

apiVersion: v1

kind: Pod

metadata:

name: pod-dynamic-storage

spec:

containers:

- name: app

image: nginx

volumeMounts:

- name: data

mountPath: /data

volumes:

- name: data

persistentVolumeClaim:

claimName: pvc-dynamic

```

What Happens:

```

1. User creates PVC with storageClassName: fast-ssd

2. PVC status: Pending

3. User creates pod using PVC

4. Scheduler assigns pod to node in us-east-1a

5. EBS CSI driver provisions 20Gi gp3 EBS volume in us-east-1a

6. PV created automatically

7. PVC bound to PV

8. Volume attached to node

9. Volume mounted in pod

10. Pod status: Running

```

Check Dynamic Provisioning:
```bash
# PVC shows dynamically created PV
kubectl get pvc pvc-dynamic

# Output:
# NAME STATUS VOLUME CAPACITY
# pvc-dynamic Bound pvc-abcd1234-5678-9012-efgh-345678901234 20Gi

# PV created automatically
kubectl get pv

# Output shows PV with name matching PVC
```

Volume Expansion (if allowed):
```yaml
# Original PVC: 20Gi
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
name: pvc-dynamic
spec:
accessModes:
- ReadWriteOnce
storageClassName: fast-ssd
resources:
requests:
storage: 20Gi
```
```

Expand to 50Gi (edit and apply)
spec:
resources:
requests:
storage: 50Gi # Changed from 20Gi
```

**Expansion Process**:
```
1. PVC updated with larger size
2. Controller resizes cloud volume (EBS resize)
3. If pod running:
- Volume expanded online (if supported)
- Or pod must be restarted
4. Filesystem expanded (resize2fs for ext4)
5. PVC capacity updated
```

```bash
kubectl get pvc pvc-dynamic -w

# Events:
# Normal VolumeResizeSuccessful Resize volume succeeded
# Normal FileSystemResizeSuccessful Filesystem resize succeeded
```

7.5 StatefulSets with Persistent Storage

StatefulSet with VolumeClaimTemplates:
```yaml
apiVersion: v1
kind: Service

```
metadata:
name: mysql-headless
spec:
clusterIP: None
selector:
app: mysql
ports:
- port: 3306

---
apiVersion: apps/v1
kind: StatefulSet
metadata:
name: mysql
spec:
serviceName: mysql-headless
replicas: 3
selector:
matchLabels:
app: mysql

template:
metadata:
labels:
app: mysql
spec:
containers:
- name: mysql
image: mysql:8.0
ports:
- containerPort: 3306
env:
```

```
- name: MYSQL_ROOT_PASSWORD
value: rootpassword
volumeMounts:
- name: data
mountPath: /var/lib/mysql

# VolumeClaimTemplates - create PVC per pod
volumeClaimTemplates:
- metadata:
name: data
spec:
accessModes: [ "ReadWriteOnce" ]
storageClassName: fast-ssd
resources:
requests:
storage: 20Gi
```

What Happens:
```
1. StatefulSet creates 3 pods:
- mysql-0
- mysql-1
- mysql-2

2. For each pod, PVC created:
- data-mysql-0 (20Gi)
- data-mysql-1 (20Gi)
- data-mysql-2 (20Gi)

3. Each PVC gets its own PV (dynamic provisioning)

4. Pods have stable storage:
```

- mysql-0 always uses data-mysql-0
- mysql-1 always uses data-mysql-1
- mysql-2 always uses data-mysql-2

5. Pod deleted and recreated:
- New pod gets same PVC
- Data persisted
```

**Verify PVCs**:
```bash
kubectl get pvc

# Output:
# NAME STATUS VOLUME CAPACITY STORAGECLASS
# data-mysql-0 Bound pvc-abc... 20Gi fast-ssd
# data-mysql-1 Bound pvc-def... 20Gi fast-ssd
# data-mysql-2 Bound pvc-ghi... 20Gi fast-ssd
```

Phase 8: Pod Scheduling and Node Management

8.1 Node Selectors

Purpose: Schedule pod on nodes with specific labels

Label Nodes:
```bash
# Add label to node
kubectl label nodes worker-1 disktype=ssd
kubectl label nodes worker-1 gpu=nvidia-tesla-v100

# View node labels

```
kubectl get nodes --show-labels

# Remove label
kubectl label nodes worker-1 disktype-
```

Pod with nodeSelector:

```yaml
apiVersion: v1
kind: Pod
metadata:
name: gpu-pod
spec:
nodeSelector:
disktype: ssd
gpu: nvidia-tesla-v100

containers:
- name: cuda-app
image: nvidia/cuda:11.0-base
```

Scheduling Logic:

```
1. Pod created with nodeSelector
2. Scheduler filters nodes:
- worker-1: disktype=ssd, gpu=nvidia ✓
- worker-2: disktype=hdd ✗
- worker-3: disktype=ssd (no GPU) ✗
3. Pod scheduled on worker-1
4. If no matching nodes: Pod stays Pending
```

Built-in Node Labels:
```
kubernetes.io/hostname: node1
kubernetes.io/os: linux
kubernetes.io/arch: amd64
node.kubernetes.io/instance-type: m5.large
topology.kubernetes.io/zone: us-east-1a
topology.kubernetes.io/region: us-east-1
```

8.2 Node Affinity

Purpose: More expressive node selection than nodeSelector

Node Affinity Types:
```
requiredDuringSchedulingIgnoredDuringExecution:
- MUST match (hard requirement)
- Pod not scheduled if no match

preferredDuringSchedulingIgnoredDuringExecution:
- PREFER match (soft requirement)
- Pod scheduled even if no match

IgnoredDuringExecution:
- If node labels change after pod scheduled, pod keeps running
```

Required Node Affinity:
```yaml
apiVersion: v1
kind: Pod
```

```
metadata:
name: node-affinity-required
spec:
affinity:
nodeAffinity:
requiredDuringSchedulingIgnoredDuringExecution:
nodeSelectorTerms:
# OR between terms
- matchExpressions:
# AND between expressions
- key: disktype
operator: In
values:
- ssd
- nvme
- key: kubernetes.io/arch
operator: In
values:
- amd64

containers:
- name: app
image: myapp
```

Operators:
```
In: Label value in list
NotIn: Label value not in list
Exists: Label key exists (any value)
DoesNotExist: Label key doesn't exist
Gt: Label value > number
```

Lt: Label value < number
```

---

**Preferred Node Affinity** (with weights):
```yaml
apiVersion: v1
kind: Pod
metadata:
name: node-affinity-preferred
spec:
affinity:
nodeAffinity:
preferredDuringSchedulingIgnoredDuringExecution:
# Preference 1 (weight 80)
- weight: 80
preference:
matchExpressions:
- key: disktype
operator: In
values:
- ssd

# Preference 2 (weight 20)
- weight: 20
preference:
matchExpressions:
- key: network
operator: In
values:
- 10Gbps

containers:

- name: app

image: myapp

```

Scoring Logic:

```

Nodes:

worker-1: disktype=ssd, network=10Gbps

worker-2: disktype=ssd

worker-3: disktype=hdd, network=10Gbps

Scores:

worker-1: 80 (disktype) + 20 (network) = 100

worker-2: 80 (disktype) + 0 = 80

worker-3: 0 + 20 (network) = 20

Scheduler picks worker-1 (highest score)

```

Combining Required and Preferred:

```yaml

apiVersion: v1

kind: Pod

metadata:

name: combined-affinity

spec:

affinity:

nodeAffinity:

# MUST have SSD

requiredDuringSchedulingIgnoredDuringExecution:

```
nodeSelectorTerms:
- matchExpressions:
- key: disktype
operator: In
values:
- ssd

PREFER GPU if available
preferredDuringSchedulingIgnoredDuringExecution:
- weight: 100
preference:
matchExpressions:
- key: gpu
operator: Exists

containers:
- name: app
image: myapp
```

8.3 Pod Affinity and Anti-Affinity

Purpose: Schedule pods relative to other pods

Pod Affinity Use Cases:
```
✓ Co-locate pods for low latency
✓ Place cache near application
✓ Data locality
```

Pod Affinity Example:

```yaml
apiVersion: v1
kind: Pod
metadata:
name: cache-pod
labels:
app: cache
spec:
containers:
- name: redis
image: redis

---
apiVersion: v1
kind: Pod
metadata:
name: app-pod
spec:
affinity:
podAffinity:
requiredDuringSchedulingIgnoredDuringExecution:
- labelSelector:
matchExpressions:
- key: app
operator: In
values:
- cache
topologyKey: kubernetes.io/hostname # Same node

containers:
- name: app
image: myapp
```

```

**Topology Keys**:
```
kubernetes.io/hostname:
- Same node

topology.kubernetes.io/zone:
- Same availability zone

topology.kubernetes.io/region:
- Same region

Custom topology:
- rack: rack-1
- datacenter: dc-west
```

**Workflow**:
```
1. cache-pod running on worker-1
2. app-pod created with podAffinity
3. Scheduler finds pods matching labelSelector (app=cache)
4. Gets topology (hostname) of those pods: worker-1
5. Schedules app-pod on same topology: worker-1
6. Both pods on same node (low latency)
```

---

**Pod Anti-Affinity** (spread pods):
```yaml
apiVersion: apps/v1
kind: Deployment

metadata:

name: web-app

spec:

replicas: 3

selector:

matchLabels:

app: web

template:

metadata:

labels:

app: web

spec:

affinity:

podAntiAffinity:

requiredDuringSchedulingIgnoredDuringExecution:

- labelSelector:

matchExpressions:

- key: app

operator: In

values:

- web

topologyKey: kubernetes.io/hostname # Different nodes

containers:

- name: nginx

image: nginx

```

Result:

```

3 replicas with anti-affinity:

```
        Pod 1 → worker-1
```

```
        Pod 2 → worker-2
        Pod 3 → worker-3
```

Each pod on different node (high availability)

If only 2 nodes: 3rd pod stays Pending

```

---

**Preferred Pod Anti-Affinity** (soft):

```yaml
spec:
affinity:
podAntiAffinity:
preferredDuringSchedulingIgnoredDuringExecution:
- weight: 100
podAffinityTerm:
labelSelector:
matchExpressions:
- key: app
operator: In
values:
- web
topologyKey: kubernetes.io/hostname
```

Behavior:

```
Preferred anti-affinity:
- Try to spread pods
- If not enough nodes, allow multiple pods per node
- Better than failing to schedule
```
```

Zone-Based Anti-Affinity (multi-AZ):

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
name: database
spec:
replicas: 3
selector:
matchLabels:
app: db

template:
metadata:
labels:
app: db
spec:
affinity:
podAntiAffinity:
requiredDuringSchedulingIgnoredDuringExecution:
- labelSelector:
matchLabels:
app: db
topologyKey: topology.kubernetes.io/zone

containers:
- name: postgres
image: postgres:14
```

Result:

```
3 database replicas spread across zones:
        db-pod-1 → us-east-1a
        db-pod-2 → us-east-1b
        db-pod-3 → us-east-1c

High availability across availability zones
```

8.4 Taints and Tolerations

Purpose: Repel pods from nodes unless they tolerate the taint

Taint Effects:

```
NoSchedule:
- New pods not scheduled unless they tolerate
- Existing pods not evicted

PreferNoSchedule:
- Try to avoid scheduling
- Schedule if no other option

NoExecute:
- New pods not scheduled unless they tolerate
- Existing pods without toleration evicted
```

Taint Nodes:

```bash
# Add taint
```

```
kubectl taint nodes worker-1 key=value:NoSchedule

# Examples
kubectl taint nodes worker-1 dedicated=gpu:NoSchedule
kubectl taint nodes worker-2 env=production:NoSchedule
kubectl taint nodes worker-3 workload=batch:NoExecute

# View taints
kubectl describe node worker-1 | grep Taints

# Remove taint
kubectl taint nodes worker-1 dedicated=gpu:NoSchedule-
```

Pod with Toleration:

```yaml
apiVersion: v1
kind: Pod
metadata:
name: gpu-pod
spec:
tolerations:
- key: dedicated
operator: Equal
value: gpu
effect: NoSchedule

containers:
- name: cuda-app
image: nvidia/cuda
```

Toleration Operators:

```yaml
```

```
# Equal: key=value must match
tolerations:
- key: dedicated
operator: Equal
value: gpu
effect: NoSchedule

# Exists: key must exist (any value)
tolerations:
- key: dedicated
operator: Exists
effect: NoSchedule

# Tolerate all taints
tolerations:
- operator: Exists
```

Scheduling Logic:

```
Node: worker-1
Taint: dedicated=gpu:NoSchedule

Pod 1 (no toleration):
✗ Cannot schedule on worker-1

Pod 2 (with toleration):
✓ Can schedule on worker-1

Note: Toleration allows scheduling, doesn't guarantee it
Use nodeSelector/affinity to force scheduling on tainted node
```

Taint + Node Affinity Pattern (dedicate nodes):

```bash
# 1. Taint GPU node
kubectl taint nodes gpu-node-1 dedicated=gpu:NoSchedule

# 2. Label GPU node
kubectl label nodes gpu-node-1 gpu=nvidia
```

```yaml
# Pod tolerates taint AND targets labeled node
apiVersion: v1
kind: Pod
metadata:
name: gpu-workload
spec:
# Tolerate taint
tolerations:
- key: dedicated
operator: Equal
value: gpu
effect: NoSchedule

# Target GPU node
nodeSelector:
gpu: nvidia

containers:
- name: cuda-app
image: nvidia/cuda
```

Result:

```
GPU nodes: Only GPU workloads scheduled (taint repels others)
GPU workloads: Only scheduled on GPU nodes (nodeSelector)
Perfect isolation!
```

NoExecute Taint (evict existing pods):

```bash
kubectl taint nodes worker-1 maintenance=true:NoExecute
```

```yaml
apiVersion: v1
kind: Pod
metadata:
name: critical-pod
spec:
tolerations:
- key: maintenance
operator: Equal
value: "true"
effect: NoExecute
tolerationSeconds: 3600 # Tolerate for 1 hour, then evict

containers:
- name: app
image: myapp
```

Behavior:

```
```

1. Node tainted with NoExecute
2. Pods without toleration: Evicted immediately
3. Pods with toleration: Continue running
4. If tolerationSeconds specified: Evicted after timeout
```

---

**Control Plane Taint**:
```bash
kubectl describe node master-node | grep Taints

# Output:
# Taints: node-role.kubernetes.io/control-plane:NoSchedule
```

DaemonSet Tolerating Control Plane:
```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
name: monitoring-agent
spec:
selector:
matchLabels:
app: monitoring

template:
metadata:
labels:
app: monitoring
spec:
# Tolerate control plane taint

```
tolerations:
- key: node-role.kubernetes.io/control-plane
effect: NoSchedule
- key: node-role.kubernetes.io/master # Older versions
effect: NoSchedule

containers:
- name: agent
image: monitoring-agent:latest
```

8.5 Pod Topology Spread Constraints

Purpose: Control pod distribution across topology (better than anti-affinity)

Basic Example:
```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
name: web-app
spec:
replicas: 6
selector:
matchLabels:
app: web

template:
metadata:
labels:
app: web
spec:
```

```
topologySpreadConstraints:
# Spread across zones
- maxSkew: 1
topologyKey: topology.kubernetes.io/zone
whenUnsatisfiable: DoNotSchedule
labelSelector:
matchLabels:
app: web

# Spread across nodes
- maxSkew: 1
topologyKey: kubernetes.io/hostname
whenUnsatisfiable: ScheduleAnyway
labelSelector:
matchLabels:
app: web

containers:
- name: nginx
image: nginx
```

Parameters:
```
maxSkew:
- Maximum allowed difference in pod count
- maxSkew=1: Difference of at most 1 pod

topologyKey:
- Label key defining topology (zone, node, rack, etc.)

whenUnsatisfiable:
- DoNotSchedule: Hard constraint (pod stays Pending)
```

- ScheduleAnyway: Soft constraint (schedule despite imbalance)

labelSelector:

- Pods to consider for spreading

```

**Spread Example**:

```

Cluster: 3 zones (us-east-1a, us-east-1b, us-east-1c)

Constraint: maxSkew=1, topologyKey=zone

Replicas: 6

Acceptable distributions:

Zone A: 2 pods

Zone B: 2 pods

Zone C: 2 pods

Skew: 2-2=0 ✓

Zone A: 3 pods

Zone B: 2 pods

Zone C: 1 pod

Skew: 3-1=2 ✗ (exceeds maxSkew=1)

```

---

**Multi-Level Spreading**:

```yaml

apiVersion: apps/v1

kind: Deployment

metadata:

name: critical-app

spec:

replicas: 12

```
selector:
matchLabels:
app: critical

template:
metadata:
labels:
app: critical
spec:
topologySpreadConstraints:
Level 1: Spread across zones (hard)
- maxSkew: 1
topologyKey: topology.kubernetes.io/zone
whenUnsatisfiable: DoNotSchedule
labelSelector:
matchLabels:
app: critical

Level 2: Spread across nodes (soft)
- maxSkew: 2
topologyKey: kubernetes.io/hostname
whenUnsatisfiable: ScheduleAnyway
labelSelector:
matchLabels:
app: critical

containers:
- name: app
image: critical-app:latest
```

Result (3 zones, 4 nodes per zone):
```
```

Zone A (4 nodes):

Node A1: 1 pod

Node A2: 1 pod

Node A3: 1 pod

Node A4: 1 pod

Zone B (4 nodes):

Node B1: 1 pod

Node B2: 1 pod

Node B3: 1 pod

Node B4: 1 pod

Zone C (4 nodes):

Node C1: 1 pod

Node C2: 1 pod

Node C3: 1 pod

Node C4: 1 pod

Perfect distribution across zones and nodes!

```

---

## **Phase 9: Resource Management**

### **9.1 Resource Requests and Limits**

**Resource Types**:

```

CPU:

- Measured in cores

- 1 core = 1000 millicores (m)

- Examples: 100m, 0.5, 2

Memory:

- Measured in bytes

- Units: Ki, Mi, Gi, Ti, Pi, Ei

- Examples: 128Mi, 1Gi, 500Mi

```

**Pod Resource Specification**:

```yaml

apiVersion: v1

kind: Pod

metadata:

name: resource-demo

spec:

containers:

- name: app

image: myapp

resources:

requests:

memory: "256Mi" # Minimum guaranteed

cpu: "500m" # 0.5 cores

limits:

memory: "512Mi" # Maximum allowed

cpu: "1000m" # 1 core

```

Requests vs Limits:

```

Requests (guaranteed):

- Scheduler uses for placement decisions

- Node must have available resources >= requests

- Pod guaranteed to get this amount

- Used for billing/QoS

Limits (maximum):

- Pod can't use more than this
- CPU: Throttled if exceeds limit
- Memory: OOMKilled if exceeds limit
- Protects node from resource exhaustion

```

QoS Classes (based on requests/limits):

1. Guaranteed (highest priority):

```yaml
# Requests == Limits for all containers
apiVersion: v1
kind: Pod
metadata:
name: qos-guaranteed
spec:
containers:
- name: app
image: myapp
resources:
requests:
memory: "256Mi"
cpu: "500m"
limits:
memory: "256Mi" # Same as requests
cpu: "500m" # Same as requests
```

2. Burstable (medium priority):

```yaml

```
Requests < Limits OR only requests specified
apiVersion: v1
kind: Pod
metadata:
name: qos-burstable
spec:
containers:
- name: app
image: myapp
resources:
requests:
memory: "128Mi"
cpu: "250m"
limits:
memory: "512Mi" # Higher than requests
cpu: "1000m"
```

3. BestEffort (lowest priority):

```yaml
# No requests or limits specified
apiVersion: v1
kind: Pod
metadata:
name: qos-besteffort
spec:
containers:
- name: app
image: myapp
# No resources specified
```

QoS Eviction Order (under resource pressure):
```
Node running out of memory:

1. Kill BestEffort pods first (lowest priority)
2. Kill Burstable pods exceeding requests
3. Kill Guaranteed pods (last resort)

Within same QoS: Kill pod using most resources
```

CPU Behavior:
```
Pod with limits.cpu: 1000m (1 core)

Pod tries to use 2 cores:
- Linux CFS (Completely Fair Scheduler) throttles
- Pod gets max 1 core of CPU time
- No eviction, just slower performance

cgroup settings:
cpu.cfs_quota_us: 100000 (limit)
cpu.cfs_period_us: 100000
= 1 core (100000/100000)
```

Memory Behavior:
```
Pod with limits.memory: 512Mi

Pod allocates 600Mi:
- OOMKiller invoked
```

- Container killed
- kubelet restarts container (based on restartPolicy)
- Pod shows OOMKilled status
- Restart count increments
```

**Check Pod Resources**:
```bash
kubectl describe pod resource-demo

# Output shows:
# Containers:
# app:
# Limits:
# cpu: 1
# memory: 512Mi
# Requests:
# cpu: 500m
# memory: 256Mi

kubectl top pod resource-demo

# Output:
# NAME CPU(cores) MEMORY(bytes)
# resource-demo 450m 300Mi
```

9.2 LimitRange

Purpose: Set default and max/min resource limits per namespace

LimitRange Example:
```

```yaml
apiVersion: v1
kind: LimitRange
metadata:
name: resource-limits
namespace: development
spec:
limits:
# Container limits
- type: Container
max:
cpu: "2"
memory: "4Gi"
min:
cpu: "100m"
memory: "64Mi"
default: # Default limits (if not specified)
cpu: "500m"
memory: "512Mi"
defaultRequest: # Default requests (if not specified)
cpu: "250m"
memory: "256Mi"
maxLimitRequestRatio:
cpu: 4 # limit / request <= 4
memory: 2

# Pod limits (sum of all containers)
- type: Pod
max:
cpu: "4"
memory: "8Gi"
```

```
# PVC limits
- type: PersistentVolumeClaim
max:
storage: "100Gi"
min:
storage: "1Gi"
```

Effect on Pods:

```yaml
# Pod without resources specified
apiVersion: v1
kind: Pod
metadata:
name: auto-resources
namespace: development
spec:
containers:
- name: app
image: myapp
# No resources specified

# LimitRange automatically injects:
# resources:
# requests:
# cpu: 250m
# memory: 256Mi
# limits:
# cpu: 500m
# memory: 512Mi
```

Validation:

```yaml
# Pod exceeding max
apiVersion: v1
kind: Pod
metadata:
name: exceeds-max
namespace: development
spec:
containers:
- name: app
image: myapp
resources:
limits:
cpu: "5" # Exceeds max 2 cores

# Error: Pod rejected
# Error: [maximum cpu usage per Container is 2, but limit is 5]
```

9.3 ResourceQuota

Purpose: Limit total resource usage per namespace

ResourceQuota Example:
```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
name: compute-quota
namespace: development
spec:
```

hard:

Compute resources

requests.cpu: "10" # Total CPU requests

requests.memory: "20Gi" # Total memory requests

limits.cpu: "20"

limits.memory: "40Gi"

Object counts

pods: "50"

services: "10"

services.loadbalancers: "2"

persistentvolumeclaims: "20"

requests.storage: "500Gi"

Storage classes

slow.storageclass.storage.k8s.io/requests.storage: "100Gi"

fast.storageclass.storage.k8s.io/requests.storage: "50Gi"

```

**Quota Enforcement**:

```

Namespace: development

Quota: requests.cpu: 10 cores

Current usage: 8 cores requested

New pod requests: 3 cores

8 + 3 = 11 > 10

→ Pod rejected with error:

"exceeded quota: compute-quota, requested: requests.cpu=3,

used: requests.cpu=8, limited: requests.cpu=10"

```

**Check Quota Usage**:
```

```bash
kubectl get resourcequota -n development

# Output:
# NAME AGE REQUEST LIMIT
# compute-quota 10m pods: 12/50, requests.cpu: 8/10, requests.memory: 15Gi/20Gi

kubectl describe resourcequota compute-quota -n development

# Output shows used vs hard limits for each resource
```

Scoped ResourceQuota (per priority class):

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
name: high-priority-quota
namespace: production
spec:
hard:
requests.cpu: "20"
requests.memory: "40Gi"

scopeSelector:
matchExpressions:
- operator: In
scopeName: PriorityClass
values:
- high-priority
```

```
apiVersion: v1
kind: ResourceQuota
metadata:
name: low-priority-quota
namespace: production
spec:
hard:
requests.cpu: "5"
requests.memory: "10Gi"

scopeSelector:
matchExpressions:
- operator: In
scopeName: PriorityClass
values:
- low-priority
```

9.4 Resource Inspection

Node Resources:

```bash
# View node capacity
kubectl describe node worker-1

# Output:
# Capacity:
# cpu: 4
# memory: 16Gi
# pods: 110
# Allocatable:
```

```
# cpu: 3800m (200m reserved for system)
# memory: 15Gi (1Gi reserved)
# pods: 110
#
# Allocated resources:
# Resource Requests Limits
# cpu 2500m (65%) 5000m (131%)
# memory 8Gi (53%) 12Gi (80%)
```

Top Commands:

```bash
# Node resource usage (requires metrics-server)
kubectl top nodes

# Output:
# NAME CPU(cores) CPU% MEMORY(bytes) MEMORY%
# worker-1 1200m 30% 8Gi 50%
# worker-2 800m 20% 6Gi 40%

# Pod resource usage
kubectl top pods -n production

# Pod usage by namespace
kubectl top pods --all-namespaces --sort-by=memory

# Container-level usage
kubectl top pod my-pod --containers
```

Metrics Server Installation:

```bash
kubectl apply -f
https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```

```
# Verify
kubectl get deployment metrics-server -n kube-system
```

Resource Monitoring with kubectl:

```bash
# Pods exceeding memory requests
kubectl get pods --all-namespaces -o json | \
jq '.items[] | select(.status.containerStatuses[].restartCount > 0) |
{name: .metadata.name, namespace: .metadata.namespace, restarts:
.status.containerStatuses[].restartCount}'

# Pods without resource limits
kubectl get pods --all-namespaces -o json | \
jq '.items[] | select(.spec.containers[].resources.limits == null) |
{name: .metadata.name, namespace: .metadata.namespace}'

# Total resource requests per namespace
kubectl get pods --all-namespaces -o json | \
jq -r '.items | group_by(.metadata.namespace) |
map({namespace: .[0].metadata.namespace,
cpu: map(.spec.containers[].resources.requests.cpu // "0") | add,
memory: map(.spec.containers[].resources.requests.memory // "0") | add}) |
.[]'
```

This completes the second major section. Would you like me to continue with:

- Autoscaling (HPA, VPA, Cluster Autoscaler)
- Kubernetes Networking Deep Dive

- Network Policies
- Security (RBAC, ServiceAccounts, Certificates)
- Cluster Maintenance
- Troubleshooting
- Advanced kubectl & Helm

Let me know and I'll continue with the same in-depth approach!

Kubernetes Deep Dive - Part 3: Autoscaling, Networking, Security & Operations

Phase 10: Autoscaling

10.1 Horizontal Pod Autoscaler (HPA)

Purpose: Automatically scale number of pod replicas based on metrics

HPA Architecture:

```
        ╔═══════════════════════════════════════════╗
        ║  HPA Controller (control plane)        ║
        ║  - Runs every 15 seconds (default)     ║
        ║  - Queries metrics API                 ║
        ║  - Calculates desired replicas         ║
        ║  - Updates Deployment/ReplicaSet       ║
        ╚═══════════════════════════════════════════╝
        ↓ queries
        ╔═══════════════════════════════════════════╗
        ║  Metrics Server / Custom Metrics API  ║
```

```
        ■  - Collects pod/node metrics           ■
        ■  - Aggregates data                     ■
        ■■■
```

requests:

cpu: 200m # Required for CPU-based HPA

memory: 128Mi

limits:

cpu: 500m

memory: 256Mi

```

**HPA Definition**:

```yaml

apiVersion: autoscaling/v2

kind: HorizontalPodAutoscaler

metadata:

name: web-app-hpa

spec:

scaleTargetRef:

apiVersion: apps/v1

kind: Deployment

name: web-app

minReplicas: 2

maxReplicas: 10

metrics:

- type: Resource

resource:

name: cpu

target:

type: Utilization

averageUtilization: 70 # Target 70% CPU

behavior: # v2 feature: scaling behavior

scaleDown:

```
stabilizationWindowSeconds: 300 # Wait 5 min before scale down
policies:
- type: Percent
value: 50 # Scale down max 50% of pods
periodSeconds: 60 # Per minute
- type: Pods
value: 2 # Or max 2 pods
periodSeconds: 60
selectPolicy: Min # Use the minimum (more conservative)

scaleUp:
stabilizationWindowSeconds: 0 # Scale up immediately
policies:
- type: Percent
value: 100 # Double pods
periodSeconds: 60
- type: Pods
value: 4 # Or add 4 pods
periodSeconds: 60
selectPolicy: Max # Use maximum (more aggressive)
```

Create HPA (Imperative):

```bash
kubectl autoscale deployment web-app \
--cpu-percent=70 \
--min=2 \
--max=10
```

HPA Algorithm:

```
Desired Replicas = ceil[currentReplicas * (currentMetricValue / targetMetricValue)]

Example:
Current replicas: 4
Current CPU usage: 200m average per pod
CPU request: 200m per pod
Current utilization: 200m/200m = 100%
Target utilization: 70%

Desired = ceil[4 * (100 / 70)]
= ceil[4 * 1.428]
= ceil[5.712]
= 6 replicas

HPA scales from 4 to 6 replicas
```

Detailed Workflow:
```
1. HPA controller runs (every 15s)
2. Queries metrics-server for pod CPU usage
3. Gets metrics for all pods in deployment
- Pod 1: 180m CPU
- Pod 2: 220m CPU
- Pod 3: 190m CPU
- Pod 4: 210m CPU
Average: (180+220+190+210)/4 = 200m

4. Calculates utilization
- CPU request per pod: 200m
- Average utilization: 200m/200m = 100%

5. Compares to target (70%)
```

```
        - 100% > 70% → Need to scale up
```

6. Calculates desired replicas

- Desired = ceil[4 * (100/70)] = 6

7. Updates Deployment spec.replicas to 6

8. Deployment controller creates 2 new pods

9. Wait for new pods to be Ready

10. Next iteration (15s later) recalculates with 6 pods

```

---

**Check HPA Status**:

```bash
kubectl get hpa

# Output:
# NAME REFERENCE TARGETS MINPODS MAXPODS REPLICAS AGE
# web-app-hpa Deployment/web-app 45%/70% 2 10 4 5m

# TARGETS: current/target
# 45%/70% means currently at 45% CPU, target is 70%

kubectl describe hpa web-app-hpa

# Shows:
# - Current metrics
# - Scaling events
# - Conditions
```

HPA Events:
```

```bash
kubectl get events --field-selector involvedObject.name=web-app-hpa

# Output:
# Normal SuccessfulRescale HPA New size: 6; reason: cpu resource utilization above target
# Normal SuccessfulRescale HPA New size: 4; reason: All metrics below target
```

Memory-Based HPA

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
name: memory-hpa
spec:
scaleTargetRef:
apiVersion: apps/v1
kind: Deployment
name: cache-service

minReplicas: 2
maxReplicas: 8

metrics:
- type: Resource
resource:
name: memory
target:
type: Utilization
averageUtilization: 80 # Target 80% memory
```

Memory HPA Considerations:
```
■■ Memory-based scaling challenges:
- Memory often doesn't decrease when load decreases
- Pods may hold memory (caches, buffers)
- Scale down might not happen automatically
- CPU-based HPA generally more reliable

Best practice: Combine CPU and memory metrics
```

Multiple Metrics HPA

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
name: multi-metric-hpa
spec:
scaleTargetRef:
apiVersion: apps/v1
kind: Deployment
name: api-service

minReplicas: 3
maxReplicas: 20

metrics:
# CPU metric
- type: Resource
resource:
```

name: cpu

target:

type: Utilization

averageUtilization: 70

Memory metric

- type: Resource

resource:

name: memory

target:

type: Utilization

averageUtilization: 80

Custom metric (requests per second)

- type: Pods

pods:

metric:

name: http_requests_per_second

target:

type: AverageValue

averageValue: "1000" # 1000 RPS per pod

```

**Multiple Metrics Logic**:

```

HPA calculates desired replicas for each metric:

```
        - CPU: 70% → needs 5 replicas
        - Memory: 60% → needs 3 replicas
        - RPS: 1200/pod → needs 8 replicas
```

HPA selects MAXIMUM: 8 replicas

(Most conservative approach - satisfies all metrics)

```
```

Custom Metrics HPA

Architecture:

```
        ┌─────────────────────────────────────────┐
        │  Application exposes metrics            │
        │  (Prometheus format)                    │
        └─────────────────────────────────────────┘
        ↓
        ┌─────────────────────────────────────────┐
        │  Prometheus scrapes metrics             │
        └─────────────────────────────────────────┘
        ↓
        ┌─────────────────────────────────────────┐
        │  Prometheus Adapter                     │
        │  - Exposes metrics to K8s API           │
        │  - custom.metrics.k8s.io                │
        └─────────────────────────────────────────┘
        ↓
        ┌─────────────────────────────────────────┐
        │  HPA queries custom metrics             │
        └─────────────────────────────────────────┘
```

Install Prometheus Adapter:

```bash
# Add Prometheus Helm repo
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
```

```
# Install Prometheus
helm install prometheus prometheus-community/kube-prometheus-stack \
--namespace monitoring --create-namespace

# Install Prometheus Adapter
helm install prometheus-adapter prometheus-community/prometheus-adapter \
--namespace monitoring \
--set prometheus.url=http://prometheus-kube-prometheus-prometheus.monitoring.svc \
--set prometheus.port=9090
```

Application with Custom Metrics:

```python
# Python Flask app exposing metrics
from flask import Flask
from prometheus_client import Counter, generate_latest, REGISTRY

app = Flask(__name__)

# Custom metric: HTTP requests
http_requests = Counter('http_requests_total', 'Total HTTP requests')

@app.route('/metrics')
def metrics():
return generate_latest(REGISTRY)

@app.route('/')
def index():
http_requests.inc() # Increment counter
return "Hello World"

if __name__ == '__main__':
app.run(host='0.0.0.0', port=8080)
```

ServiceMonitor for Prometheus:

```yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
name: app-metrics
namespace: default
spec:
selector:
matchLabels:
app: web
endpoints:
- port: http
interval: 30s
path: /metrics
```

HPA with Custom Metric:

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
name: custom-metric-hpa
spec:
scaleTargetRef:
apiVersion: apps/v1
kind: Deployment
name: web-app

minReplicas: 2
maxReplicas: 10
```

metrics:

- type: Pods

pods:

metric:

name: http_requests_per_second

target:

type: AverageValue

averageValue: "500"

```

---

#### **External Metrics HPA** (SQS, Kafka, etc.)

**Example: Scale based on SQS queue depth**:

```yaml
apiVersion: autoscaling/v2

kind: HorizontalPodAutoscaler

metadata:

name: queue-consumer-hpa

spec:

scaleTargetRef:

apiVersion: apps/v1

kind: Deployment

name: queue-consumer

minReplicas: 1

maxReplicas: 20

metrics:

- type: External

external:

metric:

name: sqs_queue_messages_visible

selector:

matchLabels:

queue_name: my-processing-queue

target:

type: AverageValue

averageValue: "30" # 30 messages per pod

```

Scaling Logic:

```

Queue has 300 messages

Target: 30 messages per pod

Desired replicas: 300 / 30 = 10 pods

Queue grows to 600 messages

Desired replicas: 600 / 30 = 20 pods (max)

Queue empties (0 messages)

Desired replicas: 0 / 30 = 0

But minReplicas: 1

Actual: 1 pod (keeps running)

```

HPA Best Practices

1. Set Resource Requests (required for CPU/memory HPA):

```yaml

resources:

requests:

cpu: 100m # Must set for CPU-based HPA

memory: 128Mi # Must set for memory-based HPA

```

2. Configure Scaling Behavior (avoid flapping):

```yaml
behavior:
scaleDown:
stabilizationWindowSeconds: 300 # Wait 5 min before scale down
policies:
- type: Percent
value: 50 # Scale down max 50% at a time
periodSeconds: 60

scaleUp:
stabilizationWindowSeconds: 0 # Scale up immediately
policies:
- type: Percent
value: 100 # Can double pods
periodSeconds: 15
```

3. Set Appropriate Min/Max:

```yaml
minReplicas: 2 # Never scale below 2 (HA)
maxReplicas: 50 # Prevent runaway scaling
```

4. Use Multiple Metrics:

```yaml
metrics:
- type: Resource
resource:
name: cpu
target:
```
```

type: Utilization

averageUtilization: 70

- type: Pods

pods:

metric:

name: http_requests_per_second

target:

type: AverageValue

averageValue: "1000"

```

**5. Monitor HPA Decisions**:

```bash

# Watch HPA

kubectl get hpa -w

# Check events

kubectl describe hpa my-hpa

# View metrics

kubectl get --raw /apis/metrics.k8s.io/v1beta1/namespaces/default/pods | jq .

```

10.2 Vertical Pod Autoscaler (VPA)

Purpose: Automatically adjust CPU/memory requests and limits

VPA vs HPA:

```

HPA:

- Scales number of pods (horizontal)

- Keeps pod size constant

- Faster response to load changes

VPA:

- Scales pod resource requests/limits (vertical)
- Keeps pod count constant
- Better for workloads with variable resource needs

Can use both: HPA for replicas, VPA for pod sizing
```

VPA Architecture:
```
        ┌─────────────────────────────────────────┐
        │  VPA Recommender                        │
        │  - Monitors resource usage              │
        │  - Calculates recommendations           │
        └─────────────────────────────────────────┘
        ↓
        ┌─────────────────────────────────────────┐
        │  VPA Updater                            │
        │  - Evicts pods that need resizing       │
        └─────────────────────────────────────────┘
        ↓
        ┌─────────────────────────────────────────┐
        │  VPA Admission Controller               │
        │  - Injects new resource values          │
        │  - Applied when pod recreated           │
        └─────────────────────────────────────────┘
```

Install VPA:
```

```bash
git clone https://github.com/kubernetes/autoscaler.git
cd autoscaler/vertical-pod-autoscaler
./hack/vpa-up.sh

# Verify
kubectl get pods -n kube-system | grep vpa
# vpa-admission-controller-...
# vpa-recommender-...
# vpa-updater-...
```

VPA Definition:

```yaml
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
name: web-app-vpa
spec:
targetRef:
apiVersion: apps/v1
kind: Deployment
name: web-app

updatePolicy:
updateMode: "Auto" # Auto | Recreate | Initial | Off

resourcePolicy:
containerPolicies:
- containerName: nginx
minAllowed:
cpu: 100m
memory: 128Mi
```

maxAllowed:

cpu: 2000m

memory: 2Gi

controlledResources: ["cpu", "memory"]

controlledValues: RequestsAndLimits # RequestsAndLimits | RequestsOnly

```

**Update Modes**:

```

Auto:

- VPA updates requests/limits automatically

- Evicts and recreates pods

- ■■ Causes downtime (pod restart)

Recreate:

- Same as Auto (legacy name)

Initial:

- VPA assigns resources only at pod creation

- No updates to running pods

- Good for Jobs/CronJobs

Off:

- VPA only provides recommendations

- No automatic updates

- Use for testing/observing

```

---

**VPA Recommendations**:

```bash

kubectl describe vpa web-app-vpa

```
Output:
Recommendation:
Container Recommendations:
Container Name: nginx
Lower Bound:
Cpu: 150m
Memory: 200Mi
Target:
Cpu: 250m
Memory: 300Mi
Uncapped Target:
Cpu: 250m
Memory: 300Mi
Upper Bound:
Cpu: 500m
Memory: 600Mi
```

Recommendation Meanings:

```
Lower Bound:
- Minimum resources for pod to run
- VPA won't recommend below this

Target:
- Recommended resources
- VPA will update to this value

Uncapped Target:
- Recommendation without maxAllowed constraints
- Shows what VPA would recommend without limits

Upper Bound:
```

- Maximum observed resource usage
- VPA won't recommend above this
```

---

**VPA Workflow Example**:
```
1. Deploy app with initial resources:
requests: cpu=100m, memory=128Mi

2. VPA Recommender monitors usage over time:
- Observes actual usage: cpu=220m, memory=280Mi
- Calculates recommendation: cpu=250m, memory=300Mi

3. VPA Updater checks if pod needs update:
- Current: 100m/128Mi
- Target: 250m/300Mi
        - Difference significant → Evict pod

4. Pod evicted and recreated

5. VPA Admission Controller intercepts pod creation:
- Injects new resources: cpu=250m, memory=300Mi

6. New pod starts with updated resources

7. Process repeats as usage patterns change
```

**VPA with PodDisruptionBudget**:
```yaml
# Prevent VPA from evicting all pods at once
apiVersion: policy/v1
kind: PodDisruptionBudget

metadata:

name: web-app-pdb

spec:

minAvailable: 1

selector:

matchLabels:

app: web

---

apiVersion: autoscaling.k8s.io/v1

kind: VerticalPodAutoscaler

metadata:

name: web-app-vpa

spec:

targetRef:

apiVersion: apps/v1

kind: Deployment

name: web-app

updatePolicy:

updateMode: Auto

# VPA respects PDB - won't evict if violates minAvailable

```

VPA + HPA Together:

```yaml

# HPA for replica scaling

apiVersion: autoscaling/v2

kind: HorizontalPodAutoscaler

metadata:

name: web-app-hpa

spec:

```
scaleTargetRef:
apiVersion: apps/v1
kind: Deployment
name: web-app
minReplicas: 2
maxReplicas: 10
metrics:
- type: Pods
pods:
metric:
name: http_requests_per_second
target:
type: AverageValue
averageValue: "1000" # Not CPU-based!

VPA for pod sizing
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
name: web-app-vpa
spec:
targetRef:
apiVersion: apps/v1
kind: Deployment
name: web-app
updatePolicy:
updateMode: Auto

■■ Important: HPA must NOT use CPU/memory metrics
        # VPA changes resources → HPA sees different utilization → conflicts
Use custom metrics (RPS, queue depth) for HPA
```

```

---

### **10.3 Cluster Autoscaler**

**Purpose**: Automatically add/remove nodes based on pod scheduling needs

**When Cluster Autoscaler Acts**:
```
Scale Up:
- Pods are Pending (unschedulable)
- Reason: Insufficient node resources
- CA adds new nodes

Scale Down:
- Node underutilized (< 50% resources used)
- All pods can be moved to other nodes
- Node empty for 10 minutes (default)
- CA removes node
```

**Cluster Autoscaler Architecture**:
```
        ┌───┐
        │ Cluster Autoscaler (Pod)   │
        │ - Watches for Pending pods   │
        │ - Calculates node needs   │
        │ - Calls cloud provider API   │
        └───┘
        ↓
        ┌───┐
        │ Cloud Provider (AWS/GCP/Azure)   │
        │ - Creates/deletes nodes   │
```
```

```
        ■  - Manages node groups/ASGs           ■
        ■■■
```

```
kind: Deployment
metadata:
name: cluster-autoscaler
namespace: kube-system
spec:
replicas: 1
selector:
matchLabels:
app: cluster-autoscaler
template:
metadata:
labels:
app: cluster-autoscaler
spec:
serviceAccountName: cluster-autoscaler
containers:
- name: cluster-autoscaler
image: k8s.gcr.io/autoscaling/cluster-autoscaler:v1.28.0
command:
- ./cluster-autoscaler
- --v=4
- --stderrthreshold=info
- --cloud-provider=aws
- --skip-nodes-with-local-storage=false
- --expander=least-waste
- --node-group-auto-discovery=asg:tag=k8s.io/cluster-autoscaler/enabled,k8s.io/cluster-autoscaler/my-cluster
- --balance-similar-node-groups
- --skip-nodes-with-system-pods=false
resources:
limits:
cpu: 100m
```

memory: 600Mi

requests:

cpu: 100m

memory: 600Mi

```

---

**Scale Up Workflow**:

```

1. User creates Deployment with 10 replicas

2. Scheduler tries to assign pods to nodes

- 7 pods scheduled successfully

- 3 pods Pending (insufficient CPU/memory)

3. Cluster Autoscaler detects Pending pods

- Analyzes resource requirements

- Determines node type needed

- Calculates minimum nodes needed

4. CA calls cloud provider API

- AWS: Increase ASG desired capacity

- Creates new EC2 instances

5. New nodes boot and join cluster

- kubelet starts

- Node registers with API server

- Node becomes Ready

6. Scheduler assigns Pending pods to new nodes

7. All 10 pods Running

Time: ~3-5 minutes (AWS EC2 boot time)

```

**Scale Down Workflow**:
```
1. Load decreases, pods deleted

2. Nodes become underutilized
- Node CPU: 30%
- Node memory: 40%

3. CA marks node as candidate for removal
- Checks if pods can be rescheduled
- Verifies PodDisruptionBudgets
- Checks for local storage pods
- Waits 10 minutes (scale-down-delay)

4. CA cordons node (no new pods)

5. CA drains node
- Evicts pods gracefully
- Pods rescheduled to other nodes

6. Node empty

7. CA calls cloud provider to delete node
- AWS: Decrease ASG capacity
- Terminates EC2 instance

8. Node removed from cluster

Time: ~10-15 minutes
```

---
```

Prevent Node Scale Down (annotations):

```yaml
# Annotate node to prevent scale down
apiVersion: v1
kind: Node
metadata:
name: worker-1
annotations:
cluster-autoscaler.kubernetes.io/scale-down-disabled: "true"
```

Prevent Pod Eviction:

```yaml
apiVersion: v1
kind: Pod
metadata:
name: important-pod
annotations:
# Prevent CA from evicting this pod
cluster-autoscaler.kubernetes.io/safe-to-evict: "false"
spec:
containers:
- name: app
image: myapp
```

Node Groups (Multiple Instance Types):

```yaml
# Cluster Autoscaler with multiple node groups
command:
- ./cluster-autoscaler
```

```
- --node-group-auto-discovery=asg:tag=k8s.io/cluster-autoscaler/enabled
- --balance-similar-node-groups=true
- --skip-nodes-with-system-pods=false
- --expander=least-waste # Choose cheapest option

# AWS ASGs:
# 1. general-purpose-asg (m5.large)
# 2. compute-optimized-asg (c5.xlarge)
# 3. memory-optimized-asg (r5.xlarge)

# CA selects appropriate ASG based on pod requirements
```

Expander Policies:
```
least-waste:
- Choose node group that wastes least resources
- Most cost-effective

most-pods:
- Choose node group that can fit most pods

priority:
- Use predefined priority order

price:
- Choose cheapest option (requires pricing data)

random:
- Random selection
```

Cluster Autoscaler Limits:

```yaml
command:
- ./cluster-autoscaler
- --max-nodes-total=100 # Max nodes in cluster
- --cores-total=0:320 # Max total cores (0=unlimited)
- --memory-total=0:1280 # Max total memory GB
- --max-node-provision-time=15m # Max time to provision node
- --scale-down-delay-after-add=10m
- --scale-down-unneeded-time=10m
- --scale-down-utilization-threshold=0.5 # 50% utilization
```

Cluster Autoscaler Best Practices:

1. Set Pod Resources (required):

```yaml
resources:
requests:
cpu: 100m
memory: 128Mi
# Without requests, CA can't determine node sizing
```

2. Use PodDisruptionBudgets:

```yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
name: app-pdb
spec:
minAvailable: 2
```

```
selector:
matchLabels:
app: web
# Prevents CA from draining too many pods at once
```

3. Avoid Local Storage:

```yaml
# Local storage prevents node scale down
# Use PVs instead of hostPath/emptyDir for important data
volumes:
- name: data
persistentVolumeClaim:
claimName: my-pvc # ✓ Allows node scale down
# Instead of:
# - name: data
# hostPath:
# path: /data # ✗ Prevents node scale down
```

4. Configure Node Groups Properly:

```bash
# Min/Max nodes per ASG
aws autoscaling update-auto-scaling-group \
--auto-scaling-group-name my-node-group \
--min-size 2 \
--max-size 10 \
--desired-capacity 3
```

5. Monitor CA Events:

```bash
kubectl logs -n kube-system deployment/cluster-autoscaler -f
```

Look for:
- Scale up decisions
- Scale down decisions
- Errors (insufficient quota, etc.)
```

---

### **10.4 Karpenter (Modern Alternative to Cluster Autoscaler)**

**Karpenter Overview**:
```
Cluster Autoscaler:
- Operates on predefined node groups
- Limited instance type flexibility
- Slower scaling decisions

Karpenter:
- Just-in-time node provisioning
- Automatically selects best instance type
- Faster scaling (seconds vs minutes)
- Consolidation (optimizes node utilization)
- Direct cloud API integration (no ASGs)
```

**Karpenter Architecture**:
```
        ┌───┐
        │ Karpenter Controller   │
        │ - Watches for unschedulable pods   │
        │ - Determines optimal instance type   │
        │ - Provisions nodes directly   │
        └───┘
```
```

```
        ↓
        ╔═══════════════════════════════════════════╗
        ║  Cloud Provider (AWS EC2)              ║
        ║  - Launches instances                  ║
        ║  - No Auto Scaling Groups needed       ║
        ╚═══════════════════════════════════════════╝
        ↓
        ╔═══════════════════════════════════════════╗
        ║  Node joins cluster                    ║
        ║  - Pod scheduled immediately           ║
        ╚═══════════════════════════════════════════╝
```

Karpenter Provisioner (defines node requirements):

```yaml
apiVersion: karpenter.sh/v1alpha5
kind: Provisioner
metadata:
name: default
spec:
# Requirements for nodes
requirements:
- key: karpenter.sh/capacity-type
operator: In
values: ["spot", "on-demand"] # Use Spot instances when possible

- key: kubernetes.io/arch
operator: In
values: ["amd64"]

- key: karpenter.k8s.aws/instance-category
```

```
operator: In
values: ["c", "m", "r"] # Compute, general, memory-optimized

- key: karpenter.k8s.aws/instance-generation
operator: Gt
values: ["5"] # Generation 5 or newer

# Limits
limits:
resources:
cpu: 1000 # Max 1000 cores
memory: 2000Gi # Max 2000Gi memory

# Node deprovisioning
ttlSecondsAfterEmpty: 30 # Delete empty nodes after 30s
ttlSecondsUntilExpired: 604800 # Refresh nodes after 7 days

# Consolidation (optimize costs)
consolidation:
enabled: true

# Provider-specific config
providerRef:
name: default

# Taints (optional)
taints:
- key: workload-type
value: batch
effect: NoSchedule

---
# AWS-specific provider config
apiVersion: karpenter.k8s.aws/v1alpha1
```

```
kind: AWSNodeTemplate
metadata:
name: default
spec:
subnetSelector:
karpenter.sh/discovery: my-cluster

securityGroupSelector:
karpenter.sh/discovery: my-cluster

instanceProfile: KarpenterNodeInstanceProfile

# User data (optional)
userData: |
#!/bin/bash
# Custom node setup

# Tags
tags:
Environment: production
ManagedBy: karpenter
```

Karpenter Workflow:
```
1. Pod created, Pending (insufficient resources)

2. Karpenter detects unschedulable pod
- Analyzes pod requirements:
* CPU: 2 cores
* Memory: 8Gi
* GPU: 1 x nvidia-tesla-v100
```

3. Karpenter selects optimal instance type

- Checks Provisioner requirements

- Evaluates AWS instance types:

* p3.2xlarge: 8 vCPU, 61Gi, 1 GPU ✓

* p3.8xlarge: 32 vCPU, 244Gi, 4 GPU (overkill)

- Chooses p3.2xlarge

4. Karpenter launches instance via EC2 API

- No ASG involved

- Directly creates EC2 instance

- Applies node configuration

5. Node joins cluster (~40 seconds)

- kubelet starts

- Node Ready

6. Pod scheduled immediately

Total time: ~60 seconds (vs 3-5 minutes with CA)

```

---

**Karpenter Consolidation** (cost optimization):

```

Scenario:

3 nodes, each 50% utilized

Total: 150% resources needed

Optimal: 2 nodes at 75% each

Karpenter Consolidation Process:

1. Identifies consolidation opportunity

2. Creates new, optimally-sized nodes

3. Drains old nodes

4. Deletes old nodes

5. Result: Same workload, fewer nodes, lower cost

Runs continuously (every 10 seconds)
```

**Karpenter vs CA Example**:
```
Workload: 10 pods requiring:
- 2 CPU, 4Gi memory each
- Total: 20 CPU, 40Gi

Cluster Autoscaler:
- Node group: m5.2xlarge (8 CPU, 32Gi)
- Needs 3 nodes (24 CPU, 96Gi)
- Waste: 4 CPU, 56Gi
- Cost: 3 x $0.384/hr = $1.15/hr

Karpenter:
- Selects: 2x m5.2xlarge + 1x m5.large
- Total: 20 CPU, 72Gi (closer match)
- Waste: 0 CPU, 32Gi
- Cost: $0.96/hr
- Savings: ~16%
```

---

**Karpenter Best Practices**:

**1. Use Multiple Provisioners** (workload separation):
```yaml
# General workloads

```
apiVersion: karpenter.sh/v1alpha5
kind: Provisioner
metadata:
name: general
spec:
requirements:
- key: karpenter.sh/capacity-type
operator: In
values: ["spot", "on-demand"]
- key: karpenter.k8s.aws/instance-category
operator: In
values: ["m", "c"]

---
# GPU workloads
apiVersion: karpenter.sh/v1alpha5
kind: Provisioner
metadata:
name: gpu
spec:
requirements:
- key: karpenter.k8s.aws/instance-category
operator: In
values: ["p", "g"] # GPU instances
- key: karpenter.sh/capacity-type
operator: In
values: ["on-demand"] # No Spot for GPU
taints:
- key: nvidia.com/gpu
effect: NoSchedule
```

2. Set Resource Limits:

```yaml
limits:
resources:
cpu: 1000
memory: 2000Gi
# Prevent runaway costs
```

3. Enable Consolidation:

```yaml
consolidation:
enabled: true
# Continuously optimize node utilization
```

4. Use Spot Instances (cost savings):

```yaml
requirements:
- key: karpenter.sh/capacity-type
operator: In
values: ["spot", "on-demand"]

# Karpenter handles Spot interruptions:
# 1. Receives 2-minute warning
# 2. Cordons node
# 3. Drains pods
# 4. Pods rescheduled to other nodes
```

Phase 11: Kubernetes Networking Deep Dive

11.1 Kubernetes Networking Model

Fundamental Requirements:

```
1. All pods can communicate with all other pods
- Without NAT
- Flat network

2. All nodes can communicate with all pods
- Without NAT

3. Pod sees its own IP
- No NAT between pod and outside world
```

Network Layers:

```
        ┌─────────────────────────────────────────┐
        │  Container-to-Container (in same pod) │
        │  - localhost communication            │
        │  - Shared network namespace           │
        └─────────────────────────────────────────┘
        ↓
        ┌─────────────────────────────────────────┐
        │  Pod-to-Pod (same node)               │
        │  - Virtual ethernet (veth) pairs      │
        │  - Bridge network                     │
        └─────────────────────────────────────────┘
        ↓
        ┌─────────────────────────────────────────┐
        │  Pod-to-Pod (different nodes)         │
        │  - Overlay network (VXLAN, etc.)      │
        │  - CNI plugin manages routing         │
```

```
        ■■■
```

- eBPF-based
- Advanced network policies
- Service mesh features
- High performance

4. Weave Net
- Mesh network
- Encryption support
- Simple setup

5. AWS VPC CNI
- Uses native AWS VPC networking
- Pods get VPC IPs
- Security groups for pods
```

---

#### **Calico Deep Dive**

**Calico Architecture**:
```
        ┌───┐
        │ Calico Node (DaemonSet)   │
        │ - Felix: Programs routes   │
        │ - BIRD: BGP routing daemon   │
        │ - confd: Watches etcd   │
        └───┘
        ↓
        ┌───┐
        │ Linux Kernel Routing Table   │
        │ - Routes to pods on other nodes   │
        └───┘
```
```

```

**Install Calico**:
```bash
kubectl apply -f https://docs.projectcalico.org/manifests/calico.yaml

# Verify
kubectl get pods -n kube-system | grep calico
# calico-kube-controllers-...
# calico-node-... (DaemonSet, one per node)

kubectl get nodes -o wide
# Should show InternalIP and PodCIDR
```

Calico IP Pool:
```yaml
apiVersion: crd.projectcalico.org/v1
kind: IPPool
metadata:
name: default-ipv4-ippool
spec:
cidr: 10.244.0.0/16 # Pod IP range
ipipMode: Never # No IP-in-IP encapsulation
natOutgoing: true # SNAT for traffic to internet
disabled: false
nodeSelector: all()
vxlanMode: CrossSubnet # VXLAN only for cross-subnet traffic
```

Calico Routing Modes:
```

1. BGP Mode (no encapsulation):

```yaml
# Calico uses BGP to advertise routes
# Node1: "I have 10.244.1.0/24"
# Node2: "I have 10.244.2.0/24"

# Routing table on Node1:
10.244.1.0/24 dev cali* scope link
10.244.2.0/24 via 192.168.1.102 dev eth0 # Node2 IP

# Pros: Best performance (no encapsulation overhead)
# Cons: Requires BGP support in network
```

2. VXLAN Mode (overlay network):

```yaml
spec:
vxlanMode: Always # Always use VXLAN

# Calico creates VXLAN tunnels between nodes
# Pod traffic encapsulated in VXLAN

# Pros: Works in any network (no BGP needed)
# Cons: Slight performance overhead (~5%)
```

3. IP-in-IP Mode:

```yaml
spec:
ipipMode: Always

# Pros: Compatible with most networks
# Cons: More overhead than VXLAN
```

Pod Network Flow (Calico):

```
        Pod A (10.244.1.5) on Node1 → Pod B (10.244.2.8) on Node2

1. Pod A sends packet to 10.244.2.8
        ↓
2. Packet hits Pod A's veth interface (cali-abc)
        ↓
3. Kernel routing (Node1):
10.244.2.0/24 via 192.168.1.102 dev eth0
        ↓
4. Packet sent to Node2 (192.168.1.102)
- VXLAN mode: Encapsulated in VXLAN
- BGP mode: Direct routing
        ↓
5. Node2 receives packet
        ↓
6. Kernel routing (Node2):
10.244.2.8 dev cali-xyz
        ↓
7. Packet delivered to Pod B
        ↓
8. Pod B sends response back
- Same process in reverse
```

Check Calico Routes:

```bash
```

View routes on node
ip route

Output:
10.244.1.0/24 dev cali* scope link
10.244.2.0/24 via 192.168.1.102 dev eth0
10.244.3.0/24 via 192.168.1.103 dev eth0

View Calico status
calicoctl node status

Output:
IPv4 BGP status
+--------------+-------------------+-------+----------+-------------+
| PEER ADDRESS | PEER TYPE | STATE | SINCE | INFO |
+--------------+-------------------+-------+----------+-------------+
| 192.168.1.102| node-to-node mesh | up | 10:30:00 | Established |
| 192.168.1.103| node-to-node mesh | up | 10:30:00 | Established |
+--------------+-------------------+-------+----------+-------------+
```

---

### **11.3 DNS in Kubernetes (CoreDNS)**

**CoreDNS Architecture**:
```
        ┌───┐
        │ Pod wants to resolve service name   │
        │ app-service.default.svc.cluster.local │
        └───┘
        ↓ (DNS query)
        ┌───┐
        │ /etc/resolv.conf in pod   │
```
```

```
        ■  nameserver 10.96.0.10  (kube-dns)     ■
        ■■■
```

```
fallthrough in-addr.arpa ip6.arpa
ttl 30
}

# Prometheus metrics
prometheus :9153

# Forward non-cluster queries
forward . /etc/resolv.conf {
max_concurrent 1000
}

# Cache
cache 30

# Detect loops
loop

# Reload config
reload

# Load balance
loadbalance
}
```

DNS Records Created:

1. Service DNS:
```
Service: my-service in namespace default

A Record:
```

```
my-service.default.svc.cluster.local → 10.96.0.20 (ClusterIP)
```

Short forms (from same namespace):

```
my-service → 10.96.0.20
my-service.default → 10.96.0.20
```

From different namespace:

my-service.default.svc.cluster.local

```

**2. Headless Service DNS**:

```

Headless Service: nginx-headless (clusterIP: None)

A Records:

```
nginx-headless.default.svc.cluster.local → 10.244.1.5, 10.244.2.8, 10.244.3.2
```

(Returns all pod IPs)

```

**3. Pod DNS** (if enabled):

```

Pod IP: 10.244.1.5

Namespace: default

A Record:

```
10-244-1-5.default.pod.cluster.local → 10.244.1.5
```

(Dots replaced with dashes in IP)

```

**4. StatefulSet Pod DNS**:

```

StatefulSet: mysql (3 replicas)

Headless Service: mysql-headless

A Records:

```
        mysql-0.mysql-headless.default.svc.cluster.local → 10.244.1.5
        mysql-1.mysql-headless.default.svc.cluster.local → 10.244.2.8
        mysql-2.mysql-headless.default.svc.cluster.local → 10.244.3.2
```

(Stable, predictable DNS names)

```

---

**DNS Resolution Flow**:

Pod queries: app-service.production.svc.cluster.local

1. Pod sends DNS query to nameserver (10.96.0.10)
        ↓
2. kube-dns Service routes to CoreDNS pod
        ↓
3. CoreDNS receives query
        ↓
4. Matches "cluster.local" zone
        ↓
5. Kubernetes plugin handles query:
- Queries API server for service "app-service" in namespace "production"
- Gets service ClusterIP: 10.100.50.10
        ↓
6. CoreDNS returns A record:
app-service.production.svc.cluster.local A 10.100.50.10
        ↓
7. Pod receives response
        ↓
8. Pod connects to 10.100.50.10
```

```

---

**Pod DNS Configuration**:
```yaml
# Automatically injected by kubelet
# /etc/resolv.conf inside pod:
nameserver 10.96.0.10
search default.svc.cluster.local svc.cluster.local cluster.local
options ndots:5

# nameserver: CoreDNS service IP
# search: DNS search domains
# ndots: If query has < 5 dots, try search domains first
```

ndots Behavior:
```
Query: app-service (0 dots)
1. Try: app-service.default.svc.cluster.local ✓ (found)
2. Return: 10.96.0.20

Query: google.com (1 dot < 5 dots)
1. Try: google.com.default.svc.cluster.local (fail)
2. Try: google.com.svc.cluster.local (fail)
3. Try: google.com.cluster.local (fail)
4. Try: google.com ✓ (found)
5. Forward to external DNS

■■ Problem: 4 unnecessary queries!

Solution: Use FQDN with trailing dot
google.com. (has 2 dots, but ends with ., skips search)
```
```

```

---

**Custom DNS Configuration**:
```yaml
apiVersion: v1
kind: Pod
metadata:
name: custom-dns-pod
spec:
# Custom DNS policy
dnsPolicy: "None" # ClusterFirst | Default | ClusterFirstWithHostNet | None

dnsConfig:
nameservers:
- 1.1.1.1
- 8.8.8.8
searches:
- custom.domain.com
options:
- name: ndots
value: "2"
- name: timeout
value: "3"

containers:
- name: app
image: myapp
```

DNS Policies:
```
```
```

ClusterFirst (default):

- Use cluster DNS (CoreDNS)
- /etc/resolv.conf points to kube-dns

Default:

- Inherit DNS from node
- Use node's /etc/resolv.conf

ClusterFirstWithHostNet:

- Use cluster DNS even with hostNetwork: true

None:

- Empty DNS config
- Must provide dnsConfig

```

---

**CoreDNS Debugging**:

```bash
# Test DNS resolution from pod
kubectl run -it --rm debug --image=busybox --restart=Never -- sh
nslookup kubernetes.default.svc.cluster.local
nslookup my-service.production

# Check CoreDNS logs
kubectl logs -n kube-system -l k8s-app=kube-dns

# CoreDNS metrics
kubectl port-forward -n kube-system svc/kube-dns 9153:9153
curl http://localhost:9153/metrics

# Common issues:
# 1. CoreDNS pod not running
```
```

```
kubectl get pods -n kube-system | grep coredns

# 2. kube-dns service misconfigured
kubectl get svc -n kube-system kube-dns

# 3. Network policy blocking DNS
# 4. Firewall blocking UDP/53
```

CoreDNS Custom Entries:

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
name: coredns-custom
namespace: kube-system
data:
example.server: |
example.com:53 {
errors
cache 30
forward . 1.1.1.1 8.8.8.8
}

custom.override: |
rewrite name my-service.production.svc.cluster.local legacy-service.default.svc.cluster.local
```

11.4 Service Mesh (Brief Introduction)

Service Mesh Purpose:

```
Problems:
- Complex service-to-service communication
- Difficult to implement retries, timeouts, circuit breaking
- Hard to observe traffic
- Mutual TLS management

Service Mesh Solution:
- Sidecar proxy in each pod
- Handles all network communication
- Traffic management, security, observability
```

Popular Service Meshes:

```
1. Istio
- Full-featured
- Envoy proxy
- Complex to operate

2. Linkerd
- Lightweight
- Simple setup
- Rust-based proxy

3. Consul
- HashiCorp
- Multi-platform

4. AWS App Mesh
- AWS-managed
- Envoy-based
```

```

**Service Mesh Architecture** (Istio example):

```
        ┌───┐
        │ Pod   │
        │ ┌──────────────┐ ┌──────────────────────┐ │
        │ │   App   │←→│ Envoy Sidecar   │ │
        │ │Container │ │ Proxy   │ │
        │ └──────────────┘ └──────────────────────┘ │
        └───┘
        ↓ (telemetry, config)
        ┌───┐
        │ Istio Control Plane   │
        │ - Pilot: Traffic management   │
        │ - Citadel: Certificate management   │
        │ - Galley: Configuration   │
        └───┘
```

This completes networking fundamentals. Should I continue with Network Policies and Security (RBAC, authentication)?

# Kubernetes Deep Dive - Part 4: Network Policies, Security, RBAC & Cluster Operations

## **Phase 12: Network Policies**

### **12.1 Network Policy Fundamentals**

**Purpose**: Control traffic flow between pods/namespaces

**Default Behavior** (without NetworkPolicy):

```

All pods can communicate with all pods

- No restrictions
- Flat network
- Security risk
```

**With NetworkPolicy**:
```
Default deny all traffic
Explicitly allow specific traffic
Microsegmentation
Zero-trust networking
```

**Network Policy Requirements**:
```
CNI plugin must support NetworkPolicies:
✓ Calico
✓ Cilium
✓ Weave Net
✗ Flannel (doesn't support NetworkPolicies)
```

---

### **12.2 NetworkPolicy Structure**

**Basic NetworkPolicy**:
```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
name: deny-all
namespace: production

```
spec:
# Select pods this policy applies to
podSelector: {} # Empty = all pods in namespace

# Policy types
policyTypes:
- Ingress # Incoming traffic
- Egress # Outgoing traffic

# No ingress/egress rules = deny all
```

Effect of Above Policy:
```
All pods in "production" namespace:
- Cannot receive any incoming traffic
- Cannot send any outgoing traffic
- Isolated completely

Exception: Traffic within same pod still works
(localhost communication between containers)
```

12.3 Ingress Network Policies

Allow Traffic from Specific Pods

Scenario: Frontend pods can access backend pods

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
```

name: backend-allow-frontend

namespace: production

spec:

Apply to backend pods

podSelector:

matchLabels:

app: backend

tier: api

policyTypes:

- Ingress

ingress:

Rule 1: Allow from frontend pods

- from:

- podSelector:

matchLabels:

app: frontend

ports:

- protocol: TCP

port: 8080

```

**Traffic Flow**:

```
        ✓ frontend pod → backend pod:8080 (allowed)
        ✗ random pod → backend pod:8080   (denied)
        ✗ frontend pod → backend pod:3000 (denied, wrong port)
        ✓ backend pod → anywhere   (allowed, no egress policy)
```

---
```

Allow Traffic from Specific Namespace

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
name: db-allow-backend-namespace
namespace: database
spec:
podSelector:
matchLabels:
app: postgres

policyTypes:
- Ingress

ingress:
- from:
# Allow from entire "production" namespace
- namespaceSelector:
matchLabels:
name: production
ports:
- protocol: TCP
port: 5432
```

How It Works:
```
Namespace "production" labeled: name=production

        ✓ Any pod in "production" namespace → postgres:5432 (allowed)

        ✗ Pod in "staging" namespace → postgres:5432       (denied)
```

Label Namespace:

```bash
kubectl label namespace production name=production
```

Combine Pod and Namespace Selectors

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
name: api-strict-access
namespace: production
spec:
podSelector:
matchLabels:
app: api

policyTypes:
- Ingress

ingress:
- from:
# AND condition: Must match BOTH
- namespaceSelector:
matchLabels:
name: production
podSelector:
matchLabels:
role: backend
ports:
```

- protocol: TCP

port: 8080

```

**Logic**:

```

Allow traffic from:

Pods with label "role=backend"

AND in namespace with label "name=production"

```
✓ backend pod in production namespace → api:8080  (allowed)

✗ frontend pod in production namespace → api:8080 (denied, wrong pod label)

✗ backend pod in staging namespace → api:8080     (denied, wrong namespace)
```

```

**Multiple `from` Entries** (OR condition):

```yaml

ingress:

- from:

# Rule 1: OR

- namespaceSelector:

matchLabels:

name: production

# Rule 2: OR

- podSelector:

matchLabels:

role: monitoring

ports:

- protocol: TCP

port: 8080

```

Logic:
```

```
Allow from:
Any pod in "production" namespace
OR
Any pod with label "role=monitoring" (any namespace)
```

Allow Traffic from Specific IP Blocks (CIDR)

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
name: allow-external-cidr
namespace: production
spec:
podSelector:
matchLabels:
app: web

policyTypes:
- Ingress

ingress:
- from:
# Allow from specific IP range
- ipBlock:
cidr: 203.0.113.0/24 # External IPs
except:
- 203.0.113.50/32 # Except this IP
ports:
```

- protocol: TCP
port: 443
```

**Use Cases**:
```
- Allow traffic from corporate network
- Allow monitoring systems
- Allow load balancer health checks
- Restrict to specific cloud region IPs
```

---

#### **Multiple Rules (Different Ports)**

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
name: multi-port-policy
namespace: production
spec:
podSelector:
matchLabels:
app: api

policyTypes:
- Ingress

ingress:
# Rule 1: Frontend access to HTTP port
- from:
- podSelector:

matchLabels:

app: frontend

ports:

- protocol: TCP

port: 8080

# Rule 2: Monitoring access to metrics port

- from:

- namespaceSelector:

matchLabels:

name: monitoring

ports:

- protocol: TCP

port: 9090

# Rule 3: Admin access to debug port

- from:

- ipBlock:

cidr: 10.0.0.0/8 # Internal network

ports:

- protocol: TCP

port: 8081

```

12.4 Egress Network Policies

Deny All Egress

```yaml

apiVersion: networking.k8s.io/v1

kind: NetworkPolicy

metadata:

name: deny-all-egress

namespace: production

spec:

podSelector: {}

policyTypes:

- Egress

# No egress rules = deny all outgoing traffic

```

Effect:

```

Pods cannot:

- Access other pods

- Access services

- Access external internet

- Resolve DNS (■■ important!)

Pods still receive traffic (no Ingress policy)

```

Allow Egress to Specific Pods

```yaml

apiVersion: networking.k8s.io/v1

kind: NetworkPolicy

metadata:

name: frontend-egress

namespace: production

spec:

podSelector:

matchLabels:

app: frontend

policyTypes:

- Egress

egress:

# Allow egress to backend pods

- to:

- podSelector:

matchLabels:

app: backend

ports:

- protocol: TCP

port: 8080

# Allow DNS (required!)

- to:

- namespaceSelector:

matchLabels:

name: kube-system

podSelector:

matchLabels:

k8s-app: kube-dns

ports:

- protocol: UDP

port: 53

- protocol: TCP

port: 53

```

Why DNS is Critical:

```

Without DNS egress rule:

- Pods cannot resolve service names
- DNS queries blocked
- All service discovery fails

Always include DNS in egress policies!
```

Allow Egress to External APIs

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
name: allow-external-api
namespace: production
spec:
podSelector:
matchLabels:
app: payment-service

policyTypes:
- Egress

egress:
# Allow to payment gateway
- to:
- ipBlock:
cidr: 198.51.100.0/24 # Payment gateway IPs
ports:
- protocol: TCP
port: 443

# Allow DNS
- to:
- namespaceSelector:
matchLabels:
name: kube-system
podSelector:
matchLabels:
k8s-app: kube-dns
ports:
- protocol: UDP
port: 53

# Allow to internal database
- to:
- namespaceSelector:
matchLabels:
name: database
podSelector:
matchLabels:
app: postgres
ports:
- protocol: TCP
port: 5432
```

Allow All Egress to Internet (except internal)

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:

```
name: allow-internet-deny-internal
namespace: production
spec:
podSelector:
matchLabels:
app: web-scraper

policyTypes:
- Egress

egress:
# Allow all except internal networks
- to:
- ipBlock:
cidr: 0.0.0.0/0
except:
- 10.0.0.0/8 # Private network
- 172.16.0.0/12 # Private network
- 192.168.0.0/16 # Private network
ports:
- protocol: TCP
port: 443
- protocol: TCP
port: 80

# DNS
- to:
- namespaceSelector:
matchLabels:
name: kube-system
podSelector:
matchLabels:
k8s-app: kube-dns
```

ports:

- protocol: UDP

port: 53

```

---

### **12.5 Combined Ingress and Egress Policies**

**Complete Isolation Example**:

```yaml
        # Three-tier application: frontend → backend → database
---
# Frontend Network Policy
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
name: frontend-policy
namespace: production
spec:
podSelector:
matchLabels:
tier: frontend

policyTypes:
- Ingress
- Egress

ingress:
# Allow from internet (via ingress controller)
- from:
- namespaceSelector:
matchLabels:

```
name: ingress-nginx
ports:
- protocol: TCP
port: 8080

egress:
Allow to backend
- to:
- podSelector:
matchLabels:
tier: backend
ports:
- protocol: TCP
port: 8080

DNS
- to:
- namespaceSelector:
matchLabels:
name: kube-system
podSelector:
matchLabels:
k8s-app: kube-dns
ports:
- protocol: UDP
port: 53

Backend Network Policy
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
name: backend-policy
```

```
namespace: production
spec:
podSelector:
matchLabels:
tier: backend

policyTypes:
- Ingress
- Egress

ingress:
# Allow from frontend only
- from:
- podSelector:
matchLabels:
tier: frontend
ports:
- protocol: TCP
port: 8080

egress:
# Allow to database
- to:
- namespaceSelector:
matchLabels:
name: database
podSelector:
matchLabels:
app: postgres
ports:
- protocol: TCP
port: 5432
```

```
# DNS
- to:
- namespaceSelector:
matchLabels:
name: kube-system
podSelector:
matchLabels:
k8s-app: kube-dns
ports:
- protocol: UDP
port: 53

---
# Database Network Policy
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
name: database-policy
namespace: database
spec:
podSelector:
matchLabels:
app: postgres

policyTypes:
- Ingress
- Egress

ingress:
# Allow from production backend only
- from:
- namespaceSelector:
matchLabels:
```

name: production

podSelector:

matchLabels:

tier: backend

ports:

- protocol: TCP

port: 5432

egress:

Database needs no egress (isolated)

Only DNS for updates

- to:

- namespaceSelector:

matchLabels:

name: kube-system

podSelector:

matchLabels:

k8s-app: kube-dns

ports:

- protocol: UDP

port: 53

```

**Traffic Flow**:

```

```
Internet → Ingress Controller → Frontend:8080 ✓

Frontend → Backend:8080 ✓

Backend → Database:5432 ✓
```

Blocked:

```
Internet → Backend ✗

Frontend → Database ✗

Database → Internet ✗
```

```
        Random Pod → Any Tier ✗
```

12.6 Advanced Network Policy Patterns

Default Deny, Selective Allow

Step 1: Deny All (namespace-wide):

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
name: default-deny-all
namespace: production
spec:
podSelector: {}
policyTypes:
- Ingress
- Egress
```

Step 2: Allow DNS (namespace-wide):

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
name: allow-dns
namespace: production
spec:
podSelector: {}
policyTypes:
```

- Egress

egress:

- to:

- namespaceSelector:

matchLabels:

name: kube-system

podSelector:

matchLabels:

k8s-app: kube-dns

ports:

- protocol: UDP

port: 53

```

**Step 3: Selective Allow (per-service)**:

```yaml

apiVersion: networking.k8s.io/v1

kind: NetworkPolicy

metadata:

name: allow-web-ingress

namespace: production

spec:

podSelector:

matchLabels:

app: web

policyTypes:

- Ingress

ingress:

- from:

- podSelector:

matchLabels:

app: frontend

ports:

- protocol: TCP

port: 8080

```

Allow Same Namespace Communication

```yaml

apiVersion: networking.k8s.io/v1

kind: NetworkPolicy

metadata:

name: allow-same-namespace

namespace: production

spec:

podSelector: {}

policyTypes:

- Ingress

- Egress

ingress:

# Allow from all pods in same namespace

- from:

- podSelector: {}

egress:

# Allow to all pods in same namespace

- to:

- podSelector: {}

# DNS

- to:

- namespaceSelector:

matchLabels:

name: kube-system

podSelector:

matchLabels:

k8s-app: kube-dns

ports:

- protocol: UDP

port: 53

```

Effect:

```

Within "production" namespace:

```
        ✓ Pod A → Pod B (allowed)

        ✓ Pod B → Pod C (allowed)

Cross-namespace:

        ✗ production/Pod A → staging/Pod X (denied)
```

Allow Monitoring (Prometheus)

```yaml

apiVersion: networking.k8s.io/v1

kind: NetworkPolicy

metadata:

name: allow-prometheus-scraping

namespace: production

spec:

podSelector:
```

matchLabels:

monitoring: "true" # Pods with this label

policyTypes:

- Ingress

ingress:

Allow Prometheus to scrape metrics

- from:

- namespaceSelector:

matchLabels:

name: monitoring

podSelector:

matchLabels:

app: prometheus

ports:

- protocol: TCP

port: 9090 # Metrics port

```

---

#### **Allow Health Checks (Kubelet)**

```yaml

apiVersion: networking.k8s.io/v1

kind: NetworkPolicy

metadata:

name: allow-health-checks

namespace: production

spec:

podSelector: {} # All pods

policyTypes:

- Ingress

ingress:

# Allow kubelet health checks

- from:

- ipBlock:

cidr: 10.0.0.0/8 # Node CIDR range

ports:

- protocol: TCP

port: 8080 # Health check port

- protocol: TCP

port: 10254 # Readiness probe port

```

12.7 Network Policy Testing and Debugging

Test Network Connectivity:

```bash

# Create test pods

kubectl run -it --rm debug-source --image=busybox --restart=Never -- sh

kubectl run -it --rm debug-target --image=nginx --restart=Never

# From source pod, test connectivity

wget -O- http://debug-target:80 # Should timeout if blocked

# Test specific port

nc -zv debug-target 80

nc -zv backend-service.production.svc.cluster.local 8080

# Test DNS

nslookup backend-service.production.svc.cluster.local

```
```

Check Network Policy:

```bash
# List network policies
kubectl get networkpolicies -n production

# Describe policy
kubectl describe networkpolicy backend-allow-frontend -n production

# Check which pods policy applies to
kubectl get pods -n production --show-labels
kubectl get pods -n production -l app=backend
```

Verify Policy with Labels:

```bash
# Add label to test
kubectl label pod test-pod app=frontend

# Remove label
kubectl label pod test-pod app-

# Check current labels
kubectl get pod test-pod --show-labels
```

Network Policy Logging (Calico):

```yaml
# Enable logging for denied traffic
apiVersion: projectcalico.org/v3
kind: GlobalNetworkPolicy
metadata:
```

```
name: log-denied-traffic
spec:
selector: all()
types:
- Ingress
- Egress

# Log denied packets
ingress:
- action: Log
source:
nets:
- 0.0.0.0/0
- action: Allow # Allow after logging

egress:
- action: Log
destination:
nets:
- 0.0.0.0/0
- action: Allow
```

View Logs:

```bash
# On node, check iptables logs
journalctl -u calico-node -f | grep -i denied

# Calico logs
kubectl logs -n kube-system -l k8s-app=calico-node | grep denied
```

Common Network Policy Issues:

1. DNS Not Working:

```yaml
# Always add DNS egress rule
egress:
- to:
- namespaceSelector:
matchLabels:
name: kube-system
podSelector:
matchLabels:
k8s-app: kube-dns
ports:
- protocol: UDP
port: 53
```

2. Health Checks Failing:

```yaml
# Allow kubelet to access health check ports
ingress:
- from:
- ipBlock:
cidr: 10.0.0.0/8 # Node network
ports:
- protocol: TCP
port: 8080
- protocol: TCP
port: 10254
```

3. Service-to-Service Communication:

```yaml
# Remember: NetworkPolicy operates on pod IPs
# Service ClusterIP is just a virtual IP
# Traffic ultimately goes to pod IPs

# Wrong: Allow to service ClusterIP (doesn't work)
egress:
- to:
- ipBlock:
cidr: 10.96.0.20/32 # Service ClusterIP

# Right: Allow to pods behind service
egress:
- to:
- podSelector:
matchLabels:
app: backend
```

12.8 Network Policy Best Practices

1. Start with Default Deny:
```yaml
# Apply to all namespaces
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
name: default-deny-all
spec:
podSelector: {}
policyTypes:
```

- Ingress
- Egress
```

**2. Always Allow DNS**:
```yaml
egress:
- to:
- namespaceSelector:
matchLabels:
name: kube-system
podSelector:
matchLabels:
k8s-app: kube-dns
ports:
- protocol: UDP
port: 53
```

3. Use Namespace Labels:
```bash
kubectl label namespace production name=production env=prod
kubectl label namespace staging name=staging env=staging
kubectl label namespace kube-system name=kube-system
```

4. Organize by Layers:
```
tier: frontend
tier: backend
tier: database

Policies:

- Frontend: Allow from ingress, egress to backend
- Backend: Allow from frontend, egress to database
- Database: Allow from backend, no egress
```

5. Document Policies:
```yaml
metadata:
annotations:
description: "Allow frontend to access backend API"
owner: "platform-team"
last-updated: "2024-01-15"
```

6. Test Before Production:
```bash
# Apply to staging first
kubectl apply -f network-policy.yaml -n staging

# Test thoroughly
# Then apply to production
kubectl apply -f network-policy.yaml -n production
```

Phase 13: Kubernetes Security - RBAC

13.1 RBAC Overview

RBAC Components:
```
        ■■■

```
        ■  - User                                ■
        ■  - ServiceAccount                      ■
        ■  - Group                               ■
        ■■■
```

- Can also be bound to namespace via RoleBinding
```

---

### **13.2 Roles and RoleBindings**

#### **Creating a Role**

**Example: Read-only access to pods**:
```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
name: pod-reader
namespace: production
rules:
# Rule 1: Get, list, watch pods
- apiGroups: [""] # "" = core API group
resources: ["pods"]
verbs: ["get", "list", "watch"]

# Rule 2: Get pod logs
- apiGroups: [""]
resources: ["pods/log"]
verbs: ["get"]
```

API Groups:
```
"": Core API (pods, services, configmaps, etc.)
apps: Deployments, StatefulSets, DaemonSets
batch: Jobs, CronJobs
networking.k8s.io: Ingress, NetworkPolicy

rbac.authorization.k8s.io: Roles, RoleBindings
```

Verbs (operations):
```
get: Read single resource
list: List resources
watch: Watch for changes
create: Create new resources
update: Update existing resources
patch: Partially update resources
delete: Delete resources
deletecollection: Delete multiple resources
```

More Complex Role:
```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
name: developer
namespace: production
rules:
# Full access to deployments
- apiGroups: ["apps"]
resources: ["deployments", "replicasets"]
verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]

# Read-only access to services
- apiGroups: [""]
resources: ["services"]

verbs: ["get", "list"]

# Access to configmaps and secrets
- apiGroups: [""]
resources: ["configmaps", "secrets"]
verbs: ["get", "list", "create", "update"]

# Access to pods (including exec and logs)
- apiGroups: [""]
resources: ["pods"]
verbs: ["get", "list", "watch", "delete"]

- apiGroups: [""]
resources: ["pods/log"]
verbs: ["get"]

- apiGroups: [""]
resources: ["pods/exec"]
verbs: ["create"]

# Access to events (for debugging)
- apiGroups: [""]
resources: ["events"]
verbs: ["get", "list", "watch"]
```

Creating RoleBinding

Bind Role to User:
```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding

metadata:

name: developer-john-binding

namespace: production

subjects:

# User

- kind: User

name: john@example.com

apiGroup: rbac.authorization.k8s.io

roleRef:

kind: Role

name: developer

apiGroup: rbac.authorization.k8s.io

```

Bind Role to ServiceAccount:

```yaml

apiVersion: rbac.authorization.k8s.io/v1

kind: RoleBinding

metadata:

name: app-pod-reader-binding

namespace: production

subjects:

- kind: ServiceAccount

name: app-serviceaccount

namespace: production

roleRef:

kind: Role

name: pod-reader

apiGroup: rbac.authorization.k8s.io

```
```

Bind Role to Group:

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
name: developers-binding
namespace: production
subjects:
- kind: Group
name: developers # Group from OIDC/LDAP
apiGroup: rbac.authorization.k8s.io

roleRef:
kind: Role
name: developer
apiGroup: rbac.authorization.k8s.io
```

Multiple Subjects in RoleBinding

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
name: pod-readers
namespace: production
subjects:
# User 1
- kind: User
name: alice@example.com
apiGroup: rbac.authorization.k8s.io
```

User 2
- kind: User
name: bob@example.com
apiGroup: rbac.authorization.k8s.io

ServiceAccount
- kind: ServiceAccount
name: monitoring-sa
namespace: monitoring

Group
- kind: Group
name: sre-team
apiGroup: rbac.authorization.k8s.io

roleRef:
kind: Role
name: pod-reader
apiGroup: rbac.authorization.k8s.io
```

---

### **13.3 ClusterRoles and ClusterRoleBindings**

#### **Cluster-Scoped Resources**

**ClusterRole for Nodes**:
```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
name: node-reader

rules:

# Read nodes

- apiGroups: [""]

resources: ["nodes"]

verbs: ["get", "list", "watch"]

# Read node metrics

- apiGroups: [""]

resources: ["nodes/status"]

verbs: ["get"]

# Read persistent volumes

- apiGroups: [""]

resources: ["persistentvolumes"]

verbs: ["get", "list", "watch"]

# Read storage classes

- apiGroups: ["storage.k8s.io"]

resources: ["storageclasses"]

verbs: ["get", "list"]

```

ClusterRoleBinding:

```yaml

apiVersion: rbac.authorization.k8s.io/v1

kind: ClusterRoleBinding

metadata:

name: node-reader-binding

subjects:

- kind: User

name: admin@example.com

apiGroup: rbac.authorization.k8s.io

```
roleRef:
kind: ClusterRole
name: node-reader
apiGroup: rbac.authorization.k8s.io
```

ClusterRole with Namespace Resources

ClusterRole for cross-namespace access:

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
name: pod-reader-all-namespaces
rules:
- apiGroups: [""]
resources: ["pods"]
verbs: ["get", "list", "watch"]
```

Use with ClusterRoleBinding (all namespaces):

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
name: pod-reader-global
subjects:
- kind: User
name: monitor@example.com
apiGroup: rbac.authorization.k8s.io
```

roleRef:

kind: ClusterRole

name: pod-reader-all-namespaces

apiGroup: rbac.authorization.k8s.io

```

**Use with RoleBinding** (specific namespace):

```yaml

apiVersion: rbac.authorization.k8s.io/v1

kind: RoleBinding

metadata:

name: pod-reader-production-only

namespace: production

subjects:

- kind: User

name: dev@example.com

apiGroup: rbac.authorization.k8s.io

roleRef:

kind: ClusterRole # Using ClusterRole

name: pod-reader-all-namespaces

apiGroup: rbac.authorization.k8s.io

# This grants access ONLY to pods in "production" namespace

# Not cluster-wide!

```

Aggregated ClusterRoles

Purpose: Combine multiple ClusterRoles into one

```yaml

```
Admin ClusterRole (aggregates others)
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
name: admin-aggregate
aggregationRule:
clusterRoleSelectors:
- matchLabels:
rbac.example.com/aggregate-to-admin: "true"
rules: [] # Rules filled in by aggregation

Deployments Admin
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
name: deployments-admin
labels:
rbac.example.com/aggregate-to-admin: "true"
rules:
- apiGroups: ["apps"]
resources: ["deployments"]
verbs: ["*"]

Services Admin
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
name: services-admin
labels:
rbac.example.com/aggregate-to-admin: "true"
```

```
rules:
- apiGroups: [""]
resources: ["services"]
verbs: ["*"]
```

Built-in Aggregated Roles:
```
admin:
- Aggregates multiple admin permissions
- Namespaced resources

edit:
- Create, update, delete resources
- Cannot view/modify roles/rolebindings

view:
- Read-only access
- Cannot view secrets
```

13.4 ServiceAccounts

What are ServiceAccounts?
```
ServiceAccounts provide identity for pods
- Each pod runs with a ServiceAccount
- Default ServiceAccount created per namespace
- Used for pod-to-API server authentication
```

Default ServiceAccount:

```bash
# Each namespace has "default" ServiceAccount
kubectl get serviceaccounts -n production

# Output:
# NAME SECRETS AGE
# default 1 30d
```

Creating ServiceAccount

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
name: app-serviceaccount
namespace: production
```

Create Imperatively:
```bash
kubectl create serviceaccount app-serviceaccount -n production
```

Using ServiceAccount in Pod

```yaml
apiVersion: v1
kind: Pod
metadata:
```

```
name: app-pod
namespace: production
spec:
serviceAccountName: app-serviceaccount # Use specific SA

containers:
- name: app
image: myapp:latest
```

What Gets Mounted:

```bash
# Inside pod, check mounted token
ls /var/run/secrets/kubernetes.io/serviceaccount/

# Output:
# ca.crt # Cluster CA certificate
# namespace # Current namespace
# token # JWT token for authentication

# View token
cat /var/run/secrets/kubernetes.io/serviceaccount/token

# Use token to call API server
TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
curl -H "Authorization: Bearer $TOKEN" \
--cacert /var/run/secrets/kubernetes.io/serviceaccount/ca.crt \
https://kubernetes.default.svc/api/v1/namespaces/production/pods
```

Grant Permissions to ServiceAccount

Create Role:

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
name: pod-manager
namespace: production
rules:
- apiGroups: [""]
resources: ["pods"]
verbs: ["get", "list", "create", "delete"]

- apiGroups: [""]
resources: ["configmaps"]
verbs: ["get", "list"]
```

Bind to ServiceAccount:

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
name: app-pod-manager-binding
namespace: production
subjects:
- kind: ServiceAccount
name: app-serviceaccount
namespace: production

roleRef:
kind: Role
name: pod-manager
apiGroup: rbac.authorization.k8s.io
```

```

**Pod Can Now**:
```bash
# Inside pod
TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)

# List pods (allowed)
curl -H "Authorization: Bearer $TOKEN" \
--cacert /var/run/secrets/kubernetes.io/serviceaccount/ca.crt \
https://kubernetes.default.svc/api/v1/namespaces/production/pods

# Create configmap (denied - not in permissions)
curl -X POST -H "Authorization: Bearer $TOKEN" \
--cacert /var/run/secrets/kubernetes.io/serviceaccount/ca.crt \
https://kubernetes.default.svc/api/v1/namespaces/production/configmaps
# Response: Forbidden
```

Disable ServiceAccount Auto-Mounting

Globally for ServiceAccount:
```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
name: no-api-access
namespace: production
automountServiceAccountToken: false # Don't mount token
```

Per-Pod Override:
```

```yaml
apiVersion: v1
kind: Pod
metadata:
name: no-api-pod
spec:
serviceAccountName: app-serviceaccount
automountServiceAccountToken: false # Override SA setting

containers:
- name: app
image: myapp
```

When to Disable:
```
✓ Pod doesn't need Kubernetes API access
✓ Security best practice (least privilege)
✓ Prevent accidental API calls
```

ServiceAccount Token Projection (Kubernetes 1.20+)

Improved Security:
```yaml
apiVersion: v1
kind: Pod
metadata:
name: token-projection-pod
spec:
serviceAccountName: app-serviceaccount
```

```
containers:
- name: app
image: myapp
volumeMounts:
- name: sa-token
mountPath: /var/run/secrets/tokens
readOnly: true

volumes:
- name: sa-token
projected:
sources:
- serviceAccountToken:
path: token
expirationSeconds: 3600 # 1 hour expiration
audience: api # Intended audience
```

Benefits:
```
Old token (secret-based):
- Never expires
- Valid forever
- Security risk if leaked

New token (projected):
- Expires after 1 hour
- Automatically rotated
- Audience-bound
- More secure
```

13.5 User Authentication

Kubernetes User Types:
```
1. ServiceAccounts
- Managed by Kubernetes
- For pods

2. Normal Users
- Not managed by Kubernetes
- External authentication
- Methods: certificates, OIDC, webhooks, etc.
```

Certificate-Based Authentication

Create User Certificate:
```bash
# 1. Generate private key
openssl genrsa -out john.key 2048

# 2. Create certificate signing request (CSR)
openssl req -new -key john.key -out john.csr -subj "/CN=john/O=developers"

# CN=john: Username
# O=developers: Group

# 3. Create Kubernetes CSR object
cat <<EOF | kubectl apply -f -
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
```

```
name: john-csr
spec:
request: $(cat john.csr | base64 | tr -d '\n')
signerName: kubernetes.io/kube-apiserver-client
usages:
- client auth
EOF

# 4. Approve CSR (admin action)
kubectl certificate approve john-csr

# 5. Get signed certificate
kubectl get csr john-csr -o jsonpath='{.status.certificate}' | base64 -d > john.crt

# 6. Create kubeconfig for john
kubectl config set-credentials john \
--client-certificate=john.crt \
--client-key=john.key \
--embed-certs=true

kubectl config set-context john-context \
--cluster=my-cluster \
--user=john \
--namespace=production

# John can now use:
kubectl --context=john-context get pods
```

OIDC Authentication (Google, Azure AD, Okta)

Configure API Server:

```bash
# /etc/kubernetes/manifests/kube-apiserver.yaml
spec:
containers:
- command:
- kube-apiserver
- --oidc-issuer-url=https://accounts.google.com
- --oidc-client-id=your-client-id.apps.googleusercontent.com
- --oidc-username-claim=email
- --oidc-groups-claim=groups
```

User Login Flow:
```
1. User authenticates with OIDC provider (Google)
2. Receives JWT token
3. Configures kubectl with token:

kubectl config set-credentials oidc-user \
--auth-provider=oidc \
--auth-provider-arg=idp-issuer-url=https://accounts.google.com \
--auth-provider-arg=client-id=your-client-id \
--auth-provider-arg=client-secret=your-secret \
--auth-provider-arg=refresh-token=refresh-token-here \
--auth-provider-arg=id-token=id-token-here

4. kubectl sends token to API server
5. API server validates with Google
6. Extracts username/groups from JWT
7. RBAC rules applied
```

13.6 Testing RBAC Permissions

Check if User Can Perform Action:

```bash
# Can I create deployments in production?
kubectl auth can-i create deployments -n production

# Output: yes or no

# Can I delete nodes? (cluster-scoped)
kubectl auth can-i delete nodes

# Can specific user do something?
kubectl auth can-i get pods \
--as john@example.com \
-n production

# Can service account do something?
kubectl auth can-i list secrets \
--as system:serviceaccount:production:app-serviceaccount \
-n production

# List all permissions for current user
kubectl auth can-i --list -n production
```

Impersonate User (testing):

```bash
# Run command as different user
kubectl get pods -n production \
--as john@example.com
```

```
# Run as service account
kubectl get pods -n production \
--as system:serviceaccount:production:app-serviceaccount

# Run as group
kubectl get pods -n production \
--as john@example.com \
--as-group developers
```

Audit Effective Permissions:

```bash
# Install kubectl-who-can plugin
kubectl krew install who-can

# Who can create pods?
kubectl who-can create pods -n production

# Output shows all subjects (users/SAs/groups) with permission

# Who can delete nodes?
kubectl who-can delete nodes

# Who can get secrets in namespace?
kubectl who-can get secrets -n production
```

13.7 Common RBAC Patterns

1. Namespace Admin

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
name: namespace-admin
namespace: production
subjects:
- kind: User
name: team-lead@example.com
apiGroup: rbac.authorization.k8s.io

roleRef:
kind: ClusterRole
name: admin # Built-in admin role
apiGroup: rbac.authorization.k8s.io
```

Admin Can:
```
✓ Create, update, delete most resources
✓ Manage RBAC within namespace
✓ View secrets

Cannot:
✗ Delete namespace
✗ Modify resource quotas
✗ Modify limit ranges
```

---

2. Read-Only User

```yaml
```

apiVersion: rbac.authorization.k8s.io/v1

kind: RoleBinding

metadata:

name: view-only

namespace: production

subjects:

- kind: User

name: auditor@example.com

apiGroup: rbac.authorization.k8s.io

roleRef:

kind: ClusterRole

name: view # Built-in view role

apiGroup: rbac.authorization.k8s.io

```

**View Can**:

```

✓ Get, list, watch most resources

Cannot:

✗ Create, update, delete anything

✗ View secrets

```

---

#### **3. CI/CD Pipeline ServiceAccount**

```yaml

# ServiceAccount

apiVersion: v1

kind: ServiceAccount

metadata:

```
name: cicd-deployer
namespace: production

Role
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
name: cicd-deployer-role
namespace: production
rules:
Manage deployments
- apiGroups: ["apps"]
resources: ["deployments"]
verbs: ["get", "list", "create", "update", "patch"]

Manage services
- apiGroups: [""]
resources: ["services"]
verbs: ["get", "list", "create", "update", "patch"]

Manage configmaps
- apiGroups: [""]
resources: ["configmaps"]
verbs: ["get", "list", "create", "update"]

Read secrets (for pulling images)
- apiGroups: [""]
resources: ["secrets"]
verbs: ["get"]

View pods (for checking deployment status)
- apiGroups: [""]
```

```
resources: ["pods"]
verbs: ["get", "list"]

- apiGroups: [""]
resources: ["pods/log"]
verbs: ["get"]

---
# RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
name: cicd-deployer-binding
namespace: production
subjects:
- kind: ServiceAccount
name: cicd-deployer
namespace: production

roleRef:
kind: Role
name: cicd-deployer-role
apiGroup: rbac.authorization.k8s.io
```

Use in CI/CD:

```bash
# Get ServiceAccount token
SA_TOKEN=$(kubectl get secret \
$(kubectl get sa cicd-deployer -n production -o jsonpath='{.secrets[0].name}') \
-n production -o jsonpath='{.data.token}' | base64 -d)

# Configure in CI/CD (GitHub Actions, GitLab CI, etc.)
```

```
kubectl config set-credentials cicd-deployer --token=$SA_TOKEN
kubectl config set-context cicd-context \
--cluster=production-cluster \
--user=cicd-deployer \
--namespace=production
```

---

4. Monitoring ServiceAccount

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
name: prometheus
namespace: monitoring

---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
name: prometheus
rules:
# Read nodes
- apiGroups: [""]
resources: ["nodes", "nodes/metrics"]
verbs: ["get", "list"]

# Read pods
- apiGroups: [""]
resources: ["pods"]
verbs: ["get", "list", "watch"]
```

```
# Read services and endpoints
- apiGroups: [""]
resources: ["services", "endpoints"]
verbs: ["get", "list", "watch"]

# Access metrics
- nonResourceURLs: ["/metrics"]
verbs: ["get"]

---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
name: prometheus
subjects:
- kind: ServiceAccount
name: prometheus
namespace: monitoring

roleRef:
kind: ClusterRole
name: prometheus
apiGroup: rbac.authorization.k8s.io
```

13.8 RBAC Security Best Practices

1. Principle of Least Privilege:

```yaml
# Bad: Give admin to everyone
roleRef:
kind: ClusterRole
```

name: cluster-admin # ✗ Too broad

Good: Specific permissions
rules:
- apiGroups: ["apps"]
resources: ["deployments"]
verbs: ["get", "list", "update"] # ✓ Only what's needed
```

**2. Avoid Wildcard Permissions**:
```yaml
# Bad
rules:
- apiGroups: ["*"] # All API groups
resources: ["*"] # All resources
verbs: ["*"] # All actions

# Good
rules:
- apiGroups: ["apps"]
resources: ["deployments", "statefulsets"]
verbs: ["get", "list", "create", "update"]
```

3. Use ServiceAccounts, Not User Credentials in Pods:
```yaml
# Bad: Mounting user kubeconfig in pod
# Good: Use ServiceAccount with minimal permissions
```

4. Regularly Audit Permissions:
```bash
# List all RoleBindings

```
kubectl get rolebindings --all-namespaces

# List all ClusterRoleBindings
kubectl get clusterrolebindings

# Check who has cluster-admin
kubectl get clusterrolebindings -o json | \
jq '.items[] | select(.roleRef.name=="cluster-admin") | .metadata.name'
```

5. Separate Roles by Environment:

```yaml
# production-admin: Limited permissions in production
# staging-admin: More permissions in staging
# dev-admin: Full admin in dev
```

6. Time-Limited Access (use external tools):

```
Tools:
- Teleport
- Vault
- AWS IAM Roles for Service Accounts (IRSA)

Provide temporary, time-limited credentials
```

This covers Network Policies and RBAC comprehensively. Should I continue with:

- Admission Controllers
- Image Security
- Cluster Maintenance (upgrades, backups)
- Troubleshooting

- Advanced kubectl & Helm?

Let me know!

Kubernetes Deep Dive - Part 5: Admission Controllers, Security, Maintenance & Troubleshooting

Phase 14: Admission Controllers

14.1 Admission Controller Overview

What are Admission Controllers?
```
Plugins that intercept API requests AFTER authentication/authorization
but BEFORE persisting to etcd

Purpose:
- Validate requests
- Mutate (modify) requests
- Enforce policies
- Set defaults
```

Admission Control Flow:
```
        ╔═══════════════════════════════════════════╗
        ║  1. Client Request (kubectl)           ║
        ╚═══════════════════════════════════════════╝
        ↓
        ╔═══════════════════════════════════════════╗
```

```
■  2. Authentication                    ■
■     (Who are you?)                    ■
■■■
```

14.2 Built-in Admission Controllers

Check Enabled Admission Controllers:

```bash
kubectl exec -n kube-system kube-apiserver-master -- \
kube-apiserver -h | grep enable-admission-plugins

# Or check API server manifest
cat /etc/kubernetes/manifests/kube-apiserver.yaml | grep enable-admission-plugins
```

Common Built-in Admission Controllers:

1. NamespaceLifecycle
```
Prevents:
- Creating resources in non-existent namespaces
- Deleting system namespaces (default, kube-system, kube-public)
- Creating resources in terminating namespaces

Example:
kubectl create deployment nginx --image=nginx -n nonexistent
# Error: namespace "nonexistent" not found
```

---

2. LimitRanger
```
Enforces LimitRange constraints in namespace

Workflow:
1. LimitRange created in namespace
```

2. Pod created without resource limits

3. LimitRanger injects default limits

4. Pod persisted with defaults

```

**Example**:

```yaml

# LimitRange

apiVersion: v1

kind: LimitRange

metadata:

name: mem-limit-range

namespace: production

spec:

limits:

- default:

memory: 512Mi

defaultRequest:

memory: 256Mi

type: Container

---

# Pod without limits

apiVersion: v1

kind: Pod

metadata:

name: example

namespace: production

spec:

containers:

- name: app

image: nginx

# No resources specified

# After LimitRanger:

# resources:

# limits:

# memory: 512Mi

# requests:

# memory: 256Mi

```

---

3. ResourceQuota

```
Enforces ResourceQuota limits

Rejects pod creation if quota exceeded
```

Example:

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
name: compute-quota
namespace: production
spec:
hard:
requests.cpu: "10"
requests.memory: "20Gi"
pods: "50"

# Try to create 51st pod:
kubectl run pod-51 --image=nginx -n production

# Error: exceeded quota: compute-quota, requested: pods=1, used: pods=50, limited: pods=50
```

---

4. ServiceAccount
```
Automatically adds ServiceAccount to pods if not specified
Mounts ServiceAccount token

Mutating Controller
```

Example:
```yaml
# Pod definition (no serviceAccount)
apiVersion: v1
kind: Pod
metadata:
name: example
spec:
containers:
- name: nginx
image: nginx

# After ServiceAccount admission controller:
# spec:
# serviceAccountName: default # Added
# volumes:
# - name: kube-api-access-xxxxx
# projected:
# sources:
# - serviceAccountToken:

# path: token
```

---

5. DefaultStorageClass
```
Assigns default StorageClass to PVC if not specified

Mutating Controller
```

Example:
```yaml
# PVC without storageClassName
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
name: my-pvc
spec:
accessModes:
- ReadWriteOnce
resources:
requests:
storage: 10Gi
# storageClassName not specified

# After DefaultStorageClass:
# spec:
# storageClassName: gp2 # Default class added
```

```

6. PodSecurityPolicy (Deprecated, replaced by Pod Security Admission)

```
Enforces security policies on pods

Deprecated in Kubernetes 1.21
Removed in Kubernetes 1.25
Replaced by: Pod Security Admission
```

---

7. NodeRestriction

```
Limits kubelet permissions

Kubelets can only modify:
- Their own Node object
- Pods bound to their node

Cannot modify other nodes or pods
```

Example:

```bash
# Kubelet on worker-1 tries to modify worker-2
# Denied by NodeRestriction

# Kubelet can only update:
kubectl patch node worker-1 --type=merge -p '{"status":{"conditions":[...]}}'
# ✓ Allowed

kubectl patch node worker-2 --type=merge -p '{"status":{"conditions":[...]}}'
# ✗ Forbidden: node "worker-2" is forbidden: node "worker-1" cannot modify node "worker-2"
```

---

8. PersistentVolumeLabel (Cloud Providers)

```
Automatically labels PVs with zone/region from cloud provider

Example (AWS EBS):
PV created in us-east-1a
Labels added:
topology.kubernetes.io/zone: us-east-1a
topology.kubernetes.io/region: us-east-1
```

14.3 Dynamic Admission Controllers (Webhooks)

Two Types:

```
1. MutatingAdmissionWebhook
- Modifies objects
- Runs before validation

2. ValidatingAdmissionWebhook
- Validates objects
- Runs after mutation
- Can only accept/reject
```

ValidatingWebhookConfiguration

Example: Require labels on all pods:

Webhook Server (external service):

```python
# webhook-server.py
from flask import Flask, request, jsonify
import base64
import json

app = Flask(__name__)

@app.route('/validate', methods=['POST'])
def validate():
admission_review = request.json

# Extract pod from request
pod = admission_review['request']['object']
labels = pod.get('metadata', {}).get('labels', {})

# Validation: Must have 'app' and 'version' labels
if 'app' not in labels or 'version' not in labels:
return jsonify({
'apiVersion': 'admission.k8s.io/v1',
'kind': 'AdmissionReview',
'response': {
'uid': admission_review['request']['uid'],
'allowed': False,
'status': {
'message': 'Pods must have "app" and "version" labels'
}
}
})

# Allow
return jsonify({
```

```
'apiVersion': 'admission.k8s.io/v1',
'kind': 'AdmissionReview',
'response': {
'uid': admission_review['request']['uid'],
'allowed': True
}
})

if __name__ == '__main__':
app.run(host='0.0.0.0', port=8443, ssl_context=('server.crt', 'server.key'))
```

Deploy Webhook Server:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
name: pod-label-webhook
namespace: webhook-system
spec:
replicas: 2
selector:
matchLabels:
app: pod-label-webhook
template:
metadata:
labels:
app: pod-label-webhook
spec:
containers:
- name: webhook
image: pod-label-webhook:v1
```

ports:

- containerPort: 8443

---

apiVersion: v1

kind: Service

metadata:

name: pod-label-webhook

namespace: webhook-system

spec:

selector:

app: pod-label-webhook

ports:

- port: 443

targetPort: 8443

```

**ValidatingWebhookConfiguration**:

```yaml

apiVersion: admissionregistration.k8s.io/v1

kind: ValidatingWebhookConfiguration

metadata:

name: pod-label-validator

webhooks:

- name: pod-labels.example.com

clientConfig:

service:

name: pod-label-webhook

namespace: webhook-system

path: "/validate"

caBundle: <base64-encoded-CA-cert>

rules:

- operations: ["CREATE", "UPDATE"]
apiGroups: [""]
apiVersions: ["v1"]
resources: ["pods"]

admissionReviewVersions: ["v1", "v1beta1"]
sideEffects: None
timeoutSeconds: 5

failurePolicy: Fail # Fail = reject if webhook unavailable
# Ignore = allow if webhook unavailable

namespaceSelector:
matchLabels:
admission-webhooks: enabled
```

Testing:
```bash
# Label namespace
kubectl label namespace production admission-webhooks=enabled

# Create pod without required labels
kubectl run test --image=nginx -n production
# Error: Pods must have "app" and "version" labels

# Create pod with labels
kubectl run test --image=nginx -n production \
--labels="app=test,version=v1"
# Success
```

```

MutatingWebhookConfiguration

Example: Auto-inject sidecar container:

Webhook Server:

```python
@app.route('/mutate', methods=['POST'])
def mutate():
admission_review = request.json
pod = admission_review['request']['object']

# Check if sidecar already exists
containers = pod.get('spec', {}).get('containers', [])
has_sidecar = any(c.get('name') == 'logging-sidecar' for c in containers)

if not has_sidecar:
# JSON Patch to add sidecar
patch = [
{
'op': 'add',
'path': '/spec/containers/-',
'value': {
'name': 'logging-sidecar',
'image': 'fluent/fluent-bit:latest',
'volumeMounts': [{
'name': 'logs',
'mountPath': '/var/log'
}]
}
}
]

# Add volume if not exists
```

```
if 'volumes' not in pod['spec']:
patch.append({
'op': 'add',
'path': '/spec/volumes',
'value': [{
'name': 'logs',
'emptyDir': {}
}]
})

import base64
patch_base64 = base64.b64encode(json.dumps(patch).encode()).decode()

return jsonify({
'apiVersion': 'admission.k8s.io/v1',
'kind': 'AdmissionReview',
'response': {
'uid': admission_review['request']['uid'],
'allowed': True,
'patchType': 'JSONPatch',
'patch': patch_base64
}
})

# No mutation needed
return jsonify({
'apiVersion': 'admission.k8s.io/v1',
'kind': 'AdmissionReview',
'response': {
'uid': admission_review['request']['uid'],
'allowed': True
}
})
```

```

**MutatingWebhookConfiguration**:
```yaml
apiVersion: admissionregistration.k8s.io/v1
kind: MutatingWebhookConfiguration
metadata:
name: sidecar-injector
webhooks:
- name: sidecar.example.com
clientConfig:
service:
name: sidecar-injector
namespace: webhook-system
path: "/mutate"
caBundle: <base64-CA-cert>

rules:
- operations: ["CREATE"]
apiGroups: [""]
apiVersions: ["v1"]
resources: ["pods"]

admissionReviewVersions: ["v1"]
sideEffects: None
timeoutSeconds: 5
failurePolicy: Ignore # Allow pod creation if webhook fails

namespaceSelector:
matchLabels:
sidecar-injection: enabled
```
```

Result:

```yaml
# Original pod spec
apiVersion: v1
kind: Pod
metadata:
name: myapp
spec:
containers:
- name: app
image: myapp:v1

# After mutation
apiVersion: v1
kind: Pod
metadata:
name: myapp
spec:
containers:
- name: app
image: myapp:v1
- name: logging-sidecar # Added by webhook
image: fluent/fluent-bit:latest
volumeMounts:
- name: logs
mountPath: /var/log
volumes:
- name: logs
emptyDir: {}
```

14.4 Pod Security Admission

Replacement for PodSecurityPolicy (Kubernetes 1.25+)

Three Levels:
```
privileged:
- Unrestricted
- No restrictions

baseline:
- Minimally restrictive
- Prevents known privilege escalations
- Default for most workloads

restricted:
- Heavily restricted
- Follows pod hardening best practices
- Most secure
```

Three Modes:
```
enforce:
- Violations rejected

audit:
- Violations allowed but logged

warn:
- Violations allowed with warning message
```

Enable Pod Security Admission:

Namespace Labels:

```yaml
apiVersion: v1
kind: Namespace
metadata:
name: production
labels:
pod-security.kubernetes.io/enforce: restricted
pod-security.kubernetes.io/enforce-version: latest
pod-security.kubernetes.io/audit: restricted
pod-security.kubernetes.io/audit-version: latest
pod-security.kubernetes.io/warn: restricted
pod-security.kubernetes.io/warn-version: latest
```

What "restricted" Level Requires:

```yaml
apiVersion: v1
kind: Pod
metadata:
name: restricted-pod
namespace: production
spec:
securityContext:
runAsNonRoot: true # Required
runAsUser: 1000 # Non-root UID
seccompProfile:
type: RuntimeDefault # Required

containers:
- name: app
```

```
image: myapp:v1
securityContext:
allowPrivilegeEscalation: false # Required
runAsNonRoot: true
capabilities:
drop:
- ALL # Drop all capabilities
seccompProfile:
type: RuntimeDefault

# No host namespaces allowed
# hostNetwork: false (default)
# hostPID: false (default)
# hostIPC: false (default)

# No privileged containers
# privileged: false (default)

# Limited volume types allowed
volumeMounts:
- name: data
mountPath: /data

volumes:
- name: data
emptyDir: {} # Allowed
# hostPath not allowed in restricted
```

Testing Restricted Policy:

```bash
# Create namespace with restricted policy
kubectl create namespace test-restricted
```

```
kubectl label namespace test-restricted \
pod-security.kubernetes.io/enforce=restricted

# Try to create privileged pod (should fail)
kubectl run privileged-pod --image=nginx -n test-restricted \
--overrides='{"spec":{"containers":[{"name":"nginx","image":"nginx","securityContext":{"privileged":true}}]}}'

# Error: pods "privileged-pod" is forbidden: violates PodSecurity "restricted:latest":
# privileged (container "nginx" must not set securityContext.privileged=true)

# Create compliant pod
kubectl run compliant-pod --image=nginx -n test-restricted \
--overrides='{
"spec": {
"securityContext": {
"runAsNonRoot": true,
"runAsUser": 1000,
"seccompProfile": {"type": "RuntimeDefault"}
},
"containers": [{
"name": "nginx",
"image": "nginx",
"securityContext": {
"allowPrivilegeEscalation": false,
"runAsNonRoot": true,
"capabilities": {"drop": ["ALL"]},
"seccompProfile": {"type": "RuntimeDefault"}
}
}]
}
}'
# Success
```

```

---

**Exempt Namespaces** (API server config):
```yaml
# /etc/kubernetes/manifests/kube-apiserver.yaml
apiVersion: v1
kind: Pod
metadata:
name: kube-apiserver
spec:
containers:
- command:
- kube-apiserver
- --admission-control-config-file=/etc/kubernetes/admission-config.yaml

---
# /etc/kubernetes/admission-config.yaml
apiVersion: apiserver.config.k8s.io/v1
kind: AdmissionConfiguration
plugins:
- name: PodSecurity
configuration:
apiVersion: pod-security.admission.config.k8s.io/v1
kind: PodSecurityConfiguration
defaults:
enforce: "baseline"
enforce-version: "latest"
audit: "restricted"
audit-version: "latest"
warn: "restricted"
warn-version: "latest"

exemptions:

usernames: []

runtimeClasses: []

namespaces:

- kube-system

- kube-public

- kube-node-lease

```

Phase 15: Image Security

15.1 Image Vulnerabilities

Scan Images (using Trivy):

```bash

# Install Trivy

brew install trivy # macOS

apt-get install trivy # Ubuntu

# Scan image

trivy image nginx:latest

# Output:

# nginx:latest (debian 11.6)

# Total: 145 (UNKNOWN: 0, LOW: 89, MEDIUM: 34, HIGH: 20, CRITICAL: 2)

#

# ■■■

```
Scan with severity threshold
trivy image --severity HIGH,CRITICAL nginx:latest

Scan and fail if vulnerabilities found
trivy image --exit-code 1 --severity CRITICAL nginx:latest
Exit code 1 if CRITICAL vulnerabilities found

Scan local Dockerfile
trivy config Dockerfile

Scan Kubernetes manifests
trivy config deployment.yaml
```

15.2 Image Signing and Verification

Why Sign Images?
```
Problems:
- Anyone can push image to registry
- Image can be tampered
- Supply chain attacks

Solution: Image Signing
- Cryptographically sign images
- Verify signature before running
- Ensure image integrity
```

Cosign (Sigstore)

Install Cosign:

```bash
# Install
brew install cosign # macOS
# Or download from https://github.com/sigstore/cosign/releases

# Generate key pair
cosign generate-key-pair

# Output:
# cosign.key (private key)
# cosign.pub (public key)
```

Sign Image:

```bash
# Sign image
cosign sign --key cosign.key myregistry.com/myapp:v1.0.0

# Enter private key password
# Signature stored in registry

# Verify signature
cosign verify --key cosign.pub myregistry.com/myapp:v1.0.0

# Output:
# Verification for myregistry.com/myapp:v1.0.0 --
# The following checks were performed on each of these signatures:
# - The cosign claims were validated
# - The signatures were verified against the specified public key
```

Enforce Signature Verification (using admission webhook):

Install Kyverno (policy engine):

```bash
kubectl create namespace kyverno
kubectl apply -f https://raw.githubusercontent.com/kyverno/kyverno/main/config/install.yaml
```

Policy to Verify Signatures:

```yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
name: verify-image-signature
spec:
validationFailureAction: enforce # Reject unsigned images
background: false
webhookTimeoutSeconds: 30

rules:
- name: verify-signature
match:
any:
- resources:
kinds:
- Pod

verifyImages:
- imageReferences:
- "myregistry.com/*" # Apply to all images from registry

attestors:
- count: 1
```

```
entries:
- keys:
publicKeys: |-
-----BEGIN PUBLIC KEY-----
MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAE...
-----END PUBLIC KEY-----
```

Testing:
```bash
# Try to create pod with unsigned image
kubectl run unsigned --image=myregistry.com/unsigned:v1
# Error: image signature verification failed

# Create pod with signed image
kubectl run signed --image=myregistry.com/myapp:v1.0.0
# Success (signature verified)
```

15.3 Private Image Registries

Pull from Private Registry:

Create Docker Registry Secret:
```bash
kubectl create secret docker-registry regcred \
--docker-server=myregistry.com \
--docker-username=myuser \
--docker-password=mypassword \
--docker-email=myuser@example.com \
-n production
```

Use in Pod:

```yaml
apiVersion: v1
kind: Pod
metadata:
name: private-image-pod
namespace: production
spec:
imagePullSecrets:
- name: regcred # Reference secret

containers:
- name: app
image: myregistry.com/myapp:v1.0.0
```

Use in ServiceAccount (for all pods):

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
name: default
namespace: production
imagePullSecrets:
- name: regcred

# All pods using this SA automatically get imagePullSecrets
```

15.4 Image Pull Policies

```yaml
apiVersion: v1
kind: Pod
metadata:
name: image-policy-demo
spec:
containers:
- name: app
image: nginx:1.25
imagePullPolicy: Always # Always | IfNotPresent | Never
```

Policies:
```
Always:
- Always pull image from registry
- Even if present locally
- Ensures latest version
- Slower (network overhead)

IfNotPresent (default for tagged images):
- Pull only if not on node
- Faster (uses cache)
- Risk: stale images if tag reused

Never:
- Never pull from registry
- Only use local images
- Fails if image not on node
```

Default Behavior:
```
```

```
        image: nginx:latest          → imagePullPolicy: Always
        image: nginx:1.25           → imagePullPolicy: IfNotPresent
        image: nginx@sha256:abc123  → imagePullPolicy: IfNotPresent
```

15.5 Image Security Best Practices

1. Use Specific Image Tags:

```yaml
# Bad
image: nginx:latest # Unpredictable, changes

# Good
image: nginx:1.25.3 # Specific version

# Best
image: nginx@sha256:abc123... # Immutable digest
```

2. Use Minimal Base Images:

```dockerfile
# Bad: Full OS
FROM ubuntu:22.04

# Good: Minimal
FROM alpine:3.18

# Best: Distroless (no shell, package manager)
FROM gcr.io/distroless/static-debian11
```

3. Run as Non-Root:

```dockerfile
```

```
# Dockerfile
FROM nginx:alpine

# Create non-root user
RUN addgroup -g 1000 appgroup && \
adduser -D -u 1000 -G appgroup appuser

# Change ownership
RUN chown -R appuser:appgroup /var/cache/nginx /var/run /var/log/nginx

USER appuser

EXPOSE 8080
CMD ["nginx", "-g", "daemon off;"]
```

```yaml
# Pod
apiVersion: v1
kind: Pod
metadata:
name: non-root-pod
spec:
securityContext:
runAsNonRoot: true
runAsUser: 1000

containers:
- name: app
image: myapp:v1
securityContext:
allowPrivilegeEscalation: false
```

4. Scan Images in CI/CD:

```yaml
# GitHub Actions
name: Build and Scan
on: [push]
jobs:
scan:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v3

- name: Build image
run: docker build -t myapp:${{ github.sha }} .

- name: Scan with Trivy
uses: aquasecurity/trivy-action@master
with:
image-ref: myapp:${{ github.sha }}
severity: 'CRITICAL,HIGH'
exit-code: '1' # Fail build if vulnerabilities
```

5. Use Read-Only Root Filesystem:

```yaml
apiVersion: v1
kind: Pod
metadata:
name: readonly-fs-pod
spec:
containers:
- name: app
image: myapp:v1
securityContext:
```

readOnlyRootFilesystem: true

Mount writable volumes where needed

volumeMounts:

- name: tmp

mountPath: /tmp

- name: cache

mountPath: /var/cache

volumes:

- name: tmp

emptyDir: {}

- name: cache

emptyDir: {}

```

**6. Drop Unnecessary Capabilities**:

```yaml

apiVersion: v1

kind: Pod

metadata:

name: drop-caps-pod

spec:

containers:

- name: app

image: myapp:v1

securityContext:

capabilities:

drop:

- ALL # Drop all capabilities

add:

- NET_BIND_SERVICE # Only add what's needed

```
```

Phase 16: Cluster Maintenance

16.1 Node Maintenance

Draining Nodes

Purpose: Safely evict pods before node maintenance

```bash
# Drain node (evict all pods)
kubectl drain worker-1 --ignore-daemonsets --delete-emptydir-data

# Options:
# --ignore-daemonsets: Don't evict DaemonSet pods
# --delete-emptydir-data: Delete pods with emptyDir volumes
# --force: Force deletion of pods not managed by controller
# --grace-period=60: Wait time before killing pod
# --timeout=5m: Max time to wait for drain

# What happens:
# 1. Node cordoned (marked unschedulable)
# 2. Pods evicted gracefully (SIGTERM)
# 3. Pods rescheduled to other nodes
# 4. Node empty (except DaemonSets)
```

Drain Workflow:
```
kubectl drain worker-1
            ↓
1. Cordon node (kubectl cordon worker-1)
- Node marked unschedulable
```

- No new pods scheduled

↓

2. Evict pods

- Send SIGTERM to containers

- Wait for graceful shutdown (terminationGracePeriodSeconds)

- If timeout: Send SIGKILL

↓

3. Pods rescheduled

- ReplicaSets create new pods

- Scheduler places on other nodes

↓

4. Node empty

- Ready for maintenance

```

**Uncordon Node** (allow scheduling again):

```bash

# After maintenance complete

kubectl uncordon worker-1

# Node now schedulable again

```

Cordon Without Draining:

```bash

# Mark node unschedulable (don't evict existing pods)

kubectl cordon worker-1

# Use case: Prevent new pods during investigation

# Existing pods keep running

```
```

Handle Drain Issues:

PodDisruptionBudget Blocking Drain:

```bash
# Check PDBs
kubectl get pdb --all-namespaces

# Example PDB
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
name: app-pdb
spec:
minAvailable: 2 # At least 2 pods must be available
selector:
matchLabels:
app: web

# If only 2 pods total, drain will block
# Solution: Temporarily update PDB or add replicas
```

Pods with Local Storage:

```bash
# Drain fails if pods use emptyDir
kubectl drain worker-1 --ignore-daemonsets
# Error: cannot delete Pods with local storage

# Force delete (data loss!)
kubectl drain worker-1 --ignore-daemonsets --delete-emptydir-data
```

Standalone Pods (not managed by controller):

```bash

# Drain fails for pods not created by Deployment/ReplicaSet/etc

kubectl drain worker-1 --ignore-daemonsets

# Error: cannot delete Pods not managed by ReplicationController, ReplicaSet, Job, DaemonSet or StatefulSet

# Force delete

kubectl drain worker-1 --ignore-daemonsets --force

# ■■ Pods won't be recreated!

```

16.2 Kubernetes Version Upgrades

Upgrade Strategy:

```

1. Upgrade control plane

- API server

- Controller manager

- Scheduler

- etcd

2. Upgrade worker nodes

- kubelet

- kube-proxy

- Container runtime

Version Skew Policy:

- kube-apiserver: N

- kubelet: N-2 (max 2 versions behind)

- kubectl: N+1 to N-1

```

Upgrade Control Plane (kubeadm)

Check Current Version:

```bash
kubectl version --short
kubeadm version
kubelet --version
```

Check Upgrade Plan:

```bash
kubeadm upgrade plan

# Output:
# Components that must be upgraded manually after you have upgraded the control plane with 'kubeadm upgrade apply':
# COMPONENT CURRENT TARGET
# kubelet 3 x v1.27.0 v1.28.0
#
# Upgrade to the latest version in the v1.27 series:
# COMPONENT CURRENT TARGET
# kube-apiserver v1.27.0 v1.28.0
# kube-controller-manager v1.27.0 v1.28.0
# kube-scheduler v1.27.0 v1.28.0
# kube-proxy v1.27.0 v1.28.0
# CoreDNS v1.10.1 v1.10.1
# etcd 3.5.7-0 3.5.9-0
```

```
**Upgrade Steps (v1.27 → v1.28)**:
```

Step 1: Upgrade kubeadm on control plane:

```bash
# Update package list
apt update

# Check available versions
apt-cache madison kubeadm

# Upgrade kubeadm
apt-mark unhold kubeadm
apt-get update && apt-get install -y kubeadm=1.28.0-00
apt-mark hold kubeadm

# Verify
kubeadm version
```

Step 2: Drain control plane node:

```bash
kubectl drain master-node --ignore-daemonsets
```

Step 3: Upgrade control plane:

```bash
# Perform upgrade
sudo kubeadm upgrade apply v1.28.0

# What happens:
# 1. Preflight checks
# 2. Download new component images
# 3. Upgrade static pod manifests (/etc/kubernetes/manifests/)
# 4. Upgrade etcd
# 5. Upgrade certificates (if needed)
```

6. Update kubeconfig files

Output:
[upgrade/successful] SUCCESS! Your cluster was upgraded to "v1.28.0". Enjoy!
```

**Step 4: Upgrade kubelet and kubectl**:
```bash
# Upgrade kubelet
apt-mark unhold kubelet kubectl
apt-get update && apt-get install -y kubelet=1.28.0-00 kubectl=1.28.0-00
apt-mark hold kubelet kubectl

# Restart kubelet
sudo systemctl daemon-reload
sudo systemctl restart kubelet

# Verify
kubelet --version
kubectl version --short
```

Step 5: Uncordon control plane:
```bash
kubectl uncordon master-node
```

Upgrade Worker Nodes

For Each Worker Node:

Step 1: Upgrade kubeadm:
```bash

```
# On worker node
apt-mark unhold kubeadm
apt-get update && apt-get install -y kubeadm=1.28.0-00
apt-mark hold kubeadm
```

Step 2: Drain node:

```bash
# From control plane
kubectl drain worker-1 --ignore-daemonsets --delete-emptydir-data
```

Step 3: Upgrade node:

```bash
# On worker node
sudo kubeadm upgrade node

# Output:
# [upgrade] Reading configuration from the cluster...
# [upgrade] FYI: You can look at this config file with 'kubectl -n kube-system get cm kubeadm-config -o yaml'
# [upgrade] Upgrading your Static Pod-hosted control plane instance to version "v1.28.0"...
# [upgrade] Successfully upgraded your Static Pod-hosted control plane to version "v1.28.0"!
```

Step 4: Upgrade kubelet and kubectl:

```bash
apt-mark unhold kubelet kubectl
apt-get update && apt-get install -y kubelet=1.28.0-00 kubectl=1.28.0-00
apt-mark hold kubelet kubectl

sudo systemctl daemon-reload
sudo systemctl restart kubelet
```

Step 5: Uncordon node:

```bash
# From control plane
kubectl uncordon worker-1
```

Verify:

```bash
kubectl get nodes

# Output:
# NAME STATUS ROLES AGE VERSION
# master-node Ready control-plane 30d v1.28.0
# worker-1 Ready <none> 30d v1.28.0
# worker-2 Ready <none> 30d v1.28.0
```

16.3 etcd Backup and Restore

Why Backup etcd?

```
etcd stores entire cluster state:
- All resources (pods, services, deployments, etc.)
- Secrets, ConfigMaps
- RBAC policies
- Custom resources

Backup = Disaster recovery
```

Backup etcd

Manual Backup:

```bash
# Find etcd pod
kubectl get pods -n kube-system | grep etcd

# Exec into etcd pod (or run on control plane node)
ETCDCTL_API=3 etcdctl \
--endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key \
snapshot save /backup/etcd-snapshot-$(date +%Y%m%d-%H%M%S).db

# Output:
# Snapshot saved at /backup/etcd-snapshot-20240115-103000.db

# Verify snapshot
ETCDCTL_API=3 etcdctl snapshot status /backup/etcd-snapshot-20240115-103000.db

# Output:
# hash, revision, total keys, size
```

Automated Backup (CronJob):

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
name: etcd-backup
namespace: kube-system
spec:
schedule: "0 */6 * * *" # Every 6 hours
```

```
jobTemplate:
spec:
template:
spec:
hostNetwork: true
nodeName: master-node # Run on control plane

containers:
- name: etcd-backup
image: k8s.gcr.io/etcd:3.5.9-0
command:
- /bin/sh
- -c
- |
ETCDCTL_API=3 etcdctl \
--endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key \
snapshot save /backup/etcd-snapshot-$(date +%Y%m%d-%H%M%S).db

# Optionally upload to S3
aws s3 cp /backup/etcd-snapshot-$(date +%Y%m%d-%H%M%S).db \
s3://my-etcd-backups/

volumeMounts:
- name: etcd-certs
mountPath: /etc/kubernetes/pki/etcd
readOnly: true
- name: backup
mountPath: /backup

volumes:
```

- name: etcd-certs

hostPath:

path: /etc/kubernetes/pki/etcd

type: Directory

- name: backup

hostPath:

path: /var/etcd-backups

type: DirectoryOrCreate

restartPolicy: OnFailure

```

---

#### **Restore etcd**

**Disaster Scenario**: Cluster completely lost

**Step 1: Stop kube-apiserver**:

```bash

# Move static pod manifest

sudo mv /etc/kubernetes/manifests/kube-apiserver.yaml /tmp/

# Wait for API server to stop

docker ps | grep kube-apiserver # Should be empty

```

Step 2: Restore from snapshot:

```bash

# Restore snapshot to new directory

ETCDCTL_API=3 etcdctl snapshot restore /backup/etcd-snapshot-20240115-103000.db \

--data-dir=/var/lib/etcd-restore \

--initial-cluster=master-node=https://192.168.1.100:2380 \

--initial-advertise-peer-urls=https://192.168.1.100:2380 \

```
--name=master-node

Output:
member xyz restored
data directory: /var/lib/etcd-restore
```

Step 3: Update etcd configuration:

```bash
# Edit etcd manifest
sudo vi /etc/kubernetes/manifests/etcd.yaml

# Change data directory:
# volumes:
# - hostPath:
# path: /var/lib/etcd-restore # Changed from /var/lib/etcd
# type: DirectoryOrCreate
# name: etcd-data
```

Step 4: Start etcd and API server:

```bash
# etcd will start automatically (static pod)

# Start API server
sudo mv /tmp/kube-apiserver.yaml /etc/kubernetes/manifests/

# Wait for pods to start
watch kubectl get pods -n kube-system
```

Step 5: Verify restoration:

```bash
kubectl get nodes
```

kubectl get pods --all-namespaces

kubectl get deployments --all-namespaces

All resources should be restored

```

---

### **16.4 Certificate Management**

**Kubernetes Certificates**:

```

/etc/kubernetes/pki/

        ■■■ ca.crt / ca.key   # Cluster CA

        ■■■ apiserver.crt / apiserver.key   # API server

        ■■■ apiserver-kubelet-client.crt/key   # API → Kubelet

        ■■■ front-proxy-ca.crt/key   # Front proxy CA

        ■■■ front-proxy-client.crt/key   # Front proxy client

        ■■■ sa.pub / sa.key   # ServiceAccount signing

        ■■■ etcd/

        ■■■ ca.crt / ca.key   # etcd CA

        ■■■ server.crt / server.key   # etcd server

        ■■■ peer.crt / peer.key   # etcd peer

        ■■■ healthcheck-client.crt/key   # Health check

```

**Check Certificate Expiration**:

```bash

# Check all certificates

kubeadm certs check-expiration

# Output:

# CERTIFICATE EXPIRES RESIDUAL TIME CERTIFICATE AUTHORITY EXTERNALLY MANAGED

# admin.conf Jan 14, 2025 10:30 UTC 364d ca no

```
apiserver Jan 14, 2025 10:30 UTC 364d ca no
apiserver-etcd-client Jan 14, 2025 10:30 UTC 364d etcd-ca no
apiserver-kubelet-client Jan 14, 2025 10:30 UTC 364d ca no
controller-manager.conf Jan 14, 2025 10:30 UTC 364d ca no
etcd-healthcheck-client Jan 14, 2025 10:30 UTC 364d etcd-ca no
etcd-peer Jan 14, 2025 10:30 UTC 364d etcd-ca no
etcd-server Jan 14, 2025 10:30 UTC 364d etcd-ca no
front-proxy-client Jan 14, 2025 10:30 UTC 364d front-proxy-ca no
scheduler.conf Jan 14, 2025 10:30 UTC 364d ca no

CERTIFICATE AUTHORITY EXPIRES RESIDUAL TIME EXTERNALLY MANAGED
ca Jan 12, 2034 10:30 UTC 9y no
etcd-ca Jan 12, 2034 10:30 UTC 9y no
front-proxy-ca Jan 12, 2034 10:30 UTC 9y no
```

Renew Certificates:

```bash
# Renew all certificates
sudo kubeadm certs renew all

# Output:
# certificate embedded in the kubeconfig file for the admin to use and for kubeadm itself renewed
# certificate for serving the Kubernetes API renewed
# certificate the apiserver uses to access etcd renewed
# certificate for the API server to connect to kubelet renewed
# certificate embedded in the kubeconfig file for the controller manager to use renewed
# certificate for liveness probes to healthcheck etcd renewed
# certificate for etcd nodes to communicate with each other renewed
# certificate for serving etcd renewed
# certificate for the front proxy client renewed
```

```
# certificate embedded in the kubeconfig file for the scheduler manager to use renewed

# Renew specific certificate
sudo kubeadm certs renew apiserver

# Restart control plane components
sudo crictl ps | grep kube-apiserver
sudo crictl stop <container-id>
# Static pods restart automatically
```

Backup Certificates:

```bash
# Backup entire PKI directory
sudo tar -czf /backup/pki-backup-$(date +%Y%m%d).tar.gz /etc/kubernetes/pki/

# Backup to remote location
sudo tar -czf - /etc/kubernetes/pki/ | ssh backup-server "cat > /backups/k8s-pki-$(date +%Y%m%d).tar.gz"
```

Phase 17: Troubleshooting

17.1 Cluster Troubleshooting

Control Plane Issues

API Server Not Responding:

```bash
# Check API server status
kubectl get nodes
# Error: The connection to the server localhost:8080 was refused
```

```
# Check API server pod
docker ps | grep kube-apiserver
# or
crictl ps | grep kube-apiserver

# Check logs
docker logs <apiserver-container-id>
# or
crictl logs <apiserver-container-id>

# Common issues:
# 1. Certificate expired
kubeadm certs check-expiration

# 2. etcd not running
docker ps | grep etcd

# 3. Port already in use
sudo netstat -tulpn | grep 6443

# 4. Manifest syntax error
sudo cat /etc/kubernetes/manifests/kube-apiserver.yaml
# Check for YAML errors

# 5. Check kubelet logs (manages static pods)
sudo journalctl -u kubelet -f
```

---

etcd Issues:

```bash
# Check etcd health
ETCDCTL_API=3 etcdctl \
```

```
--endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key \
endpoint health

# Check etcd members
ETCDCTL_API=3 etcdctl \
--endpoints=https://127.0.0.1:2379 \
--cacert=/etc/kubernetes/pki/etcd/ca.crt \
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key \
member list

# Check etcd logs
docker logs <etcd-container-id>

# etcd data corruption:
# Restore from backup (see section 16.3)
```

Scheduler/Controller Manager Issues:

```bash
# Check if running
kubectl get pods -n kube-system | grep -E 'scheduler|controller'

# Check logs
kubectl logs -n kube-system kube-scheduler-master-node
kubectl logs -n kube-system kube-controller-manager-master-node

# Common issue: Can't connect to API server
# Check API server first
```

Restart (delete pod, static pod recreates)
kubectl delete pod -n kube-system kube-scheduler-master-node
```

---

#### **Node Troubleshooting**

**Node NotReady**:
```bash
kubectl get nodes

# Output:
# NAME STATUS ROLES AGE VERSION
# worker-1 NotReady <none> 5d v1.28.0

# Describe node
kubectl describe node worker-1

# Check conditions:
# Conditions:
# Type Status Reason
# ---- ------ ------
# MemoryPressure False KubeletHasSufficientMemory
# DiskPressure True KubeletHasDiskPressure
# PIDPressure False KubeletHasSufficientPID
# Ready False KubeletNotReady

# Common causes:
# 1. kubelet not running
ssh worker-1
sudo systemctl status kubelet

# 2. Disk full

```
df -h

# 3. CNI plugin issue
ls /etc/cni/net.d/
ls /opt/cni/bin/

# 4. Certificate issue
sudo journalctl -u kubelet | grep certificate

# 5. kubelet logs
sudo journalctl -u kubelet -f
```

Node High Resource Usage:

```bash
# Check node resources
kubectl top node worker-1

# Check pod resource usage on node
kubectl top pods --all-namespaces --field-selector spec.nodeName=worker-1

# SSH to node and check
ssh worker-1

# CPU usage
top
htop

# Memory usage
free -h

# Disk usage
df -h
```

```
du -sh /*

# Check for zombie processes
ps aux | grep defunct

# Container resource usage
docker stats
# or
crictl stats
```

17.2 Pod Troubleshooting

Pod Stuck in Pending

```bash
kubectl get pod mypod
# STATUS: Pending

# Describe pod (most important troubleshooting command)
kubectl describe pod mypod

# Look for Events section:
# Events:
# Warning FailedScheduling pod has unbound immediate PersistentVolumeClaims
```

Common Causes:

1. Insufficient Resources:
```
Events:
Warning FailedScheduling 0/3 nodes are available: 3 Insufficient cpu.
```

Solution:

- Add more nodes

- Delete unused pods

- Reduce resource requests

```

**2. Node Selector Mismatch**:

```

Events:

Warning FailedScheduling 0/3 nodes are available: 3 node(s) didn't match Pod's node affinity/selector.

Solution:

kubectl get nodes --show-labels

kubectl label nodes worker-1 disktype=ssd

```

**3. Taints Not Tolerated**:

```

Events:

Warning FailedScheduling 0/3 nodes are available: 3 node(s) had taint {key=value}, that the pod didn't tolerate.

Solution: Add toleration to pod

tolerations:

- key: key

operator: Equal

value: value

effect: NoSchedule

```

**4. PVC Not Bound**:

```

Events:

Warning FailedScheduling pod has unbound immediate PersistentVolumeClaims

Solution:

kubectl get pvc

Check PVC status

Create PV or fix StorageClass

```

---

#### **Pod in CrashLoopBackOff**

```bash

kubectl get pod mypod

# STATUS: CrashLoopBackOff

# Check logs

kubectl logs mypod

# If pod restarted, check previous logs

kubectl logs mypod --previous

# Describe pod

kubectl describe pod mypod

# Events:

# Back-off restarting failed container

```

Common Causes:

1. Application Error:

```bash

# Check logs for stack trace

kubectl logs mypod --previous

```
Example errors:
- Missing environment variable
- Can't connect to database
- Configuration error
```

2. Container Runs and Exits Immediately:

```yaml
# Wrong:
command: ["echo", "hello"] # Exits immediately

# Right (keeps running):
command: ["sleep", "3600"]
# or run actual long-running process
```

3. Liveness Probe Failing:

```bash
kubectl describe pod mypod

# Events:
# Liveness probe failed: Get http://10.244.1.5:8080/health: dial tcp 10.244.1.5:8080: connect: connection refused
# Container killed

# Solution: Fix health check endpoint or adjust probe timing
```

4. Resource Limits:

```bash
# Container killed due to OOM (Out of Memory)
kubectl describe pod mypod

# Last State:
```

Terminated:

Reason: OOMKilled

Exit Code: 137

Solution: Increase memory limit
```

---

#### **Pod in ImagePullBackOff**

```bash
kubectl get pod mypod

# STATUS: ImagePullBackOff or ErrImagePull

kubectl describe pod mypod

# Events:

# Failed to pull image "myregistry.com/myapp:v1": rpc error: code = Unknown desc = failed to pull and unpack image
```

Common Causes:

1. Image Doesn't Exist:
```
Error: manifest for myapp:v99 not found

Solution: Use correct tag

kubectl set image deployment/myapp myapp=myapp:v1
```

2. Private Registry Without Credentials:
```
Error: pull access denied

Solution: Add imagePullSecrets

kubectl create secret docker-registry regcred \

--docker-server=myregistry.com \

--docker-username=user \

--docker-password=pass

# Update pod/deployment

imagePullSecrets:

- name: regcred

```

3. Registry Unreachable:

```

Error: dial tcp: lookup myregistry.com: no such host

Solution:

- Check DNS

- Check network policies

- Check firewall rules

```

Pod Terminating (Stuck)

```bash

kubectl get pod mypod

# STATUS: Terminating (for long time)

# Force delete

kubectl delete pod mypod --grace-period=0 --force

# If still stuck, remove finalizers

kubectl patch pod mypod -p '{"metadata":{"finalizers":null}}'

```

17.3 Service Troubleshooting

Service Not Accessible:

```bash
# 1. Check service exists
kubectl get svc myservice

# 2. Check endpoints
kubectl get endpoints myservice

# If endpoints empty:
# - No pods match selector
# - Pods not Ready

# Check pod labels
kubectl get pods --show-labels

# Check if labels match service selector
kubectl get svc myservice -o yaml | grep selector

# 3. Test from within cluster
kubectl run -it --rm debug --image=busybox --restart=Never -- sh
wget -O- http://myservice:80

# 4. Check kube-proxy logs
kubectl logs -n kube-system -l k8s-app=kube-proxy

# 5. Check iptables rules
# On node:
sudo iptables-save | grep myservice
```
```

```

---

### **17.4 Network Troubleshooting**

**Pod-to-Pod Communication Failing**:

```bash
# Test from source pod
kubectl exec -it source-pod -- sh
ping <target-pod-ip>
wget -O- http://<target-pod-ip>:80

# If fails:
# 1. Check NetworkPolicies
kubectl get networkpolicy --all-namespaces

# Test with policy temporarily removed
kubectl delete networkpolicy blocking-policy

# 2. Check CNI plugin
kubectl get pods -n kube-system | grep -E 'calico|flannel|cilium'

# Check CNI logs
kubectl logs -n kube-system -l k8s-app=calico-node

# 3. Check routes on node
ssh worker-1
ip route

# 4. Test connectivity at different layers
# Layer 3 (IP):
ping <target-pod-ip>
```
```

```
# Layer 4 (TCP):
nc -zv <target-pod-ip> 80

# Layer 7 (HTTP):
curl http://<target-pod-ip>:80
```

DNS Not Working:

```bash
# Test DNS from pod
kubectl exec -it mypod -- sh
nslookup kubernetes.default
nslookup myservice.mynamespace.svc.cluster.local

# If fails:
# 1. Check CoreDNS pods
kubectl get pods -n kube-system -l k8s-app=kube-dns

# 2. Check CoreDNS logs
kubectl logs -n kube-system -l k8s-app=kube-dns

# 3. Check kube-dns service
kubectl get svc -n kube-system kube-dns

# Should have ClusterIP (usually 10.96.0.10)

# 4. Check pod's DNS config
kubectl exec mypod -- cat /etc/resolv.conf

# Should contain:
# nameserver 10.96.0.10
# search default.svc.cluster.local svc.cluster.local cluster.local
```

```
# 5. Test DNS directly
kubectl exec -it mypod -- sh
nslookup myservice 10.96.0.10 # Query CoreDNS directly

# 6. Check NetworkPolicy blocking DNS
# Ensure egress to kube-system:kube-dns is allowed
```

17.5 Storage Troubleshooting

PVC Stuck in Pending:

```bash
kubectl get pvc mypvc
# STATUS: Pending

kubectl describe pvc mypvc

# Events:
# Warning ProvisioningFailed no volume plugin matched
```

Common Causes:

1. StorageClass Doesn't Exist:

```bash
kubectl get storageclass

# Create default StorageClass
kubectl patch storageclass <name> -p '{"metadata":
{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
```

2. No Available PV (Static Provisioning):

```bash
kubectl get pv

# PVs available but not binding:
# - Access mode mismatch
# - Size too small
# - Selector mismatch

# Check PV details
kubectl describe pv mypv
```

3. Dynamic Provisioner Not Running:

```bash
# Check provisioner pod
kubectl get pods -n kube-system | grep provisioner

# Check logs
kubectl logs -n kube-system <provisioner-pod>
```

Pod Can't Mount Volume:

```bash
kubectl describe pod mypod

# Events:
# Warning FailedMount MountVolume.SetUp failed for volume "pvc-123" : rpc error: code = Internal desc = failed to mount volume

# Common causes:
# 1. Volume already mounted on another node (RWO)
```

```
# 2. Volume doesn't exist
# 3. Permission issues
# 4. CSI driver issues

# Check PV status
kubectl get pv

# Check node where pod is scheduled
kubectl get pod mypod -o wide

# Check volume on node
ssh <node>
mount | grep <volume-id>
```

Phase 18: Advanced kubectl Usage

18.1 kubectl Powerful Commands

Custom Columns:

```bash
# Show pod name, node, and IP
kubectl get pods -o custom-columns=NAME:.metadata.name,NODE:.spec.nodeName,IP:.status.podIP

# Show deployment with replicas
kubectl get deployments -o custom-columns=NAME:.metadata.name,DESIRED:.spec.replicas,CURRENT:.status.replicas,READY:.status.readyReplicas

# Show PVCs with capacity and status
kubectl get pvc -o custom-columns=NAME:.metadata.name,STATUS:.status.phase,CAPACITY:.status.capacity.storage
```

JSONPath:

```bash
# Get pod IPs
kubectl get pods -o jsonpath='{.items[*].status.podIP}'

# Get pod names and IPs
kubectl get pods -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.status.podIP}{"\n"}{end}'

# Get node capacity
kubectl get nodes -o jsonpath='{.items[*].status.capacity.cpu}'

# Get all container images
kubectl get pods -o jsonpath='{.items[*].spec.containers[*].image}' | tr ' ' '\n' | sort -u

# Get pod resource requests
kubectl get pods -o jsonpath='{range
.items[*]}{.metadata.name}{"\t"}{.spec.containers[*].resources.requests.memory}{"\n"}{end}'

# Complex: Get pods not running
kubectl get pods -o jsonpath='{range
.items[?(@.status.phase!="Running")]}{.metadata.name}{"\t"}{.status.phase}{"\n"}{end}'
```

Using jq (JSON processor):

```bash
# Install jq
apt-get install jq # Ubuntu
brew install jq # macOS

# Get pod names
kubectl get pods -o json | jq '.items[].metadata.name'

# Get pods with restart count > 0
```

```bash
kubectl get pods -o json | jq '.items[] | select(.status.containerStatuses[].restartCount > 0) | .metadata.name'

# Get pods using more than 100Mi memory
kubectl top pods -o json | jq '.items[] | select(.containers[].usage.memory > "100Mi") | .metadata.name'

# Get services with type LoadBalancer
kubectl get svc -o json | jq '.items[] | select(.spec.type=="LoadBalancer") | {name: .metadata.name, ip: .status.loadBalancer.ingress[0].ip}'

# Get all container images with their pull policies
kubectl get pods -o json | jq '.items[].spec.containers[] | {name: .name, image: .image, pullPolicy: .imagePullPolicy}'

# Get nodes with labels
kubectl get nodes -o json | jq '.items[] | {name: .metadata.name, labels: .metadata.labels}'
```

Field Selectors:

```bash
# Get pods on specific node
kubectl get pods --field-selector spec.nodeName=worker-1

# Get running pods
kubectl get pods --field-selector status.phase=Running

# Get services with specific type
kubectl get svc --field-selector spec.type=LoadBalancer

# Get failed pods
kubectl get pods --field-selector status.phase=Failed

# Combine multiple selectors
kubectl get pods --field-selector status.phase=Running,spec.nodeName=worker-1
```

```

---

**Label Selectors**:
```bash
# Equality-based
kubectl get pods -l app=nginx
kubectl get pods -l app=nginx,version=v1
kubectl get pods -l 'app!=nginx'

# Set-based
kubectl get pods -l 'environment in (production,staging)'
kubectl get pods -l 'tier notin (frontend,backend)'
kubectl get pods -l 'app,version' # Has both labels

# Show labels
kubectl get pods --show-labels

# Add label
kubectl label pod mypod env=production

# Remove label
kubectl label pod mypod env-

# Overwrite label
kubectl label pod mypod env=staging --overwrite
```

Sorting:
```bash
# Sort pods by creation time
kubectl get pods --sort-by=.metadata.creationTimestamp

```
# Sort by name
kubectl get pods --sort-by=.metadata.name

# Sort nodes by CPU capacity
kubectl get nodes --sort-by=.status.capacity.cpu

# Sort PVCs by capacity
kubectl get pvc --sort-by=.spec.resources.requests.storage
```

Watch Resources:

```bash
# Watch pods
kubectl get pods -w

# Watch specific pod
kubectl get pod mypod -w

# Watch events
kubectl get events -w

# Watch with custom columns
kubectl get pods -w -o
custom-columns=NAME:.metadata.name,STATUS:.status.phase,NODE:.spec.nodeName
```

Diff Before Apply:

```bash
# See what would change
kubectl diff -f deployment.yaml
```

```
# Output shows additions (+), deletions (-), and modifications (~)

# Safe apply workflow:
kubectl diff -f deployment.yaml
# Review changes
kubectl apply -f deployment.yaml
```

Dry Run:

```bash
# Client-side dry run (doesn't send to API server)
kubectl create deployment nginx --image=nginx --dry-run=client -o yaml

# Server-side dry run (validates with API server)
kubectl apply -f deployment.yaml --dry-run=server

# Use case: Generate YAML
kubectl create deployment nginx --image=nginx --dry-run=client -o yaml > deployment.yaml
```

Apply Strategically:

```bash
# Server-side apply (recommended for complex resources)
kubectl apply -f deployment.yaml --server-side

# Force apply (override conflicts)
kubectl apply -f deployment.yaml --force

# Prune unused resources
kubectl apply -f manifests/ --prune -l app=myapp
```

```
# Record change cause
kubectl apply -f deployment.yaml --record
kubectl rollout history deployment/myapp
```

18.2 kubectl Productivity Tricks

Aliases:

```bash
# Add to ~/.bashrc or ~/.zshrc
alias k='kubectl'
alias kg='kubectl get'
alias kd='kubectl describe'
alias kdel='kubectl delete'
alias kl='kubectl logs'
alias kex='kubectl exec -it'
alias kgp='kubectl get pods'
alias kgs='kubectl get svc'
alias kgn='kubectl get nodes'

# Context switching
alias kc='kubectl config use-context'
alias kcc='kubectl config current-context'

# Usage:
k get pods
kgp -o wide
kl mypod -f
```

Shell Completion:

```bash
# Bash
echo 'source <(kubectl completion bash)' >>~/.bashrc
echo 'alias k=kubectl' >>~/.bashrc
echo 'complete -F __start_kubectl k' >>~/.bashrc

# Zsh
echo 'source <(kubectl completion zsh)' >>~/.zshrc
echo 'alias k=kubectl' >>~/.zshrc
echo 'compdef __start_kubectl k' >>~/.zshrc

# Usage: Type 'kubectl get po' and press TAB
```

Kubectl Plugins (krew):

```bash
# Install krew
(
set -x; cd "$(mktemp -d)" &&
OS="$(uname | tr '[:upper:]' '[:lower:]')" &&
ARCH="$(uname -m | sed -e 's/x86_64/amd64/' -e 's/\(arm\)\(64\)\?.*/\1\2/' -e 's/aarch64$/arm64/')" &&
KREW="krew-${OS}_${ARCH}" &&
curl -fsSLO "https://github.com/kubernetes-sigs/krew/releases/latest/download/${KREW}.tar.gz" &&
tar zxvf "${KREW}.tar.gz" &&
./"${KREW}" install krew
)

# Add to PATH
export PATH="${KREW_ROOT:-$HOME/.krew}/bin:$PATH"

# Useful plugins
```

```
kubectl krew install ctx # Context switching
kubectl krew install ns # Namespace switching
kubectl krew install tree # Resource hierarchy
kubectl krew install images # List images
kubectl krew install whoami # Show user
kubectl krew install tail # Tail logs
kubectl krew install view-secret # Decode secrets

# Usage
kubectl ctx production # Switch context
kubectl ns default # Switch namespace
kubectl tree deployment myapp # Show deployment hierarchy
kubectl images # List all images in use
```

Phase 19: Helm - Package Manager

19.1 Helm Basics

What is Helm?
```
Package manager for Kubernetes
- Templates for Kubernetes manifests
- Version control for releases
- Dependency management
- Easy upgrades/rollbacks
```

Helm Concepts:
```
Chart:
- Package of Kubernetes resources
```

- Like apt/yum package

Repository:

- Collection of charts
- Like apt repository

Release:

- Instance of chart deployed to cluster
- Can install same chart multiple times

```

---

**Install Helm**:

```bash
# macOS
brew install helm

# Linux
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

# Verify
helm version
```

19.2 Using Helm Charts

Add Repository:

```bash
# Add official charts repo
helm repo add stable https://charts.helm.sh/stable

# Add Bitnami repo

```
helm repo add bitnami https://charts.bitnami.com/bitnami

# Update repos
helm repo update

# List repos
helm repo list

# Search charts
helm search repo nginx
helm search repo bitnami/mysql
```

Install Chart:

```bash
# Simple install
helm install my-nginx bitnami/nginx

# Install with custom values
helm install my-nginx bitnami/nginx \
--set replicaCount=3 \
--set service.type=LoadBalancer

# Install with values file
helm install my-nginx bitnami/nginx -f values.yaml

# Install in specific namespace
helm install my-nginx bitnami/nginx -n production --create-namespace

# Dry run (see what would be created)
helm install my-nginx bitnami/nginx --dry-run --debug
```

Example values.yaml:

```yaml
replicaCount: 3

image:
repository: nginx
tag: "1.25.3"
pullPolicy: IfNotPresent

service:
type: LoadBalancer
port: 80

resources:
limits:
cpu: 500m
memory: 512Mi
requests:
cpu: 250m
memory: 256Mi

ingress:
enabled: true
hosts:
- host: nginx.example.com
paths:
- path: /
pathType: Prefix
```

List Releases:

```bash
```

```
# List releases in current namespace
helm list

# List all releases in all namespaces
helm list --all-namespaces

# List releases in specific namespace
helm list -n production

# Output:
# NAME NAMESPACE REVISION UPDATED STATUS CHART APP VERSION
# my-nginx default 1 2024-01-15 10:30:00 UTC deployed nginx-15.0.0 1.25.3
```

Upgrade Release:

```bash
# Upgrade with new values
helm upgrade my-nginx bitnami/nginx --set replicaCount=5

# Upgrade with values file
helm upgrade my-nginx bitnami/nginx -f new-values.yaml

# Force upgrade (even if no changes)
helm upgrade my-nginx bitnami/nginx --force

# Install or upgrade (install if doesn't exist)
helm upgrade --install my-nginx bitnami/nginx

# Check what would change
helm upgrade my-nginx bitnami/nginx --dry-run --debug
```

Rollback Release:

```bash
# View history
helm history my-nginx

# Output:
# REVISION UPDATED STATUS CHART APP VERSION DESCRIPTION
# 1 Mon Jan 15 10:30:00 2024 superseded nginx-15.0.0 1.25.3 Install complete
# 2 Mon Jan 15 11:00:00 2024 deployed nginx-15.0.0 1.25.3 Upgrade complete

# Rollback to previous version
helm rollback my-nginx

# Rollback to specific revision
helm rollback my-nginx 1

# Dry run rollback
helm rollback my-nginx 1 --dry-run
```

Uninstall Release:

```bash
# Uninstall release
helm uninstall my-nginx

# Uninstall and keep history (can rollback)
helm uninstall my-nginx --keep-history
```

Get Release Info:

```bash
# Show deployed values
helm get values my-nginx

# Show all values (including defaults)
helm get values my-nginx --all

# Show manifest
helm get manifest my-nginx

# Show notes
helm get notes my-nginx

# Show status
helm status my-nginx
```

19.3 Creating Custom Charts

Create Chart:

```bash
helm create myapp

# Directory structure:
# myapp/
#   ■■■ Chart.yaml              # Chart metadata
#   ■■■ values.yaml             # Default values
#   ■■■ templates/              # Kubernetes manifests
#   ■   ■■■ deployment.yaml
#   ■   ■■■ service.yaml
#   ■   ■■■ ingress.yaml
#   ■   ■■■ _helpers.tpl        # Template helpers
#   ■   ■■■ NOTES.txt           # Post-install notes
```

```
        # ■■■ charts/                 # Chart dependencies
```

Chart.yaml:

```yaml
apiVersion: v2
name: myapp
description: A Helm chart for MyApp
type: application
version: 1.0.0 # Chart version
appVersion: "1.0" # Application version

maintainers:
- name: DevOps Team
email: devops@example.com

keywords:
- web
- application

dependencies:
- name: postgresql
version: 12.1.0
repository: https://charts.bitnami.com/bitnami
condition: postgresql.enabled
```

---

values.yaml:

```yaml
replicaCount: 2
```

```
image:
repository: myapp
tag: "1.0.0"
pullPolicy: IfNotPresent

service:
type: ClusterIP
port: 80

ingress:
enabled: false
className: nginx
hosts:
- host: myapp.example.com
paths:
- path: /
pathType: Prefix

resources:
limits:
cpu: 1000m
memory: 1Gi
requests:
cpu: 500m
memory: 512Mi

autoscaling:
enabled: false
minReplicas: 2
maxReplicas: 10
targetCPUUtilizationPercentage: 80

postgresql:
```

enabled: true

auth:

username: myapp

password: secret

database: myappdb

```

---

**templates/deployment.yaml**:

```yaml
apiVersion: apps/v1

kind: Deployment

metadata:

name: {{ include "myapp.fullname" . }}

labels:

{{- include "myapp.labels" . | nindent 4 }}

spec:

replicas: {{ .Values.replicaCount }}

selector:

matchLabels:

{{- include "myapp.selectorLabels" . | nindent 6 }}

template:

metadata:

labels:

{{- include "myapp.selectorLabels" . | nindent 8 }}

spec:

containers:

- name: {{ .Chart.Name }}

image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion }}"

imagePullPolicy: {{ .Values.image.pullPolicy }}

ports:

```
- name: http
containerPort: 8080
protocol: TCP
livenessProbe:
httpGet:
path: /health
port: http
readinessProbe:
httpGet:
path: /ready
port: http
resources:
{{- toYaml .Values.resources | nindent 10 }}
env:
{{- if .Values.postgresql.enabled }}
- name: DB_HOST
value: {{ include "myapp.fullname" . }}-postgresql
- name: DB_USER
value: {{ .Values.postgresql.auth.username }}
- name: DB_NAME
value: {{ .Values.postgresql.auth.database }}
{{- end }}
```

Template Functions:

```yaml
# Default values
{{ .Values.image.tag | default "latest" }}

# String manipulation
{{ .Values.service.name | upper }}
```

```
{{ .Values.service.name | lower }}
{{ .Values.service.name | quote }}

# Conditionals
{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
...
{{- end }}

# Loops
{{- range .Values.env }}
- name: {{ .name }}
value: {{ .value }}
{{- end }}

# Include templates
{{ include "myapp.labels" . }}

# Indent
{{- include "myapp.labels" . | nindent 4 }}

# Convert to YAML
{{- toYaml .Values.resources | nindent 10 }}
```

_helpers.tpl (template helpers):

```yaml
{{/*
Expand the name of the chart.
*/}}
{{- define "myapp.name" -}}
```

```
{{- default .Chart.Name .Values.nameOverride | trunc 63 | trimSuffix "-" }}
{{- end }}

{{/*
Create a default fully qualified app name.
*/}}
{{- define "myapp.fullname" -}}
{{- if .Values.fullnameOverride }}
{{- .Values.fullnameOverride | trunc 63 | trimSuffix "-" }}
{{- else }}
{{- $name := default .Chart.Name .Values.nameOverride }}
{{- if contains $name .Release.Name }}
{{- .Release.Name | trunc 63 | trimSuffix "-" }}
{{- else }}
{{- printf "%s-%s" .Release.Name $name | trunc 63 | trimSuffix "-" }}
{{- end }}
{{- end }}
{{- end }}

{{/*
Common labels
*/}}
{{- define "myapp.labels" -}}
helm.sh/chart: {{ include "myapp.chart" . }}
{{ include "myapp.selectorLabels" . }}
{{- if .Chart.AppVersion }}
app.kubernetes.io/version: {{ .Chart.AppVersion | quote }}
{{- end }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
{{- end }}

{{/*
Selector labels
```

```
*/}}
{{- define "myapp.selectorLabels" -}}
app.kubernetes.io/name: {{ include "myapp.name" . }}
app.kubernetes.io/instance: {{ .Release.Name }}
{{- end }}
```

Install Custom Chart:

```bash
# Lint chart (check for errors)
helm lint myapp/

# Package chart
helm package myapp/
# Creates myapp-1.0.0.tgz

# Install from local chart
helm install my-release ./myapp/

# Install with custom values
helm install my-release ./myapp/ -f custom-values.yaml

# Template (render templates without installing)
helm template my-release ./myapp/

# Template with debugging
helm template my-release ./myapp/ --debug
```

Chart Dependencies:

```bash
```

```
# Update dependencies
helm dependency update myapp/

# Creates myapp/charts/postgresql-12.1.0.tgz

# List dependencies
helm dependency list myapp/
```

This completes the comprehensive Kubernetes deep dive covering:

- Admission Controllers
- Image Security
- Cluster Maintenance
- Troubleshooting
- Advanced kubectl
- Helm

You now have an in-depth understanding of Kubernetes from fundamentals through advanced operations! Let me know if you need any clarification or want to dive deeper into any specific topic.